Задачи с собеседований по Си

Junior · Middle · Senior — с полным разбором решений

Учебник «Алгоритмы и игры на Си»

Содержание

Как готовиться к собеседованию по Си **1**
Как проходит секция кодинга . . . 1
Что оценивают, кроме правильного ответа . . . 1
Что спрашивают на каждом уровне . . . 2
План подготовки . . . 3
Подводные камни именно в Си . . . 6
Что говорить, если не знаешь . . . 8
Что дальше . . . 8

Приложение A. Задачи с собеседований: Junior **9**
Как пользоваться . . . 9
Часть 1. Задачи с готовыми решениями . . . 10
Задача 1. Поменять местами две переменные . . . 10
Задача 2. Развернуть строку на месте . . . 12
Задача 3. Проверка палиндрома . . . 14
Задача 4. Факториал итеративно . . . 16
Задача 5. Второй по величине элемент массива . . . 18
Задача 6. Подсчёт гласных в строке . . . 20
Задача 7. Проверка на простое число . . . 22
Задача 8. Цифры числа: сколько их и какая сумма . . . 24
Часть 2. Три классики без отдельного файла . . . 26
Задача 9. Проверить чётность числа . . . 26
Задача 10. Максимум и минимум массива за один проход . . . 28
Задача 11. FizzBuzz . . . 29
Часть 3. Каверзы: «что выведет / в чём ошибка» . . . 31
Каверза 1. `sizeof('a')` — сколько это? . . . 31
Каверза 2. Целочисленное деление `5 / 2` . . . 32
Каверза 3. `printf("%d", c)` для `char` . . . 33
Каверза 4. `char` — знаковый или беззнаковый? . . . 33
Каверза 5. = вместо == в условии . . . 34
Каверза 6. Лишняя `;` после `for(...)` . . . 35
Каверза 7. `printf("%d %d", i++, ++i)` . . . 36
Каверза 8. Неинициализированная переменная . . . 37
Каверза 9. Остаток от отрицательного числа . . . 38
Ошибки, на которых сыплются в этих задачах . . . 39
Тренировочные задачи . . . 39
Мини-резюме . . . 40
Куда дальше . . . 41

Приложение Б. Задачи с собеседований: Middle **42**
Как пользоваться этим приложением . . . 42
Часть 1. Задачи на код . . . 43
Задача 1. Свои `strlen`, `strcpy`, `strcmp` . . . 43
Задача 2. Развернуть односвязный список . . . 45
Задача 3. Два указателя: середина списка, цикл и его начало . . . 47
Задача 4. Биты: popcount и степень двойки . . . 52
Задача 5. Корректный бинарный поиск . . . 55
Задача 6. Слияние двух отсортированных массивов . . . 57
Задача 7. Удалить дубликаты из отсортированного массива на месте . . . 58
Задача 8. Число, встречающееся нечётное число раз (XOR) . . . 60
Задача 9. Поменять местами два числа через указатели . . . 61
Часть 2. Каверзы . . . 63
Каверза 1. `char *s = "abc"` против `char s[] = "abc"` . . . 63
Каверза 2. `sizeof` массива внутри функции . . . 65
Каверза 3. `arr[1]`, `1[arr]` и `*(arr + 1)` . . . 66
Каверза 4. `++*p`, `*p++`, `*++p`, `(*p)++` . . . 67
Каверза 5. `const char *p` против `char * const p` . . . 68
Каверза 6. Найди баг: утечка памяти . . . 69
Каверза 7. Найди баг: use-after-free и двойной free . . . 70
Каверза 8. Возврат адреса локальной переменной . . . 71
Ошибки, на которых сыплются в этих задачах . . . 73
Тренировочные задачи . . . 73
Ответы к тренировочным задачам . . . 74
Мини-резюме . . . 81
Что дальше . . . 82

Приложение В. Задачи с собеседований: Senior **83**
Часть 1. Задачи на код . . . 84
Задача 1. Быстрая сортировка (quicksort) . . . 84
Задача 2. Сортировка слиянием (merge sort) . . . 87
Задача 3. Обобщённый стек через `void *` . . . 90
Задача 4. Свои `atoi` и `itoa` с обработкой краёв . . . 93
Задача 5. Таблица обработчиков на указателях на функции . . . 97
Задача 6. Кольцевой буфер (ring buffer) . . . 99
Задача 7. Хеш-таблица «строка → счётчик» с цепочками . . . 102
Задача 8. Точка пересечения двух односвязных списков . . . 105
Задача 9. Набросок LRU-кэша . . . 107
Задача 10. Разворот числа с защитой от переполнения . . . 111
Часть 2. Каверзы и теория . . . 113
Каверза 1. Что такое UB и где оно прячется . . . 113
Каверза 2. Strict aliasing — почему `int *` и `float *` нельзя путать . . . 115
Каверза 3. `volatile` — зачем он нужен . . . 117
Каверза 4. Две роли `static` . . . 117
Каверза 5. Stack vs heap и причины segfault . . . 118
Каверза 6. Выравнивание и padding: что выведет `sizeof` . . . 119
Каверза 7. Опасный макрос `#define SQR(x) x*x` . . . 121
Каверза 8. Идиома `do { ... } while(0)` в макросах . . . 121

Каверза 9. Порядок вычисления аргументов не определён 122
Каверза 10. Endianness — порядок байтов 123
Ошибки, на которых сыплются в этих задачах 124
Тренировочные задачи 125
Ответы к тренировочным задачам 125
Итог практикума 141
Напутствие 141

Как готовиться к собеседованию по Си

Дальше идут три приложения с задачами, которые реально дают на собеседованиях C-разработчику: **А** — Junior, **Б** — Middle, **В** — Senior. Прежде чем в них нырять, стоит понять, как устроено само собеседование и что на нём оценивают. Иначе легко готовиться не к тому: вызубрить двадцать алгоритмов и провалиться на вопросе «а что, если массив пустой?».

Эта глава — карта местности. Как проходит секция кодинга, что спрашивают на каждом уровне, в каком порядке проходить задачи, как проверять себя компилятором и санитайзерами и что говорить, когда ответа не знаешь.

Как проходит секция кодинга

Формат почти везде один. Тебе дают задачу на 15–30 минут — вслух или текстом, в редакторе с подсветкой, в Google-документе, а иногда просто на бумаге или на доске. Компилятора обычно нет: код надо написать так, чтобы он собрался «в голове». Интервьюер сидит рядом (или в звонке) и смотрит, как ты работаешь.

Задачи маленькие. Никто не просит написать базу данных — просят развернуть строку, найти второй по величине элемент, развернуть односвязный список, посчитать единичные биты. Всё это есть в приложениях А и Б, и всё это ты умеешь после книги.

Дальше почти всегда идёт вторая часть, короткая: «каверзы». Кусок кода на пять строк и вопрос «что выведет?» или «в чём ошибка?». Их гоняют, потому что за две минуты они показывают то, что задача на код показывает за двадцать.

Что оценивают, кроме правильного ответа

Самое неочевидное для новичка: работающий код — это меньше половины оценки. Смотрят вот на что.

Говоришь ли ты вслух. Молчащий кандидат — чёрный ящик. Интервьюер не видит, думаешь ты или завис, и не может подсказать. Проговаривай: «так, мне нужно пройти строку до конца… длину я знаю… значит, два индекса с двух концов навстречу». Даже неверная мысль вслух лучше тишины.

Уточняешь ли условие. Хороший кандидат до кода задаёт два-три вопроса: «массив отсортирован?», «могут быть дубликаты?», «строку можно менять на месте или нужна копия?», «что вернуть, если элемента нет?». Это не придирки — это то, что в работе называется «разобраться в задаче».

Помнишь ли про краевые случаи. Пустой ввод, ноль, отрицательное число, массив из одного элемента, все элементы равны, переполнение. Идеально — назвать их **до** того, как спросят: «на пустом массиве верну ноль и признак ошибки, потому что максимума тут нет».

Проверяешь ли ввод. `scanf` возвращает число прочитанных значений, `malloc` возвращает `NULL`, `fopen` возвращает `NULL`. Кандидат, который пишет `if (malloc вернул NULL)`, сразу выглядит взрослее.

Помнишь ли про память. Выделил через `malloc` — скажи вслух, где будет `free` и кто за него отвечает. Это главный маркер «человек понимает Си», а не «пишет на Си как на Python».

Как реагируешь на замечание. Тебе покажут ошибку — иногда настоящую, иногда чтобы посмотреть, будешь ли спорить. Правильная реакция: «сейчас проверю… да, действительно, на `n = 1` цикл не выполнится». Спорить, будучи неправым, — худшее, что можно сделать.

Как это звучит на практике, на задаче «найти второй по величине элемент» (приложение А, задача 5):

```
Я:  Уточню: массив может быть пустым? Дубликаты возможны?
Он: Дубликаты возможны, массив непустой.
Я:  Тогда край — массив из одного элемента: второго максимума нет,
    я верну признак ошибки. И дубликаты: в {5, 5, 3} второй по
    величине — это 3 или 5?
Он: Различные значения. Значит, 3.
Я:  Хорошо. Один проход, две переменные: максимум и второй максимум.
    Начальные значения возьму INT_MIN, чтобы не гадать...
```

Половина этого текста — не код. Именно она и решает.

Что спрашивают на каждом уровне

Уровни условны, планка у всех своя, но логика такая.

Junior (приложение А). Проверяют, что ты вообще пишешь код. Синтаксис, циклы, массивы, строки, простые алгоритмы: поменять переменные местами (задача 1), развернуть строку на месте (задача 2), палиндром (задача 3), факториал (задача 4), гласные в строке (задача 6), простое ли число (задача 7), FizzBuzz (задача 11). Плюс каверзы на понимание основ: `sizeof('a')` (каверза 1), `5 / 2` (каверза 2), = вместо == (каверза 5). Блестящих алгоритмов не ждут — ждут аккуратности и того, что ты понимаешь каждую свою строку.

Middle (приложение Б). Главная тема — **указатели и память**. Свои `strlen`, `strcpy`, `strcmp` (задача 1), развернуть односвязный список (задача 2), два указателя разной скорости — середина списка и цикл по Флойду (задача 3), биты: popcount и степень двойки (задача 4), бинарный поиск без ошибки на единицу (задача 5). Каверзы злее и почти все про память: `char *s = "abc"` против `char s[] = "abc"` (каверза 1), `sizeof` массива внутри функции (каверза 2), утечка (каверза 6), use-after-free и двойной `free` (каверза 7), возврат адреса локальной переменной (каверза 8). Проверяют не знание рецептов, а понимание, **что происходит в памяти**.

Senior (приложение В). Тонкости языка плюс инженерное мышление. Быстрая сортировка и сортировка слиянием (задачи 1–2), обобщённые структуры через `void *` (задача 3), таблица обработчиков на указателях на функции (задача 5), кольцевой буфер (задача 6), хеш-таблица с цепочками (задача 7), LRU-кэш (задача 9). Из теории — неопределённое поведение (каверза 1), strict aliasing (каверза 2), `volatile` (каверза 3), две роли `static` (каверза 4), выравнивание и padding (каверза 6), опасные макросы (каверзы 7–8). Здесь мало решить: надо обсудить компромиссы — сложность, память, худший случай, что сломается в продакшене.

План подготовки

Порядок — строго снизу вверх, даже если метишь в Middle или Senior. Приложение А проходится за пару вечеров и освежает базу, а его каверзы всплывают на собеседованиях любого уровня: `sizeof('a')` спрашивают и у сеньоров. Пропустить А и сесть за В — верный способ застрять на пустяке.

Разумный темп — 3–5 задач за подход, не больше. После пятой задачи подряд голова перестаёт замечать краевые случаи, а именно они и нужны. Приложение А — два-три вечера, Б — неделя, В — две-три недели, если разбираться, а не пролистывать.

Решай сам, до решения. Прочитал условие — закрой книгу. Выпиши на бумаге пример руками: вход, ожидаемый выход, один краевой случай. Только потом код. Даже провальная попытка включает голову так, как чтение готового ответа не включит никогда. Подсмотренное решение забывается к утру, выстраданное — остаётся.

Проверяй себя компилятором. Написал — собери, даже если уверен:

```
cc -std=c11 -Wall -Wextra -O2 solution.c -o solution
```

`-Wall -Wextra` ловят половину каверз из приложений сами: и `=` вместо `==` в условии, и лишнюю `;` после `for`, и неинициализированную переменную. Если хоть одно предупреждение — код не готов.

Гоняй санитайзеры. Компилятор видит не всё: выход за границу массива он пропустит. Смотри, что бывает. Вот классическая ошибка на единицу в поиске максимума — файл `maxbug.c`:

```c
#include <stdio.h>
#include <stdlib.h>

int max_of(const int *a, int n)
{
    int m = a[0];
    for (int i = 1; i <= n; i++)      /* баг: <= вместо < */
        if (a[i] > m)
            m = a[i];
    return m;
}

int main(void)
{
    int *a = malloc(4 * sizeof(int));
    if (a == NULL)
        return 1;

    a[0] = 3; a[1] = 9; a[2] = 1; a[3] = 7;
    printf("max = %d\n", max_of(a, 4));

    free(a);
    return 0;
}
```

В массиве четыре элемента, значит индексы 0–3. Цикл с `i <= n` доходит до `a[4]` — читает то, чего нет. Обычная сборка молчит, а программа врёт:

```
$ cc -std=c11 -Wall -Wextra -O2 maxbug.c -o maxbug
$ ./maxbug
max = 2147483647
```

Ни одного предупреждения — и мусор в ответе: 2147483647 — это `INT_MAX`, в массиве такого числа нет. Чтение `a[4]` — неопределённое поведение (UB): стандарт ничего не обещает, и оптимизатор с `-O2` вправе рассуждать так, будто этого чтения не бывает. Собери тот же файл без оптимизации (`-O0`) — и на этой машине он напечатает `max = 9`, верный ответ. Это самый опасный исход: баг есть, а тест проходит. Теперь пересоберём с **санитайзерами**: `-fsanitize=address` подставляет проверки на каждое обращение к памяти, `-fsanitize=undefined` — на неопределённое поведение. `-g` нужен, чтобы в отчёте были номера строк.

```
$ cc -std=c11 -Wall -Wextra -g -fsanitize=address,undefined \
     maxbug.c -o maxbug-san
$ ./maxbug-san
=================================================================
==72744==ERROR: AddressSanitizer: heap-buffer-overflow on address
    0x6020000000e0 at pc 0x000102776989 bp 0x7ff7bd7887e0
    sp 0x7ff7bd7887d8
READ of size 4 at 0x6020000000e0 thread T0
    #0 0x000102776988 in max_of maxbug.c:8
    #1 0x000102776db5 in main maxbug.c:20
    #2 0x7ff804aa4b27 in start+0xca7 (dyld:x86_64+0x12b27)

0x6020000000e0 is located 0 bytes after 16-byte region
    [0x6020000000d0,0x6020000000e0)
allocated by thread T0 here:
    #0 0x000102cd4792 in malloc+0x82
       (libclang_rt.asan_osx_dynamic.dylib:x86_64h+0xa6792)
    #1 0x000102776ad3 in main maxbug.c:15
    #2 0x7ff804aa4b27 in start+0xca7 (dyld:x86_64+0x12b27)

SUMMARY: AddressSanitizer: heap-buffer-overflow maxbug.c:8 in max_of
```

(Адреса и номер процесса у тебя будут свои. Длинные строки отчёта я перенёс с отступом, чтобы влезли в страницу, — в терминале каждая из них одна. Дальше ASan печатает ещё карту теневой памяти — она нужна редко, здесь я её не привожу.)

Читается это так: **heap-buffer-overflow** — вышли за границу блока, выданного `malloc`. `max_of maxbug.c:8` — точное место, восьмая строка, `if (a[i] > m)`. «0 bytes after 16-byte region» — прочитали ровно за концом блока в 16 байт, то есть `a[4]` в массиве из четырёх `int`. Ниже — кто этот блок выделил: `main maxbug.c:15`, наш `malloc`. Ошибка на единицу найдена за секунду.

Переполнение знакового `int` ловит второй санитайзер. Факториал, посчитанный в `int`, лопается уже на 13 — вот `ovf.c`:

```c
#include <stdio.h>

int factorial(int n)
{
    int f = 1;
    for (int i = 2; i <= n; i++)
        f *= i;
    return f;
}

int main(void)
{
    printf("13! = %d\n", factorial(13));
    return 0;
}
```

```
$ cc -std=c11 -Wall -Wextra -g -fsanitize=undefined ovf.c -o ovf-san
$ ./ovf-san
ovf.c:7:11: runtime error: signed integer overflow: 479001600 * 13
    cannot be represented in type 'int'
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior ovf.c:7:11
13! = 1932053504
```

Строка 7 — это `f *= i`, и санитайзер прямо говорит, что 479 001 600 (это 12!) на 13 в `int` не помещается. Программа при этом даже не упала — просто напечатала неправду (настоящее 13! = 6 227 020 800). Именно поэтому в приложении A, задача 4, накопитель объявлен как `long long`. Без санитайзера ты бы этого не заметил, а на собеседовании спросят обязательно.

Правило простое: любое своё решение с указателями или `malloc` прогоняй через `-fsanitize=address,undefin` Если санитайзеры молчат — код можно показывать.

Подводные камни именно в Си

Список того, на чём валятся чаще всего. Рядом — где это разобрано, там и тренируйся.

- **Неопределённое поведение (UB).** Компилятор вправе сделать что угодно, включая «работает у меня на машине». Источники: выход за границы, чтение неинициализированной переменной, изменение строкового литерала, переполнение знакового, два изменения одной переменной в одном выражении. → приложение B, каверза 1; приложение A, каверзы 7 (`i++` и `++i` в одном `printf`) и 8 (неинициализированная переменная).

- **Переполнение знакового `int`.** Границы 32-битного `int` — от −2 147 483 648 до 2 147 483 647, примерно ±2 миллиарда (`INT_MIN`, `INT_MAX` из `<limits.h>`). Проверять надо **до** операции, а не после: `if (x > INT_MAX / 10)`. → приложение А, задача 4 (факториал); приложение В, задача 10 (разворот числа с защитой от переполнения).
- **Знаковость `char`.** Стандарт не говорит, знаковый `char` или беззнаковый, — это решает компилятор: на x86-64 под macOS знаковый, на ARM под Linux беззнаковый. Отсюда беда: байт из UTF-8-строки становится отрицательным, а функции из `<ctype.h>` (`isalpha`, `tolower`) требуют значение либо `EOF`, либо представимое как `unsigned char`. Отсюда приведение `(unsigned char)` перед каждым вызовом. → приложение А, каверзы 3 и 4.
- **`sizeof` массива против указателя.** В своей функции `sizeof arr` — размер массива, а в вызванной функции параметр `int arr[]` — это уже указатель, и `sizeof` даст размер указателя — 8 на 64-битной системе. Длину массива всегда передают отдельным параметром. → приложение Б, каверза 2.
- **Приоритет и порядок операторов.** `*p++` — это не `(*p)++`; `a & 1 == 0` — это не то, что ты думаешь, потому что `==` сильнее `&`; макрос `#define SQR(x) x*x` на `SQR(1+2)` даёт 5. Не помнишь приоритет — ставь скобки. → приложение Б, каверза 4; приложение В, каверзы 7 и 8, а про то, что порядок вычисления аргументов вообще не определён, — каверза 9.
- **Время жизни переменных и `static`.** Локальная переменная умирает на `}`, и возвращать её адрес нельзя. У `static` две совершенно разные роли: внутри функции — «живёт до конца программы», на уровне файла — «не видна из других файлов». → приложение Б, каверза 8; приложение В, каверза 4.
- **Память: утечки, use-after-free, двойной `free`.** На каждый `malloc` ровно один `free`, после `free` присваивай указателю `NULL`, никогда не освобождай дважды. → приложение Б, каверзы 6 и 7.
- **Строки.** `char *s = "abc"` — указатель на неизменяемый литерал, писать в него нельзя; `char s[] = "abc"` — своя копия в массиве, менять можно. `==` сравнивает адреса, а не содержимое, для содержимого — `strcmp`. Место под `'\0'` считать обязательно. → приложение Б, каверза 1.
- **Выравнивание и padding.** `sizeof` структуры часто больше суммы размеров полей: компилятор вставляет пустые байты ради выравнивания. Порядок полей меняет размер структуры. → приложение В, каверза 6.
- **Целочисленное деление и остаток.** `5 / 2` — это 2, а `-7 % 2` — это −1, а не 1: в Си знак остатка совпадает со знаком делимого, это не математический остаток, который всегда от 0 до делителя. Поэтому проверка `n % 2 == 1` для отрицательных ломается; пиши `n % 2 != 0`. → приложение А, каверзы 2 и 9.

Что говорить, если не знаешь

Не знать — нормально, никто не держит в голове весь стандарт. Плохо не это, а враньё и молчание. Работает такой порядок.

Скажи честно и сразу. «Точно не помню» — нормальный ответ, он экономит время обоим. «Наверное, оно как-то так работает» — плохой: интервьюер видит, что ты угадываешь.

Покажи, что рядом ты знаешь. «Про strict aliasing я знаю только, что нельзя читать `int` через `float *`, а почему именно — не разбирался. Зато точно знаю, что безопасный способ — `memcpy`». Это ответ уровня «человек в теме, но конкретная деталь не изучена».

Рассуждай вслух от того, что знаешь. «`volatile` я не применял. Но раз компилятор оптимизирует чтения переменной, а тут значение может поменяться снаружи, — видимо, `volatile` запрещает такую оптимизацию. Правильно?» Ход мысли ценится почти так же, как ответ.

Спроси и запиши. «Расскажете, как правильно?» — совершенно нормальная просьба, и она превращает провал в маленький плюс: человек учится прямо на собеседовании.

Не притворяйся, что решал. Ложь про опыт вскрывается одним уточняющим вопросом, и после этого не верят уже ничему.

И то же самое про задачу, которая не идёт. Скажи, где застрял: «Я вижу решение за O(n^2) — два вложенных цикла. Чувствую, что можно за O(n), но пока не вижу как. Напишу сначала квадратичное?» Почти всегда интервьюер скажет «пиши» или подкинет подсказку. Рабочее медленное решение сильно лучше пустого экрана.

Что дальше

Дальше — три приложения по уровням. Проходи их подряд: **А**, потом **Б**, потом **В**. У каждой задачи есть условие, идея, решение и блок «что спросят дальше» — те самые дожимающие вопросы, к которым и надо готовиться. Файлы с рабочими решениями лежат в `code/19-*.c`, `code/20-*.c` и `code/21-*.c`; собирай их так же, как всё в книге:

```
cc -std=c11 -Wall -Wextra -O2 code/20-05-binsearch.c -o binsearch
./binsearch
```

Не пытайся выучить всё за ночь. Начинай с приложения А — даже если метишь выше.

Приложение A. Задачи с собеседований: Junior

Книга закончилась, а собеседования только начинаются. Это приложение — сборник задач, которые реально дают на собеседовании C-разработчику уровня **Junior** (младший, начинающий). Здесь нет ничего, чего ты не проходил: всё опирается на главы 1–18 — числа, массивы, строки, указатели, циклы, функции. Новой теории не будет. Будет другое — взгляд на знакомые приёмы глазами интервьюера.

На собесе мало написать работающий код. Спрашивающий смотрит, **как ты думаешь**: учёл ли краевые случаи, помнишь ли про переполнение, понимаешь ли, что делает каждая строка. Поэтому к каждой задаче я добавил блок «что спросят дальше» — те самые уточняющие вопросы, которыми тебя будут «дожимать».

Как пользоваться

Каждая задача оформлена так:

- **Условие** — как её формулируют вслух на собесе.
- **Идея** — как к ней подступиться, без кода.
- **Решение** — рабочий код.
- **Разбор** — почему так, а не иначе, какая сложность, где края.
- **Что спросят дальше** — продолжения и подвохи.

Главный совет: **сначала реши сам**, на бумаге или в редакторе, и только потом смотри решение. Подсмотренное решение забывается к утру. Решённое самому — остаётся. Даже если решил криво — сравни с моим вариантом, и разница тебе многое объяснит.

Приложение делится на три части:

1. **Задачи 1–8** — у каждой есть готовый файл `code/19-0N-*.c`, номер задачи совпадает с номером файла. Собирается и запускается так:

```
cc -std=c11 -Wall -Wextra -O2 code/19-07-prime.c -o prime
./prime
```

2. **Задачи 9–11** — три классики, которые решаются в несколько строк; отдельных файлов под них нет, весь код прямо в тексте.
3. **Каверзы** — короткие фрагменты «что выведет / в чём ошибка». Многие из них намеренно содержат ошибку или неопределённое поведение, поэтому в `code/` они не вынесены: в файлах книги лежит только код, который **можно** запускать.

Все выводы программ в этом приложении получены реальным запуском на Apple clang 21 (x86-64, macOS). Где результат зависит от платформы или от компилятора, я это отдельно оговариваю.

Часть 1. Задачи с готовыми решениями

Задача 1. Поменять местами две переменные

Условие. Есть две переменные `a` и `b`. Поменяй их значения местами. После обмена в `a` должно лежать то, что было в `b`, и наоборот.

Идея. Наивная попытка «в лоб» не работает:

```
a = b;   /* старое значение a потеряно навсегда */
b = a;   /* сюда попадёт b, а не старое a */
```

Первая же строка затирает `a`. Нужна **временная переменная**, чтобы запомнить значение перед затиранием.

Решение (`code/19-01-swap.c`):

```
#include <stdio.h>

int main(void)
{
    int a = 5;
    int b = 9;

    printf("До:    a = %d, b = %d\n", a, b);

    int temp = a;   /* запомнили a */
```

```
    a = b;          /* в a положили b */
    b = temp;       /* в b положили старое a */

    printf("После: a = %d, b = %d\n", a, b);
    return 0;
}
```

Запуск:

```
До:    a = 5, b = 9
После: a = 9, b = 5
```

Разбор. `temp` хранит копию `a`, пока мы перезаписываем `a` значением `b`. Потом достаём из `temp` старое `a` и кладём в `b`. Три присваивания, ни одного условия, ни одного цикла — O(1) по времени и O(1) по памяти (одна лишняя переменная). Работает для любых значений, включая `INT_MIN` и `INT_MAX`: мы ничего не складываем и не вычитаем, переполниться нечему.

Что спросят дальше. «А можно без временной переменной?» Да, два способа.

Через арифметику:

```
a = a + b;
b = a - b;   /* = старое a */
a = a - b;   /* = старое b */
```

Через XOR (исключающее «или», оператор `^`):

```
a = a ^ b;
b = a ^ b;
a = a ^ b;
```

Проверим XOR на наших числах. 5 в двоичном виде — `0101`, 9 — `1001`. `^` ставит 1 там, где биты **разные**:

```
a = a ^ b:  0101 ^ 1001 = 1100   (a = 12, «смесь» a и b)
b = a ^ b:  1100 ^ 1001 = 0101   (b = 5  — старое a)
a = a ^ b:  1100 ^ 0101 = 1001   (a = 9  — старое b)
```

Работает, потому что `x ^ y ^ y == x`: повторный XOR с тем же числом «снимает» его обратно. Арифметика устроена так же: в `a` лежит сумма 14, `14 - 9 = 5`, `14 - 5 = 9`. Оба фрагмента я собрал и запустил: `a = 9, b = 5`.

Оба выглядят эффектно, и интервьюер любит про них спросить. Но честный ответ — **в реальном коде так делать не надо**. Почему:

- Арифметический способ **переполняется**: если `a + b` вылезет за пределы `int`, это не «завернётся в минус», а неопределённое поведение — переполнение знакового типа стандарт не описывает вообще (см. главу 4 про переполнение; подробный разбор со всеми источниками UB — в Приложении В, каверза 1). Пример: `a = INT_MAX`, `b = 1` — санитайзер `-fsanitize=undefined` (см. главу «Как готовиться к собеседованию») ловит здесь три переполнения подряд. С `unsigned` этот трюк как раз корректен: беззнаковая арифметика по стандарту идёт по модулю 2^N и не переполняется.
- Оба трюка ломаются, если `a` и `b` — **одна и та же переменная** (один адрес). Так бывает в функции обмена через указатели: `swap(&x, &x)`, например когда сортировка меняет элемент сам с собой. XOR: `*a ^= *b` превращает `x` в `x ^ x`, то есть в 0, и дальше из нуля старое значение уже не достать. Сложение: `*a = x + x`, потом `*b = 2x - 2x` = 0 — тоже ноль. Я проверил на `x = 42`: оба варианта дают 0, а обмен через `temp` оставляет 42. Одинаковые **значения** в разных переменных (`a = 42`, `b = 42`) при этом не страшны — ломает именно общий адрес.
- XOR работает только с целыми: для `double` строка `a = a ^ b` просто не соберётся (`invalid operands to binary expression`).
- Оба варианта **нечитаемы** и не быстрее: современный компилятор и так оптимально реализует обмен через `temp` — на уровне ассемблера это часто просто две пересылки через регистры, а иногда компилятор вообще обходится без обмена, запомнив, в каком регистре что лежит.

Правильный вывод, который хочет услышать грамотный интервьюер: «знаю про XOR-трюк, но в продакшене пишу через `temp` — понятнее и без подвохов».

Второе продолжение: «сделай так, чтобы обмен работал из функции». Это уже задача про указатели, она разобрана в Приложении Б (задача 9).

Задача 2. Развернуть строку на месте

Условие. Дана строка. Переверни её **на месте** — без создания второго массива.

Идея. Два индекса: один на начало (`left`), другой на конец (`right`). Меняем символы местами и идём навстречу, пока они не встретятся посередине.

Проверим руками на слове `abcde` (пять символов, индексы 0…4):

```
шаг 0:  a b c d e     left=0, right=4  →  меняем a и e
шаг 1:  e b c d a     left=1, right=3  →  меняем b и d
шаг 2:  e d c b a     left=2, right=2  →  left < right ложно, стоп
```

Средний символ `c` остался на месте — так и должно быть.

Решение (`code/19-02-reverse-string.c`):

```c
#include <stdio.h>
#include <string.h>

/* Переворачивает строку на месте, без второго массива. */
void reverse(char *s)
{
    int left = 0;
    int right = (int)strlen(s) - 1;
    while (left < right) {
        char tmp = s[left];
        s[left] = s[right];
        s[right] = tmp;
        left++;
        right--;
    }
}

int main(void)
{
    char text[] = "Hello, world";
    reverse(text);
    printf("%s\n", text);
    return 0;
}
```

Запуск:

```
dlrow ,olleH
```

Разбор. `strlen(s)` — длина строки без учёта завершающего нуля `'\0'` (см. главу 10). Возвращает она `size_t` — беззнаковый тип; мы приводим результат к `int`, потому что `right` должен уметь стать отрицательным. Если бы `right` был `size_t`, то для **пустой** строки `strlen(s) - 1` дало бы не −1, а огромное число (беззнаковое вычитание заворачивается), и цикл полез бы за границы массива. Это типичная ловушка со знаковостью, о ней стоит сказать вслух.

Внутри цикла — обычный обмен через `tmp` (как в задаче 1). Условие `left < right` останавливает нас на середине: средний символ при нечётной длине трогать не нужно.

Сложность. O(n) по времени: `strlen` один раз проходит строку целиком, а потом делаем $\lfloor n/2 \rfloor$ обменов (n/2 с округлением вниз: для `abcde` при n = 5 — два обмена). O(1) по дополнительной памяти — новый буфер не выделяли.

Краевые случаи. Пустая строка "": `strlen` даёт 0, `right` = −1, условие `0 < -1` ложно, цикл не выполняется — правильно. Строка из одного символа: `left = right = 0`, тоже ноль итераций. Строка из двух: ровно один обмен.

Что спросят дальше.

- «Почему `char text[] = "..."`, а не `char *text = "..."`?» Менять строковый литерал стандарт запрещает — это неопределённое поведение, а на практике литералы обычно лежат в памяти только для чтения, и программа падает. Массив `char text[]` — твоя изменяемая копия литерала, её менять можно. Подробный разбор с двумя разными сборками одной программы — в Приложении Б, каверза 1.
- «А если в строке кириллица?» Развернётся мусор: в UTF-8 русская буква занимает два байта, и посимвольный разворот перемешает половинки букв. Наш `reverse` разворачивает **байты**, а не буквы. Для UTF-8 нужен разбор по символам — это честный и правильный ответ.
- «Сделай без `strlen`.» Найди конец сам: `int right = 0;`, потом `while (s[right] != '\0') right++;` и `right--`. Ровно то же самое, что делает `strlen`.

Задача 3. Проверка палиндрома

Условие. Палиндром читается одинаково слева направо и справа налево (`level`, `radar`). Проверь, палиндром ли данная строка.

Идея. Та же пара индексов с краёв, но теперь не меняем символы, а **сравниваем**. Нашли несовпадение — сразу не палиндром, дальше можно не смотреть.

Решение (`code/19-03-palindrome.c`):

```c
#include <stdio.h>
#include <string.h>

/* Возвращает 1, если строка — палиндром, иначе 0. */
int is_palindrome(const char *s)
{
    int left = 0;
    int right = (int)strlen(s) - 1;
    while (left < right) {
        if (s[left] != s[right]) {
            return 0;   /* нашли несовпадение — это не палиндром */
        }
        left++;
        right--;
```

```
    }
    return 1;
}

int main(void)
{
    const char *words[] = {"level", "radar", "hello", "noon"};
    for (int i = 0; i < 4; i++) {
        if (is_palindrome(words[i])) {
            printf("%s — палиндром\n", words[i]);
        } else {
            printf("%s — нет\n", words[i]);
        }
    }
    return 0;
}
```

Запуск:

```
level — палиндром
radar — палиндром
hello — нет
noon — палиндром
```

Разбор. Параметр `const char *s` — обещание «эту строку я менять не буду, только читаю». `words` — массив указателей на строковые литералы, и объявлен он как `const char *words[]`: менять литералы нельзя, и мы честно фиксируем это в типе.

Отсюда следствие, которое любят проверять. Убери `const` у параметра — и вызов `is_palindrome(words[i])` перестанет собираться молча:

```
pal.c:23:27: warning: passing 'const char *' to parameter of type
'char *' discards qualifiers
[-Wincompatible-pointer-types-discards-qualifiers]
   23 |         if (is_palindrome(words[i])) {
      |                           ^~~~~~~~
pal.c:5:25: note: passing argument to parameter 's' here
    5 | int is_palindrome(char *s)
      |                         ^
1 warning generated.
```

(Длинную первую строку я перенёс, чтобы она влезла в страницу; имя файла у тебя будет своё.)

Правило одностороннее: `char *` передать туда, где ждут `const char *`, можно, а обратно — нет. Поэтому `const` на входном параметре тебя не ограничивает, а наоборот — расширяет круг того, что функции можно скормить.

`return 0` при первом же несовпадении — это **ранний выход**: нет смысла проверять дальше, ответ уже ясен.

Сложность. Худший случай (строка действительно палиндром) — ⌊n/2⌋ сравнений, то есть O(n). Лучший случай — одно сравнение, если первый и последний символы разные. Плюс один проход `strlen` в начале — он тоже O(n). Памяти O(1).

Краевые случаи. Пустая строка и строка из одного символа считаются палиндромами — цикл не выполняется, возвращается 1. Это общепринятая трактовка, но на собесе её лучше проговорить вслух: «пустую строку считаю палиндромом, если надо иначе — скажи».

Что спросят дальше.

- «Сделай регистронезависимо: `Level` тоже палиндром.» Сравнивай не `s[left]` и `s[right]`, а их версии в одном регистре (приём приведения к нижнему регистру — в задаче 6 про гласные).
- «А если игнорировать пробелы и знаки: `A man a plan a canal Panama`?» Тогда индексы нужно «перепрыгивать» через не-буквы:

  ```c
  while (left < right && !isalpha((unsigned char)s[left]))  left++;
  while (left < right && !isalpha((unsigned char)s[right])) right--;
  ```

  Эти две строки ставятся в начало тела цикла, перед сравнением; `isalpha` («это буква?») живёт в `<ctype.h>`. Приведение к `unsigned char` тут обязательно — почему, разбираем в каверзе 4.
- «Проверь палиндром рекурсивно.» Функция сравнивает края и вызывает себя для внутренней части. Работает, но глубина рекурсии — около n/2 вызовов, то есть O(n) памяти стека (см. главу 9), — итеративно лучше.

Задача 4. Факториал итеративно

Условие. Посчитай факториал числа `n` циклом (без рекурсии). `n! = 1·2·…·n`, при этом `0! = 1`.

Идея. Накопитель `fact = 1`, домножаем на 2, 3, …, n. Стартовое значение — **единица**, а не ноль (умножение на 0 всё обнулит). И сразу помним про переполнение из главы 4.

Решение (`code/19-04-factorial.c`):

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Факториал какого числа? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        printf("Факториал определён только для n >= 0.\n");
        return 1;
    }
    if (n > 20) {
        printf("Слишком большое: long long переполнится при n > 20.\n");
        return 1;
    }

    long long fact = 1;   /* 0! = 1 и 1! = 1 — стартуем с единицы */
    for (int i = 2; i <= n; i++) {
        fact = fact * i;
    }

    printf("%d! = %lld\n", n, fact);
    return 0;
}
```

Запуск:

```
Факториал какого числа? 13
13! = 6227020800
```

Разбор. Тип накопителя — `long long` (на этой машине 8 байт), потому что `int` лопается уже на 13!: 6 227 020 800 больше, чем `INT_MAX` = 2 147 483 647. Печатаем через `%lld` — свой спецификатор для `long long`; напишешь `%d`, и `printf` возьмёт из аргументов не то количество байт, получится мусор (а компилятор с `-Wall` заругается).

Цикл идёт с 2, потому что умножать на 1 бессмысленно. Для `n = 0` и `n = 1` он не выполнится ни разу, и `fact` останется 1 — верный ответ без всякого отдельного `if`.

`scanf("%d", &n)` возвращает **количество успешно прочитанных значений**. Мы сравниваем его с 1: если пользователь ввёл `abc`, `scanf` вернёт 0, и мы честно скажем «это не число», а не станем считать факториал мусора. Пропущенная проверка `scanf` — одна из самых частых претензий на собеседовании.

Сложность. O(n) умножений, O(1) памяти. Для `n ≤ 20` это не больше девятнадцати умножений (от 2 до 20).

Что спросят дальше.

- «До какого `n` это корректно?» До 20: 20! = 2 432 902 008 176 640 000 ещё помещается в `long long` (предел 9 223 372 036 854 775 807), а 21! — уже нет. Поэтому мы заранее отсекаем `n > 20`. Грамотный кандидат **сам** проговаривает границу типа, не дожидаясь вопроса.
- «А если бы отсечки не было?» Получили бы переполнение знакового типа — это UB, а не «просто неверное число». Программа имеет право напечатать что угодно.
- «А если ввести 2147483648?» Проверь сам: программа ответит «Факториал определён только для n >= 0.». Число не влезло в `int`, а для `scanf` с `%d` такой ввод — неопределённое поведение (стандарт C11, раздел 7.21.6.2); на этой машине в `n` оказалось отрицательное число. Проверка «`scanf` вернул 1» от этого не спасает. Надёжный способ — читать строку и разбирать её самому с контролем переполнения, как в Приложении В, задача 4.
- «А рекурсией?» Можно (см. главу 9), но итеративно — без риска переполнить стек вызовов на больших `n` и обычно быстрее.
- «А факториал 100?» В `long long` не влезет никогда. Нужна «длинная арифметика»: хранить число как массив цифр и умножать столбиком.

Задача 5. Второй по величине элемент массива

Условие. Найди второй по величине элемент массива за **один** проход (не сортируя).

Идея. Держим две переменные: `max` (наибольший) и `second` (второй). Для каждого числа: если оно больше `max` — старый `max` становится `second`, а само число новым `max`. Иначе, если оно меньше `max`, но больше `second` — обновляем `second`. Дубликаты максимума вторым считать не должны.

Решение (`code/19-05-secondmax.c`):

```c
#include <stdio.h>
#include <limits.h>

int main(void)
```

```c
{
    int a[] = {3, 9, 9, 1, 7, 9, 4};
    int n = (int)(sizeof(a) / sizeof(a[0]));

    int max = INT_MIN;     /* самое большое */
    int second = INT_MIN;  /* второе по величине */

    for (int i = 0; i < n; i++) {
        if (a[i] > max) {
            second = max;  /* бывший максимум становится вторым */
            max = a[i];
        } else if (a[i] > second && a[i] < max) {
            second = a[i]; /* не максимум, но больше второго */
        }
    }

    if (second == INT_MIN) {
        printf("Второго по величине нет.\n");
    } else {
        printf("Максимум: %d, второй: %d\n", max, second);
    }
    return 0;
}
```

Запуск:

```
Максимум: 9, второй: 7
```

Разбор. `INT_MIN` из `<limits.h>` — самое маленькое значение `int` (на этой машине −2147483648). Стартуем с него, чтобы любой реальный элемент оказался больше.

Пройдём массив `{3, 9, 9, 1, 7, 9, 4}` руками:

```
элемент  max      second   что произошло
   3     3        INT_MIN  первый больше max
   9     9        3        новый максимум, старый ушёл во второй
   9     9        3        не больше max и не меньше max — мимо обеих веток
   1     9        3        меньше second — мимо
   7     9        7        меньше max, больше second — обновили
   9     9        7        снова дубликат максимума — мимо
   4     9        7        меньше second — мимо
```

Максимум 9 встречается трижды, но вторым выводится 7, а не 9 — условие `a[i] < max` отсекает дубликаты максимума. Это именно тот случай, который любят подсунуть.

Сложность. Один проход — O(n) сравнений, O(1) памяти. Через сортировку было бы O(n log n), и интервьюер обязательно спросит, почему ты не отсортировал: потому что нам нужен один элемент, а не порядок.

Что спросят дальше.

- «Что если в массиве все элементы одинаковы (`{5,5,5}`)?» Тогда второго по величине нет — наш код напечатает «Второго по величине нет.», потому что `second` так и остался `INT_MIN`. Уточни у интервьюера, считать ли дубликат вторым: трактовки бывают разные, и важно проговорить выбор.
- «А если в массиве честно лежит `INT_MIN`?» Тогда наш признак «второго нет» ложно сработает на массиве `{5, INT_MIN}`: второй там есть, это `INT_MIN`, а программа скажет «нет». Лечится флагом `int have_second = 0;` вместо сравнения с `INT_MIN`. Хороший ответ, который показывает, что ты видишь дырку в собственном решении.
- «А массив из одного элемента?» Максимум найдётся, второго не будет — код напечатает «Второго по величине нет.», и это верно.
- «А пустой массив?» Тоже «Второго нет», но и `max` останется `INT_MIN`, хотя максимума у пустого массива нет вообще. В функции, которая возвращает результат, на `n < 2` сразу возвращают признак ошибки.
- «А k-й по величине?» Обобщение — там уже нужна частичная сортировка, куча (heap) или алгоритм быстрого выбора (quickselect). Это уже уровень Middle и выше.

Задача 6. Подсчёт гласных в строке

Условие. Посчитай, сколько гласных букв в строке (английские a, e, i, o, u в любом регистре).

Идея. Идём по строке посимвольно до `'\0'`. Для каждого символа проверяем, не гласная ли он. Чтобы не плодить проверки для верхнего и нижнего регистра, приводим символ к нижнему.

Решение (`code/19-06-vowels.c`):

```c
#include <stdio.h>
#include <string.h>

/* Возвращает 1, если c — английская гласная (любого регистра). */
int is_vowel(char c)
{
    char lower = c;
    if (c >= 'A' && c <= 'Z') {
```

```
        lower = (char)(c - 'A' + 'a');   /* привели к нижнему регистру */
    }
    return lower == 'a' || lower == 'e' || lower == 'i'
        || lower == 'o' || lower == 'u';
}

int main(void)
{
    const char *s = "Programming In C";
    int count = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (is_vowel(s[i])) {
            count++;
        }
    }
    printf("В строке \"%s\" гласных: %d\n", s, count);
    return 0;
}
```

Запуск:

```
В строке "Programming In C" гласных: 4
```

Считаем руками: `o`, `a`, `i` в `Programming` и `I` в `In` — четыре. Буква `C` не гласная, пробелы тоже.

Разбор. Трюк `c - 'A' + 'a'` переводит заглавную букву в строчную. Символы в C — это коды (см. главу 2): буквы `A..Z` идут в ASCII подряд, и `a..z` тоже подряд. Разница между заглавной и строчной одной буквы постоянна и равна `'a' - 'A'` = 97 − 65 = 32, поэтому сдвиг работает для всех букв разом. (Строго говоря, стандарт Си не обещает ASCII и не обещает, что буквы идут подряд, — это обещают только для цифр `'0'..'9'`. Но на всех машинах, где ты будешь это запускать, кодировка совместима с ASCII.) Приведение `(char)` — явная пометка «сужаю обратно в `char`, так задумано»: арифметика в Си поднимает `char` до `int`, и результат выражения — `int`. clang 21 здесь молчит даже с `-Wconversion`, но другие компиляторы на такое неявное сужение могут предупредить.

Условие цикла `s[i] != '\0'` — канонический способ пройти строку до завершающего нуля. Как только упёрлись в него, строка кончилась.

Сложность. O(n) — один проход, по одному сравнению на символ. Памяти O(1).

Что спросят дальше.

- «А `<ctype.h>`?» В стандартной библиотеке есть `tolower()` — он делает ровно наш сдвиг и уважает локаль. Но у него важная тонкость: аргумент надо приводить к `unsigned char`, иначе байт с кодом больше 127 в знаковом `char` превратится в отрицательное число, и получишь UB. Правильно так: `char lower = (char)tolower((unsigned char)c);`. Почему — разбираем в каверзе 4. На собесе ценят, когда ты знаешь про готовое, но умеешь и руками, и помнишь про эту ловушку.
- «Учитывай ‘y’.» Расширение списка — проверка внимательности: в английском `y` считается гласной не всегда, и стоит уточнить условие.
- «А русские гласные?» Не через `char`: в UTF-8 русская буква — два байта, посимвольная проверка сломается. Нужен разбор UTF-8 или широкие символы `wchar_t`.

Задача 7. Проверка на простое число

Условие. Дано число `n`. Определи, простое ли оно (делится только на 1 и на себя; 0, 1 и отрицательные — не простые).

Идея. Перебираем возможные делители от 2. Нашли хоть один — число составное. Ключевая оптимизация: проверять делители только **до корня** из `n`, потому что делители идут парами (`n = d · (n/d)`), и меньший из пары всегда не больше $\sqrt{n}$: если бы оба множителя были больше $\sqrt{n}$, их произведение было бы больше n. Для 36 пары такие: $2\cdot18$, $3\cdot12$, $4\cdot9$, $6\cdot6$ — и дальше они просто повторяются зеркально ($9\cdot4$, $12\cdot3$, $18\cdot2$). Значит, если до $\sqrt{n}$ делителя нет, то его нет и после.

Решение (`code/19-07-prime.c`):

```c
#include <stdio.h>

/* Возвращает 1, если n — простое число, иначе 0. */
int is_prime(int n)
{
    if (n < 2) {
        return 0;             /* 0, 1 и отрицательные — не простые */
    }
    for (int d = 2; d <= n / d; d++) {   /* то же, что d * d <= n */
        if (n % d == 0) {
            return 0;         /* нашли делитель — составное */
        }
    }
    return 1;
}
```

```c
int main(void)
{
    int n;
    printf("Введи число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (is_prime(n)) {
        printf("%d — простое\n", n);
    } else {
        printf("%d — не простое\n", n);
    }
    return 0;
}
```

Запуск:

```
Введи число: 17
17 — простое
```

Разбор. Условие цикла `d <= n / d` — это и есть «до корня»: оно верно ровно тогда, когда `d * d <= n`, то есть когда d не больше √n. Проверим на 36: при `d = 6` имеем `36 / 6 = 6`, и `6 <= 6` — ещё проверяем; при `d = 7` уже `36 / 7 = 5` (деление нацело), `7 <= 5` ложно — стоп, и `7 * 7 = 49` тоже больше 36. Для простого 999 983 цикл проверит делители от 2 до 999 — это 998 делений вместо почти миллиона при переборе до `n`.

Почему не написать проще, `d * d <= n`? Так пишут почти все, и это ловушка: при `n` рядом с `INT_MAX` квадрат `d * d` сам вылезает за пределы `int`. Разбор этой ловушки — ниже, в «что спросят дальше».

Случаи `n < 2` отсекаем сразу — иначе единица ошибочно сошла бы за простую: цикл `d <= 1 / 2` при `d = 2` не выполнится ни разу, и функция вернула бы 1.

Сложность. O(√n) делений в худшем случае (когда `n` простое и цикл доходит до конца). Для составных чисел обычно гораздо быстрее: у половины чисел делитель 2 находится на первой же итерации.

Что спросят дальше.

- «Почему не `d <= sqrt(n)`?» Чтобы не звать дробную функцию `sqrt`: не нужен `<math.h>` (а на Linux часто ещё и ключ `-lm` при сборке), нет перевода в `double` и обратно. Для `int` корень на обычной машине посчитается точно, но для `long long` больше 2^53 (около 9·10^15) `double` уже не хранит все целые подряд, и граница цикла может сдвинуться на единицу. Целочисленное сравнение точное всегда.
- «А почему не `d * d <= n`?» Потому что `d * d` может переполниться. 46340^2 = 2 147 395 600 ещё помещается в `int`, а 46341^2 = 2 147 488 281 уже больше `INT_MAX` = 2 147 483 647. Для составных чисел цикл обрывается раньше, но если `n` простое и больше 2 147 395 600, цикл дойдёт до `d = 46341`, и умножение даст переполнение — UB. Такое число есть, и очень знакомое: сам `INT_MAX`, 2 147 483 647, — простое. Я проверил: версия с `d * d <= n`, собранная с `-O2`, на нём отвечает «не простое» — неправда. `d <= n / d` не умножает вообще и переполниться не может. Назвать эту ловушку самому — сильный ход.
- «Ускорь ещё.» Проверять 2 отдельно, а дальше только нечётные делители (`d = 3; d += 2`) — вдвое меньше итераций.
- «Все простые до N?» Это уже **решето Эратосфена** (см. главу 5) — частый следующий шаг: O(N log log N) вместо N отдельных проверок, каждая до O(√N), то есть до O(N√N) в сумме.

Задача 8. Цифры числа: сколько их и какая сумма

Условие. Дано целое число. Посчитай, сколько в нём цифр (123 → 3) и чему равна их сумма (456 → 4+5+6 = 15).

Идея. Обе задачи решаются одним циклом. Делим число на 10 нацело, пока оно не станет нулём: каждое деление откусывает одну цифру справа (см. главу 4). Заодно на каждом шаге берём остаток `n % 10` — это и есть откушенная цифра.

Разберём 456 руками:

```
n = 456   digit = 456 % 10 = 6   sum = 6    count = 1   n = 45
n = 45    digit = 45  % 10 = 5   sum = 11   count = 2   n = 4
n = 4     digit = 4   % 10 = 4   sum = 15   count = 3   n = 0
n = 0  →  цикл кончился
```

Решение (`code/19-08-digits.c`):

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Введи целое число: ");
    if (scanf("%d", &n) != 1) {
```

```
        printf("Это не число.\n");
        return 1;
    }

    long long m = n;   /* в long long модуль INT_MIN помещается */
    if (m < 0) {
        m = -m;        /* знак на количество и сумму цифр не влияет */
    }

    int count = 0;     /* сколько цифр */
    int sum = 0;       /* сумма цифр */

    if (m == 0) {
        count = 1;     /* у нуля одна цифра */
    }
    while (m > 0) {
        int digit = (int)(m % 10);
        sum += digit;
        count++;
        m /= 10;
    }

    printf("Цифр: %d, сумма цифр: %d\n", count, sum);
    return 0;
}
```

Запуск:

```
Введи целое число: 90834
Цифр: 5, сумма цифр: 24
```

Проверяем: 9+0+8+3+4 = 24, цифр действительно пять.

Разбор. Считаем мы не в самом `n`, а в его копии `m` типа `long long` — зачем, объясню в «что спросят дальше», это главный подвох задачи. `m % 10` — последняя цифра, `m /= 10` (то же, что `m = m / 10`) — отбрасываем её. `digit` вычисляем **один раз** и кладём в переменную, чтобы не писать `m % 10` второй раз (см. главу 4, шаг 3). `(int)` перед `m % 10` честно говорит: остаток от 0 до 9, в `int` он помещается. `count` и `sum` стартуют с 0: накопитель суммы всегда начинается с нейтрального элемента, для сложения это ноль.

Отдельный случай — само число **0**: цикл `while (m > 0)` для него не выполнится ни разу, и `count` остался бы нулём, а ответ должен быть 1 (одна цифра). Поэтому ноль обрабатываем вручную. С суммой всё в порядке и без этого: сумма цифр нуля равна нулю.

Сложность. Число итераций равно количеству цифр. Для $n \geq 1$ цифр ровно $\lfloor \log_{10} n \rfloor + 1$ (логарифм по основанию 10, округлённый вниз, плюс один): у 90834 логарифм примерно 4,96, вниз — 4, плюс один — 5 цифр. Отсюда сложность O(log n). Для `int` это максимум 10 шагов: у 2 147 483 647 десять цифр.

Что спросят дальше.

- «А отрицательные?» У −123 три цифры, минус не считается, поэтому мы берём модуль. И вот подвох: напиши просто `if (n < 0) n = -n;` в `int` — и для `INT_MIN` получишь UB, потому что у самого маленького `int` нет положительной пары (диапазон несимметричен: от −2 147 483 648 до 2 147 483 647, и 2 147 483 648 в `int` не влезает). Я проверил такую версию: с `-O2` она на −2147483648 случайно печатает верное `Цифр: 10, сумма цифр: 47`, а собранная с `-fsanitize=undefined` — ругается на отрицание и печатает `Цифр: 0, сумма цифр: 0`. Один и тот же код, два разных ответа — типичное лицо UB. Поэтому модуль мы берём в `long long`: там 2 147 483 648 помещается с запасом. Другой путь — вообще не брать модуль и работать с отрицательными остатками (`-123 % 10` даёт −3, см. каверзу 9). На Junior это редко спрашивают, но упомянуть — большой плюс.
- «Цифровой корень: складывай сумму цифр, пока не останется одна цифра (9875 → 29 → 11 → 2).» Это цикл поверх нашего цикла — частое продолжение. Бонус: цифровой корень равен `1 + (n - 1) % 9` для положительных `n`, и это интервьюеру нравится. Проверим: $9874 = 9 \cdot 1097 + 1$, остаток 1, плюс один — 2. Работает, потому что число и сумма его цифр дают одинаковый остаток при делении на 9 ($10 = 9 + 1$, поэтому каждый разряд «весит» по модулю 9 ровно единицу).
- «Переверни число: 123 → 321.» Тот же цикл, но вместо суммы копим `rev = rev * 10 + digit`. С защитой от переполнения — см. Приложение В, задача 10.

Часть 2. Три классики без отдельного файла

Эти три задачи решаются в несколько строк, и отдельных файлов в `code/` под них нет — весь код прямо здесь. Но спрашивают их не реже остальных.

Задача 9. Проверить чётность числа

Условие. Дано целое число. Определи, чётное оно или нечётное.

Идея. Школьный способ — остаток от деления: `n % 2 == 0` (см. главу 4). Но на собесе часто хотят увидеть **битовый** приём: у чётного числа младший бит равен 0, у нечётного — 1. Проверить младший бит можно операцией `&` (побитовое «И») с единицей.

Решение:

```c
#include <stdio.h>

int main(void)
{
    int n = 7;

    if (n % 2 != 0) {
        printf("%d — нечётное\n", n);
    } else {
        printf("%d — чётное\n", n);
    }
    return 0;
}
```

Запуск:

```
7 — нечётное
```

Разбор. Почему я написал `n % 2 != 0`, а не более привычное `n % 2 == 1`? Потому что для **отрицательных** нечётных `n % 2` даёт не +1, а −1 (см. каверзу 9), и проверка `== 1` молча провалится: −7 будет объявлено чётным. Сравнение с нулём такой ошибки не допускает — от знака оно не зависит.

Битовый вариант выглядит так:

```c
if (n & 1) {
    printf("%d — нечётное\n", n);
}
```

`n & 1` сравнивает биты `n` и числа 1 по позициям. У единицы выставлен только самый младший бит. Поэтому `n & 1` оставляет от `n` ровно этот младший бит: 1 у нечётных, 0 у чётных. В `if` ненулевое — это истина (см. главу 2).

Что спросят дальше. «Чем `n & 1` лучше `n % 2`?» Для положительных чисел — ничем существенным: оба корректны. Проверка `n % 2 != 0` компилятор с `-O2` превращает ровно в ту же команду, что и `n & 1`, — я смотрел ассемблер (`cc -O2 -S`), там одна инструкция `andl $1`. Сам по себе оста-

ток `n % 2` для знакового `n` стоит на пару команд дороже: компилятору приходится поправлять знак. А вот для отрицательных есть тонкость, о которой хорошо бы сказать честно.

Стандарт C11 разрешает три представления знаковых чисел: дополнительный код, обратный код и «знак-величина». Побитовые операции работают с битами представления, поэтому результат `n & 1` для отрицательного `n` формально **implementation-defined** — зависит от платформы. В обратном коде, например, −7 записывается как `...11111000`, и `-7 & 1` дал бы 0 — «чётное». На любой реальной машине представление дополнительное (а с C23 оно и вовсе единственное разрешённое), и `-7 & 1` даёт 1. Но правильный ответ на собеседовании звучит так: «на нашей платформе — 1, потому что дополнительный код; переносимо и без оговорок — `n % 2 != 0`».

Задача 10. Максимум и минимум массива за один проход

Условие. Дан массив целых. Найди наибольший и наименьший элементы за один проход.

Идея. Заводим `max` и `min`, инициализируем их **первым элементом** массива (не нулём! см. подвох ниже) и идём по остальным, обновляя.

Решение:

```c
#include <stdio.h>

int main(void)
{
    int a[] = {4, -2, 9, 1, 7, -8, 3};
    int n = (int)(sizeof(a) / sizeof(a[0]));

    int max = a[0];
    int min = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] > max) {
            max = a[i];
        }
        if (a[i] < min) {
            min = a[i];
        }
    }

    printf("min = %d, max = %d\n", min, max);
    return 0;
}
```

Запуск:

```
min = -8, max = 9
```

Разбор. `sizeof(a) / sizeof(a[0])` — это размер всего массива в байтах, делённый на размер одного элемента, то есть **число элементов**. Здесь 28 / 4 = 7. Приём работает, только пока `a` — настоящий массив, а не указатель (см. подвох ниже).

Цикл стартует с `i = 1`, потому что нулевой элемент уже лежит и в `max`, и в `min` — сравнивать его с самим собой незачем.

Сложность. O(n) времени, два сравнения на каждый элемент, кроме первого, то есть 2(n − 1) сравнений — примерно 2n. O(1) памяти.

Что спросят дальше.

- «Почему инициализируешь первым элементом, а не нулём?» Если поставить `max = 0`, а все числа в массиве отрицательные — максимумом ошибочно останется 0, которого в массиве нет. То же с `min` и массивом из одних положительных. Безопасно стартовать значением **из самого массива**.
- «А если массив пустой?» `a[0]` уже выход за границу — UB. Нужна проверка `if (n == 0) return ...;` в самом начале. Пустой ввод — первое, о чём спрашивают.
- «Сделай за 1.5n сравнений.» Есть приём: брать элементы **парами**, сначала сравнивать их между собой, и меньший сравнивать с `min`, а больший с `max`. Получается 3 сравнения на 2 элемента вместо 4, то есть около 3n/2 = 1.5n сравнений на весь массив.
- «А если передать массив в функцию и там взять `sizeof`?» Не сработает! В функции параметр-массив превращается в **указатель**, и `sizeof` вернёт размер указателя (8 байт на этой машине), а не массива. Поэтому длину всегда передают **отдельным параметром**: `void f(int *a, int n)`. Классический подвох — полный разбор с реальным выводом и предупреждением компилятора в Приложении Б, каверза 2.

Задача 11. FizzBuzz

Условие. Числа от 1 до 100. Кратные 3 заменяй на `Fizz`, кратные 5 — на `Buzz`, кратные и 3, и 5 — на `FizzBuzz`, остальные печатай как есть.

Идея. Вся соль — в **порядке** проверок. Самое строгое условие (кратно 15) должно идти первым, иначе число 15 поймается веткой «кратно 3» и выдаст `Fizz` вместо `FizzBuzz` (полный разбор — в главе 4).

Решение (готовый файл — `code/04-fizzbuzz.c` из главы 4):

```
#include <stdio.h>
```

```c
int main(void)
{
    for (int i = 1; i <= 100; i++) {
        if (i % 15 == 0) {
            printf("FizzBuzz\n");
        } else if (i % 3 == 0) {
            printf("Fizz\n");
        } else if (i % 5 == 0) {
            printf("Buzz\n");
        } else {
            printf("%d\n", i);
        }
    }
    return 0;
}
```

Первые пятнадцать строк вывода:

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

Разбор. В цепочке `if / else if` срабатывает **первая** подходящая ветка, остальные даже не проверяются. Поэтому «кратно 15» (то же, что «кратно и 3, и 5») ставим раньше «кратно 3» и «кратно 5». Поменяй порядок — и на 15-й строке вместо `FizzBuzz` вылезет `Fizz`. Это ровно та ошибка, ради которой задачу и дают.

Сложность. O(n) — одна итерация на число, внутри константа операций.

Что спросят дальше. «Сделай без `% 15`, только через `% 3` и `% 5`.» Собери строку из флагов:

```
int by3 = (i % 3 == 0);
int by5 = (i % 5 == 0);
if (by3) printf("Fizz");
if (by5) printf("Buzz");
if (!by3 && !by5) printf("%d", i);
printf("\n");
```

Этот вариант легко расширяется (добавить «кратно 7 → Bazz» — две новые строки, `int by7 = ...` и `if (by7) printf("Bazz");`, и `&& !by7` в последнем условии; тогда 105 = 3 · 5 · 7 напечатается как `FizzBuzzBazz`), и интервьюер часто просит именно его: проверить, что ты не зубрил, а понимаешь структуру. Обрати внимание — `printf` без `\n` внутри и один общий перевод строки в конце: так `Fizz` и `Buzz` склеиваются в `FizzBuzz` сами собой. Я сверил: на числах от 1 до 100 обе версии печатают одно и то же, байт в байт.

Часть 3. Каверзы: «что выведет / в чём ошибка»

Здесь короткие фрагменты. На собесе их показывают и спрашивают: «что напечатает?» или «найди ошибку». Цель — не код написать, а проверить, насколько хорошо ты знаешь язык в деталях. Сначала подумай сам, потом читай ответ.

Все фрагменты я запускал, завернув каждый в обычную программу: `#include <stdio.h>`, `#include <limits.h>`, пустая строка, `int main(void)`, `{`, сам фрагмент, `return 0;`, `}`. Файлы назывались `k1.c`, `k2.c` и так далее по номеру каверзы — отсюда имена и номера строк в сообщениях компилятора ниже. Длинные строки сообщений я перенёс, чтобы влезли в страницу.

Важная общая мысль: у части таких вопросов **нет одного правильного ответа**. Если фрагмент содержит неопределённое поведение (UB) или опирается на свойства платформы, ответ «выведет вот столько» просто неверен. Правильный ответ — назвать это своим именем. Ниже я отмечаю каждый такой случай.

Каверза 1. `sizeof('a')` — сколько это?

```
printf("%zu %zu\n", sizeof(int), sizeof('a'));
```

Вопрос. Что выведет? Подсказка-ловушка: `'a'` — это же один символ, значит 1 байт?

Ответ. На этой машине выведет:

```
4 4
```

Сюрприз в том, что в языке **C** символьный литерал `'a'` имеет тип не `char`, а **`int`**! Поэтому `sizeof('a')` равен `sizeof(int)`.

Важно: это особенность именно C. В C++ `'a'` — это `char`, и там `sizeof('a')` равно 1. На собесе по C правильный ответ — «столько же, сколько `sizeof(int)`».

И вторая половина ответа: число 4 — это **не** гарантия стандарта. Стандарт требует только, чтобы `int` вмещал хотя бы числа от −32767 до 32767 (то есть имел минимум 16 бит) и чтобы его диапазон был не уже, чем у `short`. На современных 32- и 64-битных системах `int` — 4 байта, на 8- и 16-битных микроконтроллерах бывает 2 байта. Единственный размер, зафиксированный намертво, — `sizeof(char)`, он **всегда** 1: char и есть единица измерения для `sizeof`.

`%zu` — спецификатор для типа `size_t`, который возвращает `sizeof` (беззнаковый тип для размеров). Печатать его через `%d` формально неверно: `size_t` на 64-битной системе шире `int`, и `printf` возьмёт из аргументов не то количество байт.

Каверза 2. Целочисленное деление `5 / 2`

```c
printf("%d\n", 5 / 2);
double x = 5 / 2;
printf("%f\n", x);
```

Вопрос. Что в обеих строках?

Ответ. Реальный запуск:

```
2
2.000000
```

Первая строка: деление двух **целых** идёт нацело, дробная часть отбрасывается (см. главу 2), а не округляется. `5 / 2` это 2, а не 2.5 и не 3. Кстати, `7 / 2` тоже 3, а не 4 — отбрасывание, не округление.

Вторая печатает `2.000000`, а **не** 2.5, и это главный подвох. Хотя `x` объявлен как `double`, выражение `5 / 2` считается **до** записи в `x`: оба операнда целые → целочисленное деление → 2 → и уже 2 превращается в `2.0`. Тип переменной слева не влияет на то, как вычисляется правая часть. Это общее правило Си, и оно ловит очень многих.

Чтобы получить 2.5, дробным должен быть хотя бы один операнд: `5.0 / 2`, или `(double)5 / 2`, или `5 / 2.0`. Достаточно одного — второй операнд подтянется к `double` автоматически.

Каверза 3. `printf("%d", c)` для `char`

```
char c = 'A';
printf("%d\n", c);
```

Вопрос. Что выведет?

Ответ.

```
65
```

Символ в С — это маленькое целое, его **код** (см. главу 2). Код буквы `A` в таблице ASCII равен 65; стандарт Си ASCII не требует, но на всех привычных машинах кодировка с ним совпадает. Спецификатор `%d` просит напечатать число, поэтому печатается не сама буква, а её код. Хочешь увидеть букву `A` — используй `%c`.

И наоборот полезно помнить: `printf("%c\n", 66)` напечатает `B` — число истолкуется как код символа. На этой двойственности (символ = число) держатся трюки вроде перевода регистра из задачи 6 и перевода символа-цифры в число через `c - '0'`.

Каверза 4. `char` — знаковый или беззнаковый?

```
char c = (char)200;
printf("%d\n", c);
```

Вопрос. Что выведет — 200 или что-то другое?

Ответ. Правильный ответ: **зависит от платформы**, стандарт этого не фиксирует. Тип `char` в Си — не то же самое, что `signed char` или `unsigned char`; это **третий**, отдельный тип, и знаковый он или нет, решает компилятор. Стандарт называет это implementation-defined поведением.

На этой машине (clang, x86-64, macOS) `char` знаковый, и вывод такой:

```
-56
```

Откуда −56? Байт `200` в двоичном виде — `11001000`. В знаковый `char` (от −128 до 127) число 200 не помещается, и что получится при таком преобразовании, стандарт тоже оставляет на усмотрение компилятора. Все распространённые компиляторы просто берут те же 8 бит и читают их в дополнительном коде: старший бит весит не +128, а −128, поэтому −128 + 64 + 8 = −56. То же самое проще: 200 − 256 = −56. Проверить, какой `char` у тебя, можно через `<limits.h>`: у нас `CHAR_MIN = -128`, `CHAR_MAX = 127` — значит, знаковый. На ARM под Linux или на PowerPC ты увидел бы `CHAR_MIN = 0` и вывод `200`. (Не путай с Mac на процессорах Apple: это тоже ARM, но Apple сделала `char` знаковым, как на x86.)

Почему это важно на практике. Все функции из `<ctype.h>` — `isalpha`, `isdigit`, `tolower`, `toupper` — принимают `int` и требуют, чтобы значение было либо `EOF`, либо представимым как `unsigned char`. Передашь туда знаковый `char` с кодом больше 127 (например, байт из UTF-8-строки) — получишь отрицательное число, выход за таблицу и UB:

```
char c = s[i];
if (isalpha(c)) { ... }              /* ОПАСНО: c может быть < 0 */
if (isalpha((unsigned char)c)) { ... }   /* правильно */
```

То же самое с `strcmp`: стандарт требует сравнивать символы **как `unsigned char`**, иначе строки с русскими буквами сравнивались бы не в том порядке. Именно поэтому в Приложении Б в самодельной `my_strcmp` стоит приведение к `unsigned char`.

Хочешь однозначности — пиши `signed char` или `unsigned char` явно. Голый `char` используй только для текста.

Каверза 5. = вместо == в условии

```
int n = 5;
if (n = 0) {
    printf("ноль\n");
} else {
    printf("не ноль\n");
}
printf("n = %d\n", n);
```

Вопрос. Что выведет?

Ответ. Реальный запуск:

```
не ноль
n = 0
```

Здесь стоит один `=` (присваивание), а не `==` (сравнение). Поэтому `n = 0` сначала **кладёт** 0 в `n`, а потом результатом всего выражения становится само присвоенное значение — 0. Ноль в условии — это «ложь», поэтому срабатывает ветка `else`. Заодно `n` теперь равно 0, хотя мы вроде бы только «проверяли» — вторая строка вывода это подтверждает.

Если бы было `if (n = 5)`, условие стало бы истинным (5 — ненулевое), и такие баги особенно коварны: код «как будто работает».

Спасает компилятор. Собери с `-Wall` — и он сам покажет пальцем:

```
k5.c:7:11: warning: using the result of an assignment as a condition
without parentheses [-Wparentheses]
    7 |     if (n = 0) {
      |         ~~^~~
k5.c:7:11: note: place parentheses around the assignment to silence
this warning
    7 |     if (n = 0) {
      |           ^
      |         (   )
k5.c:7:11: note: use '==' to turn this assignment into an equality
comparison
    7 |     if (n = 0) {
      |           ^
      |           ==
1 warning generated.
```

(Первая подсказка — «поставь лишние скобки, если присваивание задумано», вторая — «или ты хотел ==».)

Защитный приём — «йода-условие»: писать `if (0 == n)`; тогда опечатку `0 = n` компилятор отвергнет как ошибку (константе присвоить нельзя). Приём спорный — читается непривычно, — но на собеседовании про него приятно знать.

Каверза 6. Лишняя `;` после `for(...)`

```c
int sum = 0;
for (int i = 1; i <= 5; i++);
{
    sum += i;
}
printf("%d\n", sum);
```

Вопрос. Найди ошибку. Что не так?

Ответ. Лишняя точка с запятой сразу после `for(...)`. Эта `;` — **пустое тело цикла**. То есть цикл крутится 5 раз и ничего не делает. А блок в фигурных скобках ниже — это уже **обычный блок**, не связанный с циклом; он выполнится **один раз**.

Хуже того: этот фрагмент вообще не скомпилируется — `i` объявлена внутри `for` и за его пределами не видна. Вот что реально говорит clang:

```
k6.c:9:16: error: use of undeclared identifier 'i'
    9 |         sum += i;
      |                ^
k6.c:7:33: warning: for loop has empty body [-Wempty-body]
    7 |     for (int i = 1; i <= 5; i++);
      |                                 ^
k6.c:7:33: note: put the semicolon on a separate line to silence this
warning
1 warning and 1 error generated.
```

Обрати внимание: компилятор выдал и ошибку, и предупреждение про пустое тело — то есть увидел обе проблемы разом.

Если бы `i` была объявлена снаружи цикла (`int i;` и `for (i = 1; ...)`), код собрался бы — с тем же предупреждением про пустое тело — и напечатал `6`: цикл ничего не сделал, а блок один раз прибавил к нулю значение `i`, оставшееся после цикла (6). Логика при этом всё равно сломана: ждали 1+2+3+4+5 = 15. Я проверил и этот вариант — вывод действительно `6`.

Мораль: точка с запятой сразу после `for`, `while` или `if` — почти всегда ошибка. Настоящий пустой цикл (например, `while (*p++);`) пишут либо с `{}`, либо с `;` на отдельной строке — именно это и советует clang.

Каверза 7. `printf("%d %d", i++, ++i)`

```c
int i = 1;
printf("%d %d\n", i++, ++i);
```

Вопрос. Что выведет?

Ответ. Честный ответ: **неизвестно**, «правильного» числа тут нет. Это **неопределённое поведение** (undefined behavior, UB). В одном выражении переменная `i` меняется дважды (`i++` и `++i`) без точки следования между изменениями. Стандарт C не определяет ни порядок вычисления аргументов `printf`, ни момент применения `++`, — а модификация одного объекта дважды без упорядочивания прямо объявлена неопределённым поведением.

Компилятор про это знает и предупреждает:

```
k7.c:7:24: warning: multiple unsequenced modifications to 'i'
[-Wunsequenced]
    7 |     printf("%d %d\n", i++, ++i);
      |                        ^   ~~
1 warning generated.
```

Если запустить, на этой машине печатается `1 3` — и с `-O0`, и с `-O2`. Но это **не ответ на вопрос**. Другой компилятор, другая версия того же компилятора или другие флаги имеют полное право выдать `2 3`, `1 2` или вообще что угодно: программа некорректна, и обсуждать её вывод бессмысленно.

Если на собесе спрашивают «что выведет» — правильный ответ не «2 3» и не «1 3», а именно: «это UB, результат не определён, так писать нельзя». Интервьюер проверяет, узнаёшь ли ты опасную конструкцию, а не считаешь ли в уме «как у меня на ноутбуке». А если он настаивает, что ответ один-единственный, — покажи ему `-Wunsequenced`.

Лечится разнесением по строкам:

```
int a = i++;   /* теперь порядок однозначен */
int b = ++i;
printf("%d %d\n", a, b);
```

Между полными выражениями (то есть между `;`) точка следования есть, и всё определено: `a` получит 1, `b` — 3.

Каверза 8. Неинициализированная переменная

```
int sum;
for (int i = 1; i <= 5; i++) {
    sum += i;
}
printf("%d\n", sum);
```

Вопрос. Что выведет?

Ответ. Заранее **неизвестно**, и это снова UB, а не «какое-то число». `sum` объявлена, но не инициализирована — в локальной переменной лежит мусор (что осталось в этой ячейке стека). К мусору прибавляется 1+2+3+4+5 = 15.

Вот три запуска подряд одной и той же программы, собранной с `-O2`:

```
-1302150376
-1227496680
-1154252008
```

Числа разные при каждом запуске. Собранная с `-O0` та же программа у меня три раза подряд напечатала одно и то же `1250286487` — тоже мусор, просто стабильный. Ни то ни другое не 15, и ни на что из этого нельзя полагаться. Может «повезти» и вывестись 15, если в ячейке был ноль, — и это худший исход: баг спрячется и вылезет у пользователя.

Компилятор ловит это сам:

```
k8.c:8:9: warning: variable 'sum' is uninitialized when used here
[-Wuninitialized]
    8 |         sum += i;
      |         ^~~
k8.c:6:12: note: initialize the variable 'sum' to silence this warning
    6 |     int sum;
      |            ^
      |             = 0
1 warning generated.
```

Фикс — один символ: `int sum = 0;`. Любой накопитель перед циклом обнуляй (сумма — в 0, произведение — в 1). И не игнорируй предупреждения: `-Wall` тут буквально дописал за тебя правильный код.

Отдельно запомни: **глобальные** и `static`-переменные обнуляются автоматически, а локальные — нет. Отсюда и путаница у новичков: «у меня же в прошлый раз работало».

Каверза 9. Остаток от отрицательного числа

```c
printf("%d %d\n", -7 / 2, -7 % 2);
```

Вопрос. Что выведет?

Ответ.

```
-3 -1
```

В современном C (стандарт C99 и новее) деление целых округляется **к нулю**: `-7 / 2` это −3, а не −4. Раньше, в C89, направление округления было отдано на откуп компилятору, — но это уже история.

Остаток подчиняется тождеству `(a / b) * b + a % b == a`. Подставим: `(-3) * 2 + x == -7`, значит `x == -1`. Отсюда `-7 % 2` равен −1, а не +1. Проще запомнить так: **знак остатка совпадает со знаком делимого** (если остаток не ноль). Обрати внимание: это не тот остаток, что в школьной математике. Там $-7 = 2 \cdot (-4) + 1$, остаток 1 — он всегда от 0 до делителя. Оператор `%` в Си — не «математический остаток», а «то, что осталось после деления с отбрасыванием дробной части».

Практический вывод: проверять нечётность через `n % 2 == 1` **опасно** — для отрицательных нечётных получится −1, и проверка провалится. Пиши `n % 2 != 0`. Это прямая связь с задачей 9 про чётность.

Ещё одно применение того же правила — разворот числа в Приложении В (задача 10): там `-123 % 10` даёт −3, и знак сохраняется сам собой, без единого `if`.

Ошибки, на которых сыплются в этих задачах

Собрал в одном месте то, что чаще всего ломается именно в задачах этого уровня. Пробеги глазами перед собеседованием.

- **Накопитель без начального значения.** `int sum;` вместо `int sum = 0;` — и в сумме мусор (каверза 8). Для произведения начальное значение 1, не 0.
- **Старт `max`/`min` с нуля.** На массиве из одних отрицательных максимум окажется нулём, которого в массиве нет. Стартуй элементом массива (задача 10).
- **Забытая проверка `scanf`.** `scanf("%d", &n)` возвращает число прочитанных значений. Не проверил — работаешь с мусором в `n`.
- **`sizeof` массива внутри функции.** Даёт размер указателя, а не массива. Длину передавай параметром.
- **`n % 2 == 1` для отрицательных.** Даёт −1, проверка проваливается (каверза 9).
- **Пустая строка и `strlen(s) - 1` в `size_t`.** Беззнаковое вычитание заворачивается в огромное число, и цикл уходит за границы. Держи индексы в `int` или проверяй пустоту отдельно.
- **`-n` для `INT_MIN`.** Переполнение знакового, то есть UB. Всплывает в «возьми модуль» (задача 8).
- **`ctype.h` без `(unsigned char)`.** UB на байтах с кодом больше 127 (каверза 4).
- **Одна переменная меняется дважды в одном выражении.** UB (каверза 7). Разнеси по строкам.
- **Точка с запятой после `for`/`while`/`if`.** Пустое тело, логика ломается молча (каверза 6).

Тренировочные задачи

Решай без подглядывания в разборы. Каждая — это чуть изменённое условие из тех, что уже разобраны: интервьюеры именно так и «дожимают».

1. Перепиши задачу 3 (палиндром) так, чтобы регистр не имел значения и игнорировались все не-буквы. Проверь на строке `"A man, a plan, a canal: Panama"` — должно получиться «палиндром». Ещё проверки: `"Level"` и `"No 'x' in Nixon"` — палиндромы, `"hello"` и `"ab!"` — нет, пустая строка и `"!!"` — палиндромы (букв нет, сравнивать нечего). Подсказка: сравнивай `tolower((unsigned char)s[left])` и `tolower((unsigned char)s[right])`.

2. Допиши задачу 5 (второй максимум) так, чтобы она правильно работала на массиве `{5, INT_MIN}`. Подсказка: замени сравнение с `INT_MIN` на отдельный флаг. Ожидаемо: для `{5, INT_MIN}` и для `{INT_MIN, 5}` — `Максимум: 5, второй: -2147483648`; для `{5, 5, 5}` и для `{42}` — по-прежнему «Второго по величине нет.»; для массива из задачи — `Максимум: 9, второй: 7`. Учти, что флаг нужен и для максимума: у второго максимума стартового значения «меньше всех» больше нет.
3. Реализуй счётчик цифр из задачи 8 для `long long` и убедись, что для `LLONG_MIN` программа не падает и не выдаёт мусор. Подвох: у `long long` нет типа шире, куда можно убежать от `-LLONG_MIN`, так что модуль не бери, а работай с отрицательными остатками (`n % 10` для отрицательного `n` даёт цифру со знаком минус — возьми её модуль, он от 0 до 9) и цикл веди до `n != 0`. Ожидаемо: для −9223372036854775808 — 19 цифр, сумма 89; для 9223372036854775807 — 19 цифр, сумма 88; для 0 — 1 цифра, сумма 0.
4. Напиши функцию `int count_words(const char *s)`, которая считает слова, разделённые пробелами. Проверь на строках `""`, `"   "`, `"one"`, `"  dva   slova  "`. Ожидаемо: 0, 0, 1, 2. Подсказка: слово начинается там, где после пробела (или в самом начале строки) идёт не пробел, — как в главе 10.
5. Напиши «обратный FizzBuzz»: программа получает строку `Fizz`, `Buzz`, `FizzBuzz` или число и печатает наименьшее число от 1 до 100, которое ей соответствует. Ожидаемо: `Fizz` → 3, `Buzz` → 5, `FizzBuzz` → 15, `7` → 7. А для `9` ответа нет: число 9 FizzBuzz печатает как `Fizz`, так что строку `9` не выдаёт никогда, — программа должна так и сказать.
6. Возьми задачу 7 (простое число) и верни «наивное» условие `d * d <= n` вместо `d <= n / d`. Собери с `-fsanitize=undefined` и введи 2147483647. Ожидаемо: санитайзер сообщит `signed integer overflow: 46341 * 46341 cannot be represented in type 'int'`, а обычная сборка с `-O2` ответит «не простое», хотя 2147483647 простое. Для всех `n` до 200 000 обе версии отвечают одинаково — ошибка живёт только у самого края диапазона `int`, и поэтому её так трудно поймать тестами.

Мини-резюме

Что стоит унести из этого приложения — не сами решения (их в интернете тысячи), а привычки, которые отличают подготовленного кандидата:

- **Краевые случаи проговаривай вслух**: пустой массив, один элемент, ноль, отрицательное, дубликаты, переполнение. Интервьюер часто ставит плюс именно за то, что ты сам про них вспомнил.
- **Помни про типы и переполнение.** `int` лопается на ~2 млрд; символьный литерал — это `int` в C; деление целых идёт нацело; знаковость `char` зависит от платформы. Эти четыре факта спрашивают чаще всего.

- **Знай про опасные приёмы, но не используй их без нужды.** XOR-своп, арифметический обмен — рассказать можешь, в код кладёшь `temp`.
- **UB называй своим именем.** Увидел `i++` и `++i` в одном выражении, неинициализированную переменную, изменение строкового литерала, переполнение знакового — отвечай «это неопределённое поведение», а не угадывай результат. И отдельно называй implementation-defined: размер `int`, знаковость `char` — тут ответ есть, но он свой на каждой платформе.
- **Компилируй с `-Wall -Wextra`.** Больше половины каверз из третьей части компилятор ловит сам — мы это только что видели по его сообщениям.

Куда дальше

Это был уровень Junior — задачи, где главное показать, что основы языка сидят крепко, а голова замечает краевые случаи. В **Приложении Б (Middle)** мы поднимем планку: связные списки и их развороты, работа с указателями всерьёз, бинарный поиск без ошибок на единицу, битовые приёмы, аккуратная работа с памятью под присмотром санитайзеров, и каверзы потяжелее — про строковые литералы, распад массива в указатель, `const`, утечки и время жизни объектов. Если задачи отсюда даются легко и без подглядывания — ты готов двигаться дальше.

Приложение Б. Задачи с собеседований: Middle

В Приложении А мы размялись на простых задачах уровня Junior. Теперь — ступень выше. Это вопросы, которые задают на собеседовании, когда хотят проверить, что ты понимаешь Си **по-настоящему**, а не просто умеешь печатать `printf`. Здесь всё крутится вокруг пяти тем, на которых ломается большинство:

- **указатели** — кто на что смотрит и кто кого меняет;
- **память** — где живёт строка, когда `malloc`, когда `free`;
- **строки** — массив `char` с `'\0'` на конце (см. главу 3);
- **биты** — числа как набор нулей и единиц;
- **связные списки** — узлы и стрелки `next` (см. главу 11).

Как пользоваться этим приложением

Не читай решение сразу. Прочитай условие, закрой книгу и попробуй сам — хоть на бумаге, хоть в редакторе. На собеседовании ценят не зазубренный код, а ход мысли. Поэтому у каждой задачи есть блок «идея» (как рассуждать) и «доп. вопросы» — то, что интервьюер спросит следом, чтобы понять, не списал ли ты.

Задачи идут от простого к сложному. Сначала — **задачи на код**. У задач 1–8 номер совпадает с номером файла в `code/`: задача 5 — это `code/20-05-binsearch.c`, и так далее. Каждый файл собирается без единого предупреждения:

```
cc -std=c11 -Wall -Wextra -O2 code/20-05-binsearch.c -o binsearch
./binsearch
```

У задачи 9 отдельного файла нет — она в пять строк, код прямо в тексте.

Потом идут **каверзы**: коварные вопросы про язык, где правильный ответ не «что выведет», а «почему так, и можно ли на это полагаться». Часть из них содержит намеренные ошибки

и неопределённое поведение, поэтому в `code/` их нет: в файлах книги лежит только тот код, который можно запускать.

Все выводы и все сообщения компилятора в этом приложении получены реальным запуском: Apple clang 21, x86-64, macOS. Где результат зависит от платформы или от флагов сборки — я это отдельно оговариваю.

Поехали.

Часть 1. Задачи на код

Задача 1. Свои `strlen`, `strcpy`, `strcmp`

Условие. Реализуй три функции стандартной библиотеки `<string.h>` своими руками, без её вызова: длину строки, копирование, сравнение.

Идея. Строка в Си — это массив `char`, заканчивающийся нулевым символом `'\0'` (см. главу 3). Значит, любой обход строки — это «иди по символам, пока не упрёшься в `'\0'`». На этом держатся все три функции.

Решение (`code/20-01-mystr.c`):

```c
#include <stdio.h>

/* Своя длина строки: считаем символы до '\0'. */
size_t my_strlen(const char *s)
{
    size_t n = 0;
    while (s[n] != '\0') {  /* пока не дошли до конца строки */
        n++;
    }
    return n;
}

/* Своё копирование: льём src в dst посимвольно, вместе с '\0'. */
char *my_strcpy(char *dst, const char *src)
{
    char *start = dst;          /* запомнили начало, чтобы вернуть */
    while ((*dst = *src) != '\0') {
        dst++;
        src++;
    }
```

```
    return start;
}

/* Своё сравнение: 0 если равны, иначе разница первых отличий. */
int my_strcmp(const char *a, const char *b)
{
    while (*a != '\0' && *a == *b) {
        a++;
        b++;
    }
    return (unsigned char)*a - (unsigned char)*b;
}
```

В файле после этих трёх функций идёт `main`, который их гоняет; я его не привожу, чтобы не отвлекаться, — открой файл и посмотри.

Объяснение.

- `size_t` — специальный беззнаковый тип для размеров и длин. Именно его возвращает настоящая `strlen`. Печатают `size_t` через `%zu`. Стандарт объявляет его сразу в нескольких заголовках, среди них `<stddef.h>`, `<string.h>` и `<stdio.h>`, поэтому одного `#include <stdio.h>` здесь хватает.
- `const char *s` — указатель на строку, которую мы **только читаем**. Слово `const` обещает «я не буду менять то, на что смотрит `s`». Это хороший тон: компилятор подстрахует, и тому, кто вызывает функцию, видно, что аргумент в безопасности. И `const` работает в одну сторону: передать `char *` туда, где ждут `const char *`, можно свободно, а наоборот — нет, clang предупредит: `passing 'const char *' to parameter of type 'char *' discards qualifiers`. Поэтому `const` на входном параметре расширяет круг того, что тебе можно передать, а не сужает.
- В `my_strcpy` трюк в строке `while ((*dst = *src) != '\0')`. Сначала `*dst = *src` копирует один символ **и** возвращает его значение (присваивание в Си — это выражение, а не только действие). Это значение тут же сравниваем с `'\0'`. Получается: скопировали символ; если это был конец строки — выходим, но `'\0'` уже скопирован, что нам и нужно. Скобки вокруг `*dst = *src` обязательны: у `!=` приоритет выше, чем у `=`, и без скобок вышло бы `*dst = (*src != '\0')`, то есть в приёмник попали бы единицы и ноль вместо букв.
- `my_strcpy` возвращает `start` — указатель на начало приёмника. Так делает и стандартная `strcpy`: удобно писать `printf("%s", my_strcpy(buf, "abc"))`.
- В `my_strcmp` идём, пока символы совпадают и строка не кончилась. На первом различии (или на общем конце) выходим и возвращаем разность кодов символов. Если строки равны, оба `*a` и `*b` — это `'\0'`, разность 0.

- Приведение к `unsigned char` в `my_strcmp` не украшательство: `char` бывает знаковым (на этой машине он именно такой — см. Приложение А, каверза 4), и байт с кодом 200 стал бы −56. Тогда строка с русской буквой оказалась бы «меньше» строки с латинской. Стандарт требует сравнивать именно как `unsigned char`, и мы это требование выполняем.

Запуск:

```
my_strlen("Hello") = 5
my_strcpy -> skopiruem
my_strcmp(abc, abc) = 0
my_strcmp(abc, abd) = -1
my_strcmp(abd, abc) = 1
```

−1 получилось как `'c' - 'd'` = 99 − 100. Число могло бы быть и другим — важен только знак.

Сложность. O(n) — это запись «время растёт не быстрее, чем пропорционально n» (подробно — в главе 8). Здесь n — длина строки: `my_strlen` и `my_strcpy` делают ровно n шагов цикла, `my_strcmp` — не больше длины более короткой строки. Дополнительной памяти O(1), то есть несколько переменных, сколько бы длинной ни была строка.

Доп. вопросы интервьюера.

- *Чем опасна `strcpy`?* Она не знает размер приёмника `dst` и пишет, пока в `src` не кончатся символы. Если `dst` короче — затрёт чужую память (переполнение буфера). Это классический источник уязвимостей. Безопаснее `snprintf` или `strncpy` — но у `strncpy` своя ловушка: если источник не влез, она **не** дописывает `'\0'`.
- *Почему `strcmp` возвращает не просто 1/−1, а разность?* Стандарт гарантирует только знак результата (меньше/равно/больше нуля), не само число. Полагайся на знак, не на конкретную величину: разные реализации вернут разное.
- *Что будет, если `dst` и `src` в `strcpy` перекрываются?* UB. Для перекрывающихся областей есть `memmove`, а `memcpy` и `strcpy` этого не допускают.

Задача 2. Развернуть односвязный список

Условие. Дан односвязный список `1 -> 2 -> 3 -> ... -> NULL`. Разверни его, чтобы стало `... -> 3 -> 2 -> 1 -> NULL`. На месте, без выделения нового списка.

Идея. У каждого узла есть стрелка `next`, смотрящая вперёд. Развернуть список — значит у каждого узла **перенаправить стрелку назад**. Идём по списку и на каждом шаге разворачиваем одну стрелку. Но если просто написать `cur->next = prev`, мы потеряем доступ к хвосту — поэтому хвост надо запомнить **до** разворота. Нужны три указателя: `prev` (узел позади), `cur` (текущий), `next` (запасённый хвост).

Нарисуем. Было:

```
prev      cur
 |         |
 ▼         ▼
NULL    [1|·] ──► [2|·] ──► [3|·] ──► NULL
```

Запомнили `next = cur->next` (это `[2]`), развернули стрелку `cur` на `prev`:

```
        prev   next
          |     |
          ▼     ▼
NULL ◄── [1|·] [2|·] ──► [3|·] ──► NULL
```

Сдвинули `prev` и `cur` на шаг вперёд (`prev = cur; cur = next`) и повторяем. Когда `cur` станет `NULL`, `prev` будет указывать на бывший последний узел — это новая голова.

Решение (`code/20-02-reverse-list.c`, привожу ядро):

```c
struct Node {
    int value;
    struct Node *next;
};

/* Разворот списка через три указателя: prev, cur, next. */
struct Node *reverse(struct Node *head)
{
    struct Node *prev = NULL;       /* позади ничего нет */
    struct Node *cur = head;        /* бегунок по списку */
    while (cur != NULL) {
        struct Node *next = cur->next;  /* запомнили хвост */
        cur->next = prev;               /* развернули стрелку назад */
        prev = cur;                     /* prev сдвинулся вперёд */
        cur = next;                     /* cur сдвинулся вперёд */
    }
    return prev;                    /* prev — новая голова */
}
```

Объяснение. Порядок четырёх строк внутри цикла — священен.

1. `next = cur->next` — сохранили, куда идти дальше, пока стрелка цела.
2. `cur->next = prev` — развернули стрелку текущего узла назад.
3. `prev = cur` — `prev` догнал `cur`.

4. `cur = next` — `cur` шагнул на сохранённый хвост.

Поменяешь местами хоть две — либо потеряешь хвост, либо зациклишь узел сам на себя. Например, если поставить `cur->next = prev` перед сохранением `next`, то `cur->next` уже перезаписан и хвост списка недостижим: остальные узлы утекут.

Функция возвращает `prev`, потому что после цикла `cur` уже `NULL`, а `prev` стоит на последнем обработанном узле — он и есть новая голова. В `main` обязательно перезаписываем голову: `head = reverse(head)`. Забудешь — и старая `head` будет указывать на узел, у которого `next` теперь `NULL`: список «схлопнется» в один элемент, а остальные утекут.

`->` — это сокращение: `cur->next` то же самое, что `(*cur).next`. Сначала разыменовать указатель, потом взять поле.

Сложность. Один проход — O(n) по времени, O(1) по памяти: три указателя, новый список не выделяем, `malloc` ни разу не зовём.

Краевые случаи. Пустой список (`head == NULL`): цикл не выполнится, вернём `NULL` — верно. Список из одного узла: одна итерация, `prev` станет этим узлом, его `next` станет `NULL` — тоже верно.

Запуск:

```
До:    5 -> 4 -> 3 -> 2 -> 1 -> NULL
После: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
```

Список собран через `push_front` в цикле `for (i = 1; i <= 5; i++)`, поэтому «до» он идёт от 5 к 1 — вставка в начало переворачивает порядок сама по себе.

Доп. вопросы.

- *Как развернуть рекурсивно?* Можно (см. главу 9 про рекурсию), но тогда расход памяти O(n) на стек вызовов, и на длинном списке возможно переполнение стека. Итеративная версия лучше.
- *Как развернуть только кусок списка, с позиции m по n?* Тот же приём, но нужно запомнить узел перед `m` и узел `m` (он станет хвостом перевёрнутого куска), а после разворота сшить три части.
- *А если список двусвязный?* Там достаточно у каждого узла поменять местами `prev` и `next` и переставить голову с хвостом.

Задача 3. Два указателя: середина списка, цикл и его начало

Все три части этой задачи решает один и тот же приём — **два указателя разной скорости**, — и все они живут в одном файле `code/20-03-floyd.c`. Спрашивают их обычно подряд: сначала середину, потом «а теперь найди цикл», потом «а где он начинается».

3а. Середина списка

Условие. Найди средний узел односвязного списка за один проход, не зная заранее его длину.

Идея. Самый прямой способ — пройти список дважды: сначала посчитать длину, потом дойти до середины. Но можно за **один** проход трюком «два указателя разной скорости». Медленный `slow` шагает по одному узлу, быстрый `fast` — по два. Когда `fast` добежит до конца, `slow` будет ровно посередине: он прошёл вдвое меньше.

Решение:

```c
/* Середина списка: медленный шагает на 1, быстрый на 2. */
struct Node *middle(struct Node *head)
{
    struct Node *slow = head;
    struct Node *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;          /* +1 */
        fast = fast->next->next;    /* +2 */
    }
    return slow;
}
```

Объяснение. Главное здесь — условие цикла `fast != NULL && fast->next != NULL`. Быстрый прыгает через два узла, и перед прыжком надо убедиться, что **оба** узла существуют: и сам `fast`, и `fast->next`. Иначе `fast->next->next` разыменует `NULL` — падение.

Порядок проверок важен: `&&` вычисляет левую часть первой, и если `fast` уже `NULL`, до `fast->next` дело не дойдёт. Это **укороченное вычисление** (short-circuit, см. главу 2): правая часть `&&` не вычисляется, если левая уже дала «ложь». Переставь проверки местами — и получишь разыменование `NULL` на списке чётной длины.

Проследим на списке из 7 узлов (`1 2 3 4 5 6 7`):

```
старт:  slow=1  fast=1
шаг 1:  slow=2  fast=3
шаг 2:  slow=3  fast=5
шаг 3:  slow=4  fast=7   → fast->next == NULL, цикл кончился
```

`slow` стоит на четвёртом узле — это и есть середина семи элементов.

Сложность. O(n) времени, O(1) памяти. Посчитаем точнее: для 7 узлов быстрый сделал 3 прыжка по 2 узла (6 переходов по `next`), медленный — 3 перехода, всего 9. Вариант «сначала посчитай длину, потом дойди до середины» делает 6 переходов на подсчёт и 3 на второй проход —

тоже 9. Так что по числу переходов выигрыша нет, оба около `1.5·n`. Трюк ценен другим: он работает за **один** проход, и его можно применить там, где второго прохода не будет (например, список достраивается на лету). На собесе скажи об этом честно — интервьюер оценит, что ты не повторяешь миф «один проход всегда быстрее двух».

Доп. вопросы.

- *А если узлов чётное число?* Тогда `slow` остановится на «правом из двух центральных»: для `1 2 3 4 5 6` это 4. Хочешь левый (3) — крути цикл, пока `fast->next != NULL && fast->next->next != NULL`. Я прогнал оба варианта на списках длиной от 1 до 8: на нечётных длинах они совпадают, на чётных дают правый и левый центр. Только не забудь отдельно проверить пустой список: в этой версии `fast->next` вычисляется сразу, и при `head == NULL` это разыменование `NULL`.
- *А пустой список?* В нашей версии `slow` останется `NULL`, вызывающий должен это проверить перед `->value`.

3б. Цикл в списке (черепаха и заяц, алгоритм Флойда)

Условие. Список может быть «закольцован»: последний узел указывает не на `NULL`, а на один из предыдущих. Обычный обход по такому списку зациклится навечно. Определи, есть ли в списке цикл.

Идея. Та же пара указателей. Заяц (`fast`) бежит вдвое быстрее черепахи (`slow`). Если список заканчивается `NULL` — заяц добежит до конца, цикла нет. Если есть кольцо — заяц никогда не выйдет, он начнёт бегать по кругу и рано или поздно **догонит** черепаху сзади. Как на круговой беговой дорожке: быстрый бегун обязательно настигнет медленного. Момент `slow == fast` и означает цикл.

Решение:

```c
/* Обнаружение цикла: черепаха и заяц (Флойд). 1 если цикл есть. */
int has_cycle(struct Node *head)
{
    struct Node *slow = head;
    struct Node *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {         /* заяц догнал черепаху */
            return 1;
        }
    }
```

```
    return 0;                       /* быстрый упёрся в NULL — цикла нет */
}
```

Объяснение. Сравниваем именно **указатели** (`slow == fast`), то есть адреса узлов, а не их значения: важно, что оба бегунка попали в один и тот же узел. Два разных узла могут хранить одинаковое число — это ничего не значит.

Почему они обязательно встретятся. Введём два числа. Пусть μ («мю») — сколько шагов от головы до первого узла кольца, а λ («лямбда») — длина кольца в узлах. В нашем примере ниже список `1 2 3 4 5 6 7`, и узел 7 смотрит на узел 3: μ = 2 (от 1 до 3 два шага), λ = 5 (узлы 3, 4, 5, 6, 7).

Через μ шагов черепаха входит в кольцо. Заяц к этому моменту уже внутри и стоит где-то на круге. Будем мерить, на сколько узлов заяц **отстаёт** от черепахи, если идти по кольцу вперёд: это число от 0 до λ − 1. За один шаг черепаха уходит на 1 узел, а заяц — на 2, то есть отставание уменьшается **ровно на 1**. Число, которое каждый шаг уменьшается на 1, не может перепрыгнуть через 0: за не больше чем λ − 1 шагов оно станет нулём, и указатели совпадут. Значит, встреча случится не позже, чем через μ + λ шагов от старта, — по времени это O(n), где n — число узлов. Я проверил это программой на всех списках до 40 узлов, для каждого возможного места закольцовки: встреча всегда наступала не позже шага μ + λ.

Проверка `slow == fast` стоит **после** сдвигов. Поставь её до — и цикл вернёт 1 сразу на старте, когда оба указателя ещё на голове.

3в. Где начинается цикл

Условие. Список закольцован. Найди узел, с которого начинается кольцо (в нашем примере — узел 3).

Идея. Вторая фаза того же алгоритма: после встречи поставь один указатель обратно в голову, второй оставь на месте встречи и двигай оба **по одному** узлу. Место, где они совпадут, — начало кольца.

Решение:

```
/* Начало цикла (вторая фаза Флойда) или NULL, если цикла нет. */
struct Node *cycle_start(struct Node *head)
{
    struct Node *slow = head;
    struct Node *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
```

```
        fast = fast->next->next;
        if (slow == fast) {          /* фаза 1: встретились внутри кольца */
            struct Node *p = head;   /* фаза 2: второй бегунок — в голову */
            while (p != slow) {      /* оба шагают по одному узлу */
                p = p->next;
                slow = slow->next;
            }
            return p;                /* встретились ровно в начале кольца */
        }
    }
    return NULL;
}
```

Почему это работает. Пусть до встречи черепаха сделала k шагов. Заяц за то же время сделал 2k шагов, то есть на k шагов больше. Оба стоят в одном узле, значит, эти лишние k шагов заяц потратил на целые круги по кольцу: k делится на λ без остатка.

Теперь посмотри на черепаху: она стоит в узле, до которого от головы k шагов. Сделаем ей ещё μ шагов — получится k + μ шагов от головы. Первые μ из них приводят к началу кольца, а оставшиеся k — это целое число кругов, после которых возвращаешься туда же. Значит, через μ шагов черепаха будет **в начале кольца**. Указатель `p`, стартовавший из головы, тоже через μ шагов окажется в начале кольца — по определению μ. Раньше они совпасть не могут: пока `p` не дошёл до кольца, он снаружи, а черепаха внутри. Вот они и встречаются ровно на первом узле кольца.

Проверим на нашем примере: μ = 2, λ = 5. Первая фаза: черепаха делает шаги 1→2→3→4→5→6, заяц 1→3→5→7→4→6 — встреча в узле 6 после k = 5 шагов (и 5 делится на λ = 5). Вторая фаза: `p` идёт 1→2→3, черепаха 6→7→3 — через 2 шага (это μ) оба в узле 3. Верно.

В программе мы сначала проверяем нормальный список (цикла нет), потом вручную закольцовываем его (последний узел направляем на третий) и проверяем снова. Полный вывод файла `20-03-floyd.c`:

```
Середина списка из 7: 4
Цикл есть? 0
После закольцовки цикл есть? 1
Цикл начинается в узле 3
```

Важная деталь из кода: перед `free` цикл надо **разорвать** (`last->next = NULL`). Иначе `free_list` после узла 7 перейдёт на уже освобождённый узел 3: сначала прочитает его поле `next` (use-after-free), потом освободит его второй раз (double free) — оба бага разобраны в каверзе 7.

Сложность. O(n) по времени (обе фазы — не больше μ + λ шагов каждая), O(1) по памяти. Альтернатива — хранить посещённые узлы в хеш-таблице: тоже O(n) времени в среднем, но O(n) памяти. На собесе стоит назвать оба варианта и объяснить, почему выбираешь Флойда.

Доп. вопросы.

- *Как узнать длину цикла?* После встречи оставь один указатель на месте и гони второй по кольцу по одному узлу, считая шаги до возврата. В нашем примере насчитаешь 5.
- *А если список длинный, но цикла нет — не зря ли мы гоняем зайца?* Нет: заяц дойдёт до `NULL` за n/2 прыжков, это те же O(n).

Задача 4. Биты: popcount и степень двойки

Два вопроса, которые идут парой и держатся на одном и том же приёме. Файл — `code/20-04-popcount.c`.

4a. Подсчёт единичных битов (popcount)

Условие. Дано целое число. Посчитай, сколько единиц в его двоичной записи. Например, 7 это `111` — три единицы; 8 это `1000` — одна.

Идея. Наивно: смотрим младший бит (`n & 1`), прибавляем его к счётчику, сдвигаем число вправо (`n >> 1`), повторяем, пока число не обнулится. Это делает столько шагов, сколько цифр в двоичной записи числа — до старшей единицы включительно. У `1024` (`10000000000`) это 11 шагов, у `0xFFFFFFFF` — все 32. Красивее — **приём Брайана Кернигана**: операция `n &= n - 1` за один раз **гасит самый младший единичный бит**. Тогда число итераций равно числу единиц, а не общему числу битов.

Почему `n & (n - 1)` убирает младшую единицу? Вычитание единицы переворачивает младший единичный бит в 0, а все нули правее него — в единицы. Например, `n = 1100`, `n - 1 = 1011`. Их `&` даёт `1000` — младшая единица исчезла, старшие целы:

```
  1100   n
  1011   n - 1
& ————
  1000   n & (n - 1)
```

Решение:

```c
/* Наивный popcount: проверяем младший бит и сдвигаем вправо. */
int popcount_naive(unsigned int n)
{
    int count = 0;
```

```c
    while (n != 0) {
        count += n & 1u;    /* младший бит: 0 или 1 */
        n >>= 1;            /* сдвинули число на один бит вправо */
    }
    return count;
}

/* Приём Брайана Кернигана: n &= n - 1 гасит младший единичный бит. */
int popcount_kernighan(unsigned int n)
{
    int count = 0;
    while (n != 0) {
        n &= n - 1;         /* убрали одну единицу */
        count++;            /* итераций ровно столько, сколько единиц */
    }
    return count;
}
```

Объяснение.

- Аргумент — `unsigned int` (беззнаковое). Битовые трюки и сдвиги надёжнее всего делать на беззнаковых типах. У знаковых есть две ямы (стандарт C11, раздел 6.5.7): сдвиг вправо отрицательного числа — implementation-defined, то есть каждый компилятор сам решает, тянуть ли знаковый бит, и описывает это в документации; а сдвиг **влево** отрицательного числа — UB. С `unsigned` таких вопросов нет: всё определено, а выход за границу честно заворачивается по модулю 2 в степени N, где N — число битов типа. У `unsigned int` на этом маке N = 32 (стандарт гарантирует только «не меньше 16»), то есть по модулю 2^{32} = 4 294 967 296.
- `n & 1u` — побитовое И с единицей оставляет только младший бит: результат 0 или 1. Суффикс `u` у литерала `1u` подчёркивает, что работаем с беззнаковым, и убирает лишние вопросы о приведении типов.
- `n >>= 1` — то же, что `n = n >> 1`: сдвиг всех битов на одну позицию вправо, как деление на 2 без остатка.
- `n &= n - 1` — то же, что `n = n & (n - 1)`: гасим младшую единицу.

Запуск (файл гоняет обе функции и заодно проверку из пункта 4б):

```
    0: бит=0 (Керниган=0), степень_двойки=0
    1: бит=1 (Керниган=1), степень_двойки=1
    2: бит=1 (Керниган=1), степень_двойки=1
```

```
    7: бит=3 (Керниган=3), степень_двойки=0
    8: бит=1 (Керниган=1), степень_двойки=1
  255: бит=8 (Керниган=8), степень_двойки=0
 1024: бит=1 (Керниган=1), степень_двойки=1
```

Обе функции дают одно число, но версия Кернигана для числа `1024` (это `10000000000`) сделает **один** виток цикла, а наивная — одиннадцать.

Сложность. Наивная делает столько итераций, сколько двоичных цифр в числе (до старшей единицы): от 0 для нуля до 32 для чисел с выставленным старшим битом. Керниган делает столько итераций, сколько в числе единиц: тоже от 0 до 32, но никогда не больше наивной. На «разреженных» числах вроде `1024` разница заметна (1 против 11), на `0xFFFFFFFF` обе делают 32 шага.

Доп. вопросы.

- *Есть ли это в железе?* Да, у многих процессоров есть инструкция popcount; в GCC/Clang к ней даёт доступ встроенная функция `__builtin_popcount` (это расширение компилятора, не стандартный Си). Я посмотрел, во что clang на этом маке превращает её с `-O2`: без дополнительных флагов — в десяток сдвигов и масок, а с флагом `-mpopcnt` — в одну инструкцию `popcntl`. Работает она в обоих случаях.
- *Как посчитать единицы в `unsigned long long`?* Тот же код, только параметр типа `unsigned long long` и литерал `1ull`. Счётчик `int` спокойно вмещает 64.

4б. Степень ли двойки

Условие. Проверь, является ли число степенью двойки (1, 2, 4, 8, 16, ...).

Идея. У степени двойки в двоичной записи **ровно один** единичный бит: `1`, `10`, `100`, `1000`. А мы только что видели, что `n & (n - 1)` гасит младшую единицу. Если единица была одна-единственная — после гашения останется 0. Значит, проверка: `n != 0 && (n & (n - 1)) == 0`.

Решение:

```c
/* Степень двойки: ровно один единичный бит, и число положительное. */
int is_power_of_two(unsigned int n)
{
    return n != 0 && (n & (n - 1)) == 0;
}
```

Объяснение. Условие `n != 0` обязательно. Без него ноль прошёл бы проверку: `0 - 1` для беззнакового заворачивается в самое большое значение типа, `UINT_MAX` (на этом маке `0xFFFFFFFF`),

и `0 & 0xFFFFFFFF` равно 0, так что `is_power_of_two(0)` ошибочно вернуло бы «истина». Ноль не степень двойки, поэтому отсекаем его явно.

Результат функции — значение логического выражения: в Си это честные 1 (истина) или 0 (ложь), поэтому можно смело писать `return` без всякого `if`.

Сложность. O(1) — несколько операций, без циклов.

Доп. вопросы.

- *Почему не считать popcount и не сравнивать с 1?* Можно, но это цикл вместо трёх операций.
- *А для знакового `int`?* Тогда сначала отсекай `n <= 0`. Главная ловушка — `INT_MIN` (−2147483648): в дополнительном коде это ровно один выставленный бит, старший, и «беззнаковый» трюк объявил бы его степенью двойки. А в знаковом `int` вычисление `n - 1` для `INT_MIN` — переполнение, то есть UB. Условие `n > 0 && (n & (n - 1)) == 0` закрывает обе дыры: до `n - 1` дело доходит только для положительных.
- *Как округлить до ближайшей степени двойки вверх?* Серия сдвигов и `|`, либо инструкция подсчёта ведущих нулей (`__builtin_clz`).

Задача 5. Корректный бинарный поиск

Условие. Реализуй бинарный поиск в отсортированном массиве. Главная ловушка — ошибки «на единицу» (off-by-one): неверная граница цикла или неверный сдвиг `lo`/`hi` ведут к зацикливанию или пропуску элемента.

Идея. Держим отрезок `[lo, hi]`, где может находиться искомое. Смотрим середину. Угадали — готово. Если в середине меньше искомого — всё левее бесполезно, двигаем `lo`. Если больше — двигаем `hi`. Отрезок сжимается, пока не найдём или пока он не опустеет.

Решение (`code/20-05-binsearch.c`, ядро):

```c
/* Бинарный поиск в отсортированном массиве.
   Возвращает индекс target или -1, если не найдено. */
int binary_search(const int *a, int n, int target)
{
    int lo = 0;
    int hi = n - 1;             /* оба конца ВКЛЮЧИТЕЛЬНО */
    while (lo <= hi) {          /* пока отрезок непустой */
        int mid = lo + (hi - lo) / 2;   /* середина без переполнения */
        if (a[mid] == target) {
            return mid;
        } else if (a[mid] < target) {
```

```
            lo = mid + 1;       /* искомое правее */
        } else {
            hi = mid - 1;       /* искомое левее */
        }
    }
    return -1;
}
```

Объяснение. Три места, где обычно ошибаются, и как мы их обходим:

- **Границы включительные.** `hi = n - 1`, и цикл идёт, пока `lo <= hi` (не <). Когда `lo == hi`, отрезок ещё содержит один элемент — его нужно проверить. Напишешь `lo < hi` — пропустишь этот последний элемент, и поиск будет иногда врать «не найдено».
- **Сдвиг с +1/−1.** После проверки середины сам `mid` уже исключён, поэтому новые границы — `mid + 1` и `mid - 1`, а не `mid`. Если сдвигать на `mid`, то на отрезке из двух элементов `mid` всё время равен `lo`, границы не двигаются и цикл виснет навсегда.
- **Середина без переполнения.** Пишем `lo + (hi - lo) / 2`, а не `(lo + hi) / 2`. На больших индексах сумма `lo + hi` может переполнить `int` — а это UB, и обычно результат становится отрицательным, что даёт `a[-1]` и выход за границы. Разность `hi - lo` неотрицательна и безопасна. Классический баг: он много лет жил в `Arrays.binarySearch` стандартной библиотеки Java, пока его не исправили в 2006 году.

Пустой массив обрабатывается сам собой: `n = 0` даёт `hi = -1`, условие `0 <= -1` ложно, вернём −1.

Запуск (массив `{-7, -3, 0, 2, 5, 8, 11, 17, 42}`):

```
   5 найдено на индексе 4
  42 найдено на индексе 8
  -7 найдено на индексе 0
 100 не найдено
   0 найдено на индексе 2
```

Обрати внимание на границы: находятся и первый элемент (−7), и последний (42) — это ровно те две позиции, которые чаще всего теряются при off-by-one. А 100, которого в массиве нет, честно даёт «не найдено», а не индекс наугад.

Сложность. O(log n): каждый шаг вдвое урезает отрезок. Точнее, в худшем случае цикл делает $\lfloor \log_2 n \rfloor + 1$ итераций (формулу и логарифм мы разбирали в главе 7). Для миллиона элементов: $2^{19} = 524\,288 \le 1\,000\,000 < 2^{20}$, значит $\lfloor \log_2 n \rfloor = 19$ и итераций не больше 20. Для миллиарда: $2^{29} \le 10^9 < 2^{30}$, итераций не больше 30. Память O(1).

Доп. вопросы.

- *Что вернуть, если в массиве есть дубликаты искомого?* Базовая версия вернёт «какой-то» из них. Чтобы найти первый или последний, цикл меняют так, чтобы он не останавливался на совпадении, а продолжал сужать сторону.
- *А если массив не отсортирован?* Алгоритм тихо вернёт мусор. Бинарный поиск не проверяет отсортированность — это предусловие, о котором надо сказать вслух.
- *Есть ли готовая функция?* Есть `bsearch` в `<stdlib.h>` — принимает компаратор, как `qsort` (см. главу 8).

Задача 6. Слияние двух отсортированных массивов

Условие. Даны два отсортированных массива. Слей их в один отсортированный. Это сердце сортировки слиянием (merge sort).

Идея. Идём по обоим массивам сразу, двумя индексами. Сравниваем текущие элементы и в результат кладём **меньший**, продвигая его индекс. Так — пока один из массивов не кончится. Остаток второго дописываем как есть: он уже отсортирован.

Решение (`code/20-06-merge.c`, ядро):

```c
/* Слияние двух отсортированных массивов a и b в результат out.
   out должен вмещать na + nb элементов. */
void merge(const int *a, int na, const int *b, int nb, int *out)
{
    int i = 0, j = 0, k = 0;
    while (i < na && j < nb) {       /* пока есть в обоих */
        if (a[i] <= b[j]) {
            out[k++] = a[i++];       /* меньший идёт первым */
        } else {
            out[k++] = b[j++];
        }
    }
    while (i < na) {                 /* хвост a, если остался */
        out[k++] = a[i++];
    }
    while (j < nb) {                 /* хвост b, если остался */
        out[k++] = b[j++];
    }
}
```

Объяснение. Запись `out[k++] = a[i++]` делает три вещи разом: кладёт `a[i]` в `out[k]`, увеличивает `i` и увеличивает `k`. Постфиксный `++` сначала отдаёт старое значение, а потом увеличивает переменную — поэтому индексы в самом присваивании ещё старые (подробный разбор всех четырёх форм `++` с указателями — в каверзе 4).

Сравнение `a[i] <= b[j]` именно «меньше или равно», а не «меньше». Это делает слияние **устойчивым** (стабильным): при равенстве берём элемент из первого массива, и равные сохраняют исходный относительный порядок. Для голых чисел разницы не видно, но если бы мы сливали записи с полями (фамилия + возраст), стабильность стала бы заметна сразу.

Два хвостовых цикла дописывают то, что осталось. Сработает ровно один из них: после первого цикла хотя бы один массив исчерпан.

Ответственность за размер `out` — на вызывающем: там должно быть место под `na + nb` элементов. В `main` массив `out[11]` именно такой (5 + 6 = 11). Если ошибиться на единицу, получишь запись за границу массива — тихое UB, которое поймает только санитайзер.

Запуск (`{1,3,5,7,9}` и `{2,4,6,8,10,12}`):

```
1 2 3 4 5 6 7 8 9 10 12
```

Сложность. O(na + nb) по времени — каждый элемент копируем ровно один раз, а сравнений не больше na + nb − 1: каждое сравнение отправляет в `out` один элемент, а последний элемент всегда уходит без сравнения. Память — под результат, O(na + nb).

Можно ли слить на месте, без второго массива, и всё равно за линейное время? Теоретически да: такие алгоритмы есть (первый опубликовал Кронрод в 1969 году), но они настолько сложны и у них такие большие постоянные множители, что на собеседовании достаточно сказать «существуют, но обычно берут буфер».

Доп. вопросы.

- *Как слить «на месте» в конец первого массива (классическая задача merge sorted array)?* Идти **с конца** обоих массивов, заполняя результат справа налево — тогда не затрёшь ещё не прочитанные элементы.
- *А если массивов k штук?* Тогда берут кучу (min-heap) из k голов: O(N log k) вместо O(N · k).

Задача 7. Удалить дубликаты из отсортированного массива на месте

Условие. Дан отсортированный массив, например `[1, 1, 2, 2, 2, 3, 5, 5, 8]`. Убери повторы **на месте**, не выделяя второй массив. Верни новую длину; уникальные элементы должны оказаться в начале.

Идея. Раз массив отсортирован, все одинаковые элементы лежат рядом. Заведём два индекса (приём «два указателя» в одном массиве): `r` (reader) бежит по всем элементам, `w` (writer) показы-

вает, куда положить следующий уникальный. Если `a[r]` отличается от последнего записанного `a[w-1]` — это новый уникальный, копируем его в `a[w]` и двигаем `w`.

Решение (`code/20-07-dedup.c`, ядро):

```c
/* Удаление дубликатов из ОТСОРТИРОВАННОГО массива на месте.
   Возвращает новую длину; уникальные элементы — в начале a. */
int dedup_sorted(int *a, int n)
{
    if (n == 0) {
        return 0;
    }
    int w = 1;                  /* w — куда писать следующий уникальный */
    for (int r = 1; r < n; r++) {   /* r — что читаем */
        if (a[r] != a[w - 1]) {     /* отличается от последнего записанного */
            a[w] = a[r];
            w++;
        }
    }
    return w;
}
```

Объяснение. Первый элемент `a[0]` уникален всегда, поэтому стартуем с `w = 1` и `r = 1`.

Сравниваем именно с `a[w - 1]` — последним, что мы уже оставили, а не с соседом `a[r - 1]`. На серии `1 1 1 2` разницы нет, но общая привычка сравнивать с «последним записанным» переносится на задачи, где допускают не более k повторов, и там `a[r-1]` уже не годится.

Проверка `n == 0` нужна, иначе `a[w - 1]` при пустом массиве полез бы в `a[-1]` — выход за границу и UB.

Прогоним `{1, 1, 2, 2, 2, 3, 5, 5, 8}` руками:

```
r=1  a[1]=1 == a[0]=1   пропуск          w=1
r=2  a[2]=2 != a[0]=1   a[1]=2           w=2
r=3  a[3]=2 == a[1]=2   пропуск          w=2
r=4  a[4]=2 == a[1]=2   пропуск          w=2
r=5  a[5]=3 != a[1]=2   a[2]=3           w=3
r=6  a[6]=5 != a[2]=3   a[3]=5           w=4
r=7  a[7]=5 == a[3]=5   пропуск          w=4
r=8  a[8]=8 != a[3]=5   a[4]=8           w=5
```

Запуск:

```
Уникальных: 5
1 2 3 5 8
```

Сложность. O(n) времени, O(1) дополнительной памяти — записи идут в тот же массив, и `w` никогда не обгоняет `r`, поэтому мы не затираем непрочитанное.

Доп. вопросы.

- *Почему важно, что массив отсортирован?* Только тогда дубликаты рядом и хватает сравнения с предыдущим уникальным. Для неотсортированного понадобится множество (хеш) — O(n) памяти, либо сначала сортировка за O(n log n).
- *Что в хвосте массива после удаления?* Старый мусор за позицией `w`. Мы его не трогаем — вызывающий пользуется только первыми `w` элементами. Если это важно, хвост можно занулить.
- *А если разрешено оставлять не более двух повторов?* Тогда сравнивай с `a[w - 2]` и стартуй с `w = 2` и `r = 2`, а массив из двух и меньше элементов возвращай как есть (`if (n <= 2) return n;`). На `{1, 1, 1, 2, 2, 2, 3}` получится 5 элементов: `1 1 2 2 3`.

Задача 8. Число, встречающееся нечётное число раз (XOR)

Условие. В массиве все числа встречаются **парами**, и только одно — нечётное число раз. Найди его. Желательно за один проход и без лишней памяти.

Идея. Спасает операция XOR (исключающее ИЛИ, `^`). Она сравнивает числа бит за битом: в результате стоит 1 там, где биты **разные**, и 0 там, где одинаковые. Например, `6 ^ 3`: `110 ^ 011 = 101`, то есть 5. У XOR два золотых свойства: `x ^ x == 0` (у числа с самим собой все биты одинаковые — одни нули) и `x ^ 0 == x` (ноль ничего не меняет). Проксорим **все** элементы подряд. Парные погасят друг друга и дадут 0, а одиночка останется — это и есть ответ. Порядок неважен: XOR коммутативен и ассоциативен, поэтому пары «найдут друг друга» независимо от того, где они лежат.

Решение (`code/20-08-oddtimes.c`, ядро):

```c
/* Число, встречающееся нечётное число раз.
   XOR всех элементов: парные гасят друг друга, остаётся одиночка. */
int find_odd(const int *a, int n)
{
    int x = 0;
    for (int i = 0; i < n; i++) {
        x ^= a[i];      /* a ^ a == 0, x ^ 0 == x */
    }
```

```
    return x;
}
```

Объяснение. `x ^= a[i]` — то же, что `x = x ^ a[i]`. Стартуем с `x = 0`, потому что 0 — нейтральный элемент для XOR (как 0 для сложения и 1 для умножения). После прохода все, кто шёл парами, взаимно уничтожились, в `x` лежит число с непарным количеством.

XOR даёт 1 в тех битах, где операнды различаются. Поэтому `x ^ x` — это все нули, а `x ^ 0` оставляет `x` как есть.

Мелочь, о которой спросят: XOR на **знаковых** `int` работает с битами представления числа. Здесь это безопасно: на всех современных машинах отрицательные числа хранятся в дополнительном коде (в стандарте C23 это стало обязательным), и XOR двух настоящих `int` всегда даёт настоящий `int`. Переполнения у XOR не бывает в принципе: он не переносит единицы из разряда в разряд, как сложение.

Запуск (в массиве `4` встречается 3 раза, остальные парами):

```
Нечётное число раз встречается: 4
```

Сложность. O(n) времени, O(1) памяти — одна переменная. Наивный подход «для каждого посчитать вхождения» — $O(n^2)$, подход через хеш — O(n) времени и O(n) памяти. XOR выигрывает у обоих.

Доп. вопросы.

- *А если два числа непарные?* Тогда общий XOR даст их «смесь». Их можно разделить: взять любой выставленный бит в этой смеси (например, младший), разбить массив на две группы по этому биту и проксорить каждую отдельно. Младший бит достают выражением `x & -x`, но считай его в `unsigned`: `(unsigned)x & -(unsigned)x`. В знаковом `int` при `x == INT_MIN` вычисление `-x` — переполнение, то есть UB. Решение — в ответах к тренировочной задаче 5.
- *А если числа встречаются по три раза, кроме одного?* XOR уже не спасёт — там считают биты по модулю 3.
- *Где XOR ещё полезен?* Обмен двух чисел без временной переменной, простое шифрование, контрольные суммы, RAID-массивы.

Задача 9. Поменять местами два числа через указатели

Отдельного файла у этой задачи нет — она в пять строк и целиком здесь.

Условие. Напиши функцию `swap`, которая меняет местами значения двух переменных. Так, чтобы изменения были видны **снаружи** функции.

Идея. В Си аргументы передаются по значению — функция получает копии (см. главу 3). Поменяв копии, оригиналы не тронешь. Чтобы дотянуться до самих переменных, передаём их **адреса** и работаем через указатели.

Решение:

```c
#include <stdio.h>

void swap(int *a, int *b)   /* принимаем адреса двух переменных */
{
    int t = *a;             /* запомнили значение по a */
    *a = *b;                /* в a положили значение по b */
    *b = t;                 /* в b положили запомненное */
}

int main(void)
{
    int x = 5, y = 9;
    printf("до:    x=%d y=%d\n", x, y);
    swap(&x, &y);           /* передаём АДРЕСА x и y */
    printf("после: x=%d y=%d\n", x, y);
    return 0;
}
```

Запуск:

```
до:    x=5 y=9
после: x=9 y=5
```

Объяснение. `int *a` — указатель: `a` хранит адрес чужой переменной. `*a` — значение по этому адресу, то есть сама переменная вызывающего. Меняя `*a` и `*b`, мы меняем `x` и `y` в `main`. При вызове пишем `&x` и `&y` — берём адреса.

Именно поэтому `scanf` тоже хочет `&`: ему нужен адрес, чтобы записать в твою переменную. Забытая `&` в `scanf` — это передача **значения** там, где ждали адрес. Это UB: на практике `scanf` истолкует мусорное число как адрес и полезет писать по случайному месту в памяти. С `-Wall` clang ловит это ещё при сборке: `format specifies type 'int *' but the argument has type 'int'`.

Доп. вопросы.

- *Почему версия `void swap(int a, int b)` не работает?* Она меняет местами копии; снаружи всё остаётся как было. Компилятор не ругнётся — код формально корректен, просто бесполезен.
- *А как поменять местами два указателя?* Тогда нужен указатель на указатель: `void swap_ptr(int **a, int **b)`.
- *Можно ли без временной переменной, через XOR?* Можно: `*a ^= *b; *b ^= *a; *a ^= *b;`. Но есть подвох: если `a` и `b` указывают на **одну** переменную, она обнулится (я проверил: `xswap(&x, &x)` при `x = 9` оставляет 0) — первый же `*a ^= *b` превращает её в `x ^ x`, то есть в ноль. Версия с временной `t` надёжнее и читается лучше — на собеседовании предпочитают её.

Часть 2. Каверзы

Эти вопросы не про «реши задачу», а про «понимаешь ли ты язык». Любимая территория собеседующих: ответ короткий, но за ним стоит понимание памяти, указателей и неопределённого поведения. Решений в `code/` для них нет — некоторые специально содержат ошибку, и класть такое в файлы книги как «образец» нельзя.

Каверза 1. `char *s = "abc"` против `char s[] = "abc"`

```c
char *s1 = "abc";       /* указатель на строковый ЛИТЕРАЛ */
char  s2[] = "abc";     /* МАССИВ-копия, лежит у тебя */

s2[0] = 'A';            /* ок: меняем свой массив, выйдет "Abc" */
s1[0] = 'A';            /* НЕЛЬЗЯ: запись в литерал — UB */
```

В чём разница. `s2` — это **массив**: компилятор выделил тебе личные 4 байта (`a`, `b`, `c`, `\0`) и скопировал туда содержимое литерала. Твой массив — меняй на здоровье.

`s1` — это **указатель** на строковый литерал `"abc"`. Стандарт C11 (раздел 6.4.5) говорит коротко: попытка изменить строковый литерал — **неопределённое поведение** (undefined behavior, UB), то есть стандарт не обещает про исход вообще ничего. Где именно лежит литерал, стандарт не диктует; на macOS и Linux компилятор кладёт его в область памяти только для чтения.

И вот это стоит увидеть своими глазами, потому что «не обещает ничего» звучит абстрактно. Вот программа целиком:

```c
#include <stdio.h>


int main(void)
```

```
{
    char *s1 = "abc";
    char s2[] = "abc";

    s2[0] = 'A';
    printf("s2 = %s, sizeof s2 = %zu\n", s2, sizeof s2);
    printf("сейчас попробуем записать в литерал...\n");
    fflush(stdout);         /* вытолкнуть текст до возможного падения */
    s1[0] = 'A';
    printf("s1 = %s\n", s1);
    return 0;
}
```

`fflush(stdout)` заставляет `printf` сразу отдать накопленный текст в терминал: если программа упадёт, строка уже будет на экране. Соберём этот файл двумя способами, а код выхода покажем командой `echo $?` сразу после запуска.

Без оптимизации (`-O0`):

```
s2 = Abc, sizeof s2 = 4
сейчас попробуем записать в литерал...
[код выхода: 138]
```

Программа умерла на строке `s1[0] = 'A'`. Код 138 — это 128 плюс номер сигнала 10, то есть `SIGBUS` (на macOS у него номер 10): запись в память, которая открыта только на чтение. Ожидаемое падение.

Тот же файл с `-O2`:

```
s2 = Abc, sizeof s2 = 4
сейчас попробуем записать в литерал...
s1 = abc
[код выхода: 0]
```

Ничего не упало. И — приглядись — `s1` остался `abc`, хотя мы вроде бы записали туда `'A'`. Оптимизатор рассуждает так: «запись в литерал — UB, значит, по стандарту такого случиться не может, значит, эту строку можно считать несуществующей». Он её просто выбросил.

Вот это и есть самое опасное лицо UB. Не падение — падение как раз подарок, ты сразу видишь проблему. Опасно, когда программа с UB **молча делает не то**, и поведение меняется от флагов сборки: отладил на `-O0`, выкатил на `-O2` — и получил другой продукт. Полагаться нель-

зя ни на один из двух исходов. Отвечая на собесе, так и говори: «это UB; я видел и падение, и молчаливое игнорирование записи — зависит от уровня оптимизации».

Правило. Хочешь менять строку — заводи массив (`char s[]`). Указатель на литерал годится только для чтения; чтобы не забыть, объявляй его `const char *s1 = "abc";` — тогда компилятор сам запретит запись ещё на этапе сборки, и до UB дело не дойдёт.

Ещё одна разница, про которую спрашивают. `sizeof s2` — это 4 (размер массива), а `sizeof s1` — 8 (размер указателя на этом 64-битном маке). И `strlen` у обоих 3. Не путай размер хранилища с длиной строки.

Каверза 2. `sizeof` массива внутри функции

```c
#include <stdio.h>

void f(int arr[])                /* это int *arr в маскировке! */
{
    printf("внутри функции sizeof arr = %zu\n", sizeof arr);
}

int main(void)
{
    int a[10];
    printf("в main sizeof a = %zu\n", sizeof a);
    f(a);
    return 0;
}
```

В чём подвох. В месте объявления `sizeof a` знает настоящий размер массива — 40 байт (10 чисел по 4 байта; `int` здесь 4-байтный). Но при передаче в функцию массив **распадается** (decays) в указатель на первый элемент. Параметр `int arr[]` — это синтаксическая маскировка под `int *arr`; даже число в скобках (`int arr[10]`) компилятор проигнорирует. Поэтому внутри функции `sizeof arr` — это размер **указателя**, 8 байт на 64-битной системе.

Проверка на живой программе даёт ровно это:

```
в main sizeof a = 40
внутри функции sizeof arr = 8
```

Причём компилятор про такую ошибку знает и сам предупреждает — ещё один довод всегда собирать с `-Wall`:

```
warning: sizeof on array function parameter will return size of
'int *' instead of 'int[]' [-Wsizeof-array-argument]
    5 |     printf("внутри функции sizeof arr = %zu\n", sizeof arr);
      |                                                        ^
note: declared here
    3 | void f(int arr[])
      |            ^
```

Следствие. Внутри функции по самому массиву его длину не узнать. Поэтому длину **всегда передают отдельным параметром**: `f(int *arr, int n)`. Так устроены все наши функции в этом приложении. Это же объясняет, почему строкам нужна `strlen`: длину массива `char` иначе не вычислить, спасает только метка `'\0'`.

Что ещё спросят. «А если параметр объявить как `int (*arr)[10]` — указатель на массив из 10 элементов?» Тогда `sizeof *arr` даст честные 40. Приём рабочий, но редкий: он жёстко фиксирует длину.

Каверза 3. `arr[1]`, `1[arr]` и `*(arr + 1)`

```c
int arr[] = {10, 20, 30};
printf("%d\n", arr[1]);     /* 20 */
printf("%d\n", *(arr + 1)); /* 20 — то же самое */
printf("%d\n", 1[arr]);     /* 20 — и это тоже!? */
```

Запуск подтверждает — все три строки печатают одно:

```
20
20
20
```

Объяснение. По определению языка `arr[i]` — это **ровно** `*(arr + i)`: «возьми адрес начала, прибавь `i`, разыменуй». Сложение коммутативно: `arr + i` и `i + arr` — одно и то же. Значит, `*(arr + i)` равно `*(i + arr)`, а это по тому же правилу записывается как `i[arr]`. Поэтому `1[arr]` — легальный (хоть и дикий) способ написать `arr[1]`.

`arr + 1` — это **арифметика указателей**: «следующий элемент», то есть адрес сдвигается не на 1 байт, а на размер одного элемента (`sizeof(int)`, здесь 4). Компилятор сам умножает на размер типа. Если бы это был массив `double`, шаг был бы 8 байт — но в коде это выглядит одинаково.

Зачем это знать. На собеседовании этим проверяют, понимаешь ли ты, что индексация — синтаксический сахар над арифметикой указателей. Писать `1[arr]` в реальном коде, разумеется, нельзя — это запутывает.

Отсюда же следует ответ на другой частый вопрос: почему индексы в Си начинаются с нуля? Потому что `arr[0]` — это `*(arr + 0)`, то есть сам адрес начала, без сдвига. Ноль тут не соглашение, а арифметика.

Каверза 4. `++*p`, `*p++`, `*++p`, `(*p)++`

Четыре похожих записи, которые делают **разное**. Разберём по очереди. Пусть:

```
int b[] = {10, 20, 30};
int *p = b;             /* p смотрит на b[0] (значение 10) */
```

- **`++*p`** — это `++(*p)`. Разыменуй `p` (получим `b[0]`) и сделай префиксный инкремент значения. Результат: `b[0]` стало 11, `p` не двигался. Выражение даёт 11.
- **`(*p)++`** — постфиксный инкремент **значения**, на которое смотрит `p`. `b[0]` станет 11, но выражение вернёт **старое** значение 10. `p` на месте.
- **`*p++`** — здесь приоритет у постфиксного `++`: это `*(p++)`. Сначала берётся текущий `p` (адрес `b[0]`), потом `p` сдвигается на следующий элемент. Звёздочка разыменует **старый** адрес. Итог: выражение даёт 10 (значение `b[0]`), а `p` теперь смотрит на `b[1]`. Значения в массиве не менялись. Эта запись — рабочая лошадка при проходе по строке: `while (*p++)`.
- **`*++p`** — сначала префиксный `++p` (сдвигаем `p` на `b[1]`), потом разыменование. Выражение даёт 20, `p` смотрит на `b[1]`.

Все четыре, запущенные по очереди (каждый раз с чистым массивом `{10, 20, 30}` и `p`, возвращённым на начало):

```
++*p   -> выражение 11, b[0]=11, p смотрит на b[0]
(*p)++ -> выражение 10, b[0]=11, p смотрит на b[0]
*p++   -> выражение 10, b[0]=10, p смотрит на b[1]
*++p   -> выражение 20, b[0]=10, p смотрит на b[1]
```

Сравни первые две строки: обе меняют `b[0]` на 11, но `++*p` отдаёт наружу новое значение, а `(*p)++` — старое. И сравни вторую пару: там массив не тронут вовсе, двигается только указатель.

Как не путаться. Запомни приоритеты: постфиксный `++` цепляется к `p` сильнее, чем `*`. Поэтому `*p++` — это «разыменуй, а указатель сдвинь», а не «сдвинь значение». Если хочешь менять **значение** — ставь скобки: `(*p)++`. На собеседовании именно скобки показывают, что ты понимаешь, что делаешь.

И отдельное предупреждение: в одном выражении такие штуки не смешивают. `*p++ = *q++` — нормальная идиома, потому что каждая переменная меняется один раз. А вот `p[i++] = i++` — уже UB (см. Приложение А, каверза 7).

Каверза 5. `const char *p` против `char * const p`

```c
char a = 'a', b = 'b';

const char *p1 = &a;    /* нельзя менять *p1, можно менять p1 */
/* *p1 = 'x';  -- ошибка компиляции */
p1 = &b;                /* ок: указатель переставили */

char * const p2 = &a;   /* можно менять *p2, нельзя менять p2 */
*p2 = 'x';              /* ок: значение поменяли (a стало 'x') */
/* p2 = &b;  -- ошибка компиляции */
```

Как читать. Правило простое: читай объявление **справа налево от имени**.

- `const char *p1` → «`p1` — указатель на `const char`». Под запретом **то, на что смотрим** (`*p1`). Сам указатель свободно переставляется.
- `char * const p2` → «`p2` — `const`-указатель на `char`». Под запретом **сам указатель** (его нельзя перенаправить). А значение по нему менять можно.

Если раскомментировать обе запрещённые строки, компилятор скажет ровно то, чего мы ждём:

```
error: read-only variable is not assignable
    6 |     *p1 = 'x';
      |     ~~~ ^
error: cannot assign to variable 'p2' with const-qualified type
'char *const'
   11 |     p2 = &b;
      |     ~~ ^
note: variable 'p2' declared const here
```

Заметь: это **ошибки**, а не предупреждения. `const` — не совет компилятору, а запрет.

Зачем. `const char *` — самая частая форма для параметров-строк, которые функция только читает (как в нашей `my_strlen`). Такой параметр сразу говорит вызывающему: «твою строку не испорчу». А бывает и `const char * const p` — нельзя ни то, ни другое.

И важная тонкость: `const` защищает от **случайной** правки, а не от злонамеренной. Приведением типа `(char *)p1` его можно снять, и компилятор смолчит. Но если сам объект был объявлен как `const` (например, `const char c = 'a';`), запись в него через такой указатель — UB по стандарту (C11, 6.7.3). А если объект вдобавок лежит в памяти только для чтения, запись упадёт, как в каверзе 1.

Каверза 6. Найди баг: утечка памяти

```c
#include <stdio.h>
#include <stdlib.h>

char *make_greeting(const char *name)
{
    char *buf = malloc(64);
    if (buf == NULL) return NULL;
    snprintf(buf, 64, "Привет, %s!", name);
    return buf;
}

int main(void)
{
    printf("%s\n", make_greeting("Аня"));   /* БАГ: утечка */
    return 0;
}
```

Где баг. `make_greeting` выделяет память через `malloc` и возвращает указатель. Но в `main` мы передаём этот указатель сразу в `printf` и **теряем** его — переменной, которая хранила бы адрес, нет. Освободить такой блок уже нечем: `free` вызвать не от чего. Это **утечка памяти**: блок занят до конца программы.

Второй, менее заметный баг: `make_greeting` может вернуть `NULL` (если памяти не хватило), а `printf("%s", NULL)` — это UB. Проверять надо и результат `malloc` внутри функции, и результат самой функции снаружи.

Как чинить. Сохрани адрес, проверь, попользуйся, освободи:

```c
char *msg = make_greeting("Аня");
if (msg != NULL) {
    printf("%s\n", msg);
    free(msg);              /* на каждый malloc — свой free */
}
```

Правило из главы 3 в полный рост: на каждый `malloc` однажды должен прийтись ровно один `free`. Если функция **возвращает** выделенную память, это надо оговаривать в её документации или прямо в имени (`..._alloc`, `..._new`, `..._dup`) — иначе вызывающий не догадается, что теперь он в ответе за освобождение. Ровно так устроена `strdup`.

Как ловить. Утечку не видно ни глазом, ни по выводу программы — только инструментами:

```
cc -std=c11 -Wall -Wextra -g -fsanitize=address prog.c -o prog
```

На Linux адресный санитайзер сам сообщит про leak в конце работы. На macOS с процессором Intel поиск утечек в санитайзере не работает — я проверил, программа под ним молча печатает приветствие и выходит с кодом 0. Зато на macOS есть системная утилита `leaks`. Собери программу без санитайзера и запусти так: `leaks -atExit -- ./prog`. В конце её отчёта будет строка:

```
Process 59853: 1 leak for 80 total leaked bytes.
```

Одна утечка. Байтов 80, а не 64: `leaks` считает размер блока, который аллокатор реально отвёл под запрос, а он бывает чуть больше запрошенного. Номер процесса у тебя будет другим.

Каверза 7. Найди баг: use-after-free и двойной free

```c
int *p = malloc(sizeof(int));
*p = 42;
free(p);                /* блок вернули аллокатору */

printf("%d\n", *p);     /* БАГ 1: use-after-free — читаем мёртвое */
free(p);                /* БАГ 2: double free — освобождаем дважды */
```

Что не так. После `free(p)` блок больше не наш: `free` отдаёт его аллокатору (той части библиотеки, что обслуживает `malloc`), и тот вправе выдать его следующему `malloc`. Операционной системе память при этом обычно не возвращается. Указатель `p` всё ещё хранит старый адрес (он стал «висячим», dangling), но трогать память по нему нельзя:

- `*p` после `free` — **use-after-free**: читаем (или пишем) освобождённую память. UB: может показать старое значение, мусор или уронить программу. Хуже всего первый вариант — аллокатор ещё не успел отдать блок кому-то другому, всё «работает», и баг ждёт своего часа.
- второй `free(p)` — **double free**: освобождение уже освобождённого блока. Тоже UB, частая причина падений и уязвимостей: аллокатор ведёт свои служебные структуры прямо рядом с блоком, и повторное освобождение их портит.

Как чинить. Не обращайся к памяти после `free`, а указатель сразу обнуляй:

```c
free(p);
p = NULL;       /* теперь p честно «никуда»; free(NULL) безопасен */
```

Привычка `p = NULL` после `free` лечит оба бага разом: повторный `free(NULL)` по стандарту не делает ничего (C11, 7.22.3.3), а случайное `*p` по `NULL` — тоже UB, но на macOS и Linux оно по-

чти всегда сразу роняет программу, а не портит данные втихую. Вот что показал запуск кода выше без санитайзера: `printf` вывел мусор, каждый раз разный (например, `-1899806656`, но не 42!), а второй `free` уронил программу с кодом 134 — это 128 + 6, сигнал `SIGABRT`: аллокатор сам заметил двойное освобождение и прервал работу.

Оба бага мгновенно ловятся адресным санитайзером — он показывает и место освобождения, и место повторного обращения:

```
cc -std=c11 -g -fsanitize=address prog.c -o prog && ./prog
```

Первая же строка отчёта называет баг по имени:

```
ERROR: AddressSanitizer: heap-use-after-free on address ...
READ of size 4 at ... thread T0
```

Санитайзер останавливает программу на первом баге, поэтому до двойного `free` дело уже не доходит. Убери строку с `printf` — и он так же поймает `attempting double-free`.

Именно поэтому все программы этого приложения прогнаны под `-fsanitize=address,undefined` — и молчат.

Каверза 8. Возврат адреса локальной переменной

```c
#include <stdio.h>

int *broken(void)
{
    int x = 42;
    return &x;          /* БАГ: x умрёт по выходе из функции */
}

int main(void)
{
    int *p = broken();
    printf("%d\n", *p); /* читаем мёртвую память — UB */
    return 0;
}
```

Где баг. `x` — локальная переменная, она живёт на стеке только пока работает `broken`. После `return` её память освобождается — точнее, объявляется свободной: следующий же вызов любой функции положит туда свои данные. Адрес `&x` становится висячим. Обращение `*p` в `main` — неопределённое поведение.

Хорошая новость: этот баг компилятор ловит сам, если собирать с `-Wall`:

```
warning: address of stack memory associated with local variable 'x'
returned [-Wreturn-stack-address]
    6 |     return &x;
      |            ^
```

А вот чем оборачивается игнор предупреждения. Три запуска подряд одного и того же бинарника, собранного с `-O2`:

```
-1114688744
-1126284520
-1159945448
```

Не 42, не ноль и каждый раз разное: в этой ячейке стека лежат остатки служебных данных, а адреса от запуска к запуску сдвигает рандомизация адресного пространства. У тебя числа будут другими.

А теперь забавная деталь: соберём тот же файл с `-O0` (без оптимизации) — и три запуска честно печатают `42`, `42`, `42`. Значение просто не успели затереть. Это ровно та ловушка, о которой речь: UB иногда «работает», и именно поэтому на него нельзя опираться. Полагаться нельзя ни на 42, ни на мусор.

Адресный санитайзер по умолчанию этот баг не ловит. Его надо попросить переменной окружения:

```
cc -std=c11 -g -fsanitize=address prog.c -o prog
ASAN_OPTIONS=detect_stack_use_after_return=1 ./prog
```

Тогда он останавливает программу с отчётом `ERROR: AddressSanitizer: stack-use-after-return`.

Как чинить. Есть три законных способа:

1. вернуть **значение**, а не адрес (`int broken(void)` с `return x;`) — самый простой и обычно правильный;
2. выделить память динамически (`malloc`) и отдать ответственность за `free` вызывающему — как в каверзе 6;
3. принять место для результата от вызывающего параметром: `void fill(int *out) { *out = 42; }` — так устроены многие функции стандартной библиотеки: `scanf` пишет по переданным адресам, `strtol` через второй параметр сообщает, где кончилось число.

Возвращать адрес `static`-переменной технически можно — она живёт до конца программы, — но это общая память на все вызовы: два вызова подряд затрут результат друг друга, а в многопоточной программе будет гонка.

Ошибки, на которых сыплются в этих задачах

- **Забыть `head = reverse(head)`.** Функция вернула новую голову, а её никто не принял — список потерян.
- **Переставить проверки в `fast != NULL && fast->next != NULL`.** Порядок в && имеет значение: сначала `fast != NULL`, потом `fast->next != NULL`, и только потом прыжок `fast->next->next`.
- **`(lo + hi) / 2` в бинарном поиске.** Переполнение `int` на больших индексах — UB; на практике обычно получается отрицательный индекс.
- **`lo < hi` вместо `lo <= hi`.** Теряется последний элемент отрезка.
- **Битовые операции на знаковых типах.** Сдвиг влево отрицательного — UB; сдвиг вправо — implementation-defined. Бери `unsigned`.
- **Забыть, что массив в функции — это указатель.** `sizeof` соврёт, длину надо передавать отдельно.
- **Писать в строковый литерал.** UB. У нас на `-O0` программа упала, а на `-O2` молча проигнорировала запись; другой компилятор вправе сделать третье.
- **Потерять адрес из `malloc`.** Утечка, которую не видно по выводу.
- **Не обнулить указатель после `free`.** Use-after-free и double free ждут за углом.
- **Вернуть адрес локальной переменной.** Компилятор предупредит — прочитай предупреждение.

Тренировочные задачи

1. Допиши `my_strncpy(char *dst, const char *src, size_t n)` — копирует не больше `n` символов. Реализуй две версии: как в стандарте (может не дописать `'\0'`) и «безопасную» (всегда дописывает). Объясни, чем вторая лучше.
2. Напиши `struct Node *reverse_between(struct Node *head, int m, int n)` — разворот куска списка с позиции `m` по `n` (позиции считаем с 1). Проверь на краях: `m == 1`, `n == длина`, `m == n`.
3. Реализуй поиск **первого** элемента, который не меньше искомого, в отсортированном массиве с дубликатами (его называют lower bound). Проверь на массиве из одних одинаковых чисел.
4. Переделай `merge` из задачи 6 так, чтобы он сливал `a` и `b` в конец массива `a` (места хватает), идя с конца. Ни одного лишнего буфера.
5. Найди в массиве **два** числа, встречающихся нечётное число раз (остальные встречаются чётное число раз), за O(n) времени и O(1) памяти. Подсказка: `x & -x` даёт младший выставленный бит — считай его в `unsigned`.

6. Возьми программу из каверзы 8 и почини её тремя способами из разбора. Каждый вариант собери с `-Wall` и убедись, что предупреждение исчезло.

Ответы к тренировочным задачам

Сначала попробуй сам. Все решения ниже собраны с `-std=c11 -Wall -Wextra -O2` без предупреждений, прогнаны под `-fsanitize=address,undefined`, а те, где есть `malloc`, ещё и через `leaks` — чисто.

Задача 1.

```c
#include <stdio.h>
#include <string.h>

/* Как в стандарте: не больше n символов; если src длиннее,
   '\0' НЕ дописывается. Остаток до n заполняется нулями. */
char *my_strncpy(char *dst, const char *src, size_t n)
{
    size_t i = 0;
    for (; i < n && src[i] != '\0'; i++) {
        dst[i] = src[i];
    }
    for (; i < n; i++) {
        dst[i] = '\0';
    }
    return dst;
}

/* «Безопасная»: dst вмещает size байт, '\0' есть всегда. */
char *safe_strncpy(char *dst, const char *src, size_t size)
{
    if (size == 0) {
        return dst;
    }
    size_t i = 0;
    for (; i + 1 < size && src[i] != '\0'; i++) {
        dst[i] = src[i];
    }
    dst[i] = '\0';
```

```
    return dst;
}

int main(void)
{
    char a[4];
    my_strncpy(a, "Hello", sizeof a);
    printf("my_strncpy:   %.4s (a[3] = '%c')\n", a, a[3]);

    char b[4];
    safe_strncpy(b, "Hello", sizeof b);
    printf("safe_strncpy: %s (длина %zu)\n", b, strlen(b));
    return 0;
}
```

```
my_strncpy:   Hell (a[3] = 'l')
safe_strncpy: Hel (длина 3)
```

Стандартная версия честно скопировала 4 символа `Hell` и **не** поставила `'\0'`: в `a[3]` лежит `'l'`. Печатать такой массив через `%s` нельзя — `printf` побежит дальше за границу искать ноль. Поэтому здесь `%.4s`: точность `.4` велит напечатать не больше 4 символов. Безопасная версия оставила место под `'\0'` и отрезала строку до `Hel` — зато результат всегда корректная строка. Второй цикл в `my_strncpy` дописывает нули до `n`: так ведёт себя и стандартная `strncpy`, если источник короче.

Задача 2.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int value;
    struct Node *next;
};

/* Развернуть узлы с позиции m по n (нумерация с 1, 1 <= m <= n). */
struct Node *reverse_between(struct Node *head, int m, int n)
{
    struct Node dummy = {0, head};   /* фиктивный узел перед головой */
    struct Node *before = &dummy;
```

```
    for (int i = 1; i < m; i++) {
        before = before->next;       /* узел перед позицией m */
    }
    struct Node *first = before->next;   /* станет хвостом куска */
    struct Node *prev = NULL;
    struct Node *cur = first;
    for (int i = m; i <= n; i++) {       /* обычный разворот куска */
        struct Node *next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    before->next = prev;     /* начало куска — бывший его конец */
    first->next = cur;       /* хвост куска смотрит на остаток */
    return dummy.next;
}

struct Node *build(int len)
{
    struct Node *head = NULL;
    for (int i = len; i >= 1; i--) {
        struct Node *x = malloc(sizeof *x);
        if (x == NULL) {
            exit(1);
        }
        x->value = i;
        x->next = head;
        head = x;
    }
    return head;
}

void print_and_free(struct Node *head)
{
    while (head != NULL) {
        struct Node *next = head->next;
        printf("%d ", head->value);
        free(head);
```

```
        head = next;
    }
    printf("\n");
}

int main(void)
{
    print_and_free(reverse_between(build(5), 2, 4));
    print_and_free(reverse_between(build(5), 1, 5));
    print_and_free(reverse_between(build(5), 1, 2));
    print_and_free(reverse_between(build(5), 3, 3));
    return 0;
}
```

```
1 4 3 2 5
5 4 3 2 1
2 1 3 4 5
1 2 3 4 5
```

Фиктивный узел `dummy` стоит перед головой, и благодаря ему случай `m == 1` ничем не отличается от остальных: «узел перед куском» есть всегда. После разворота `prev` указывает на бывший конец куска (он стал началом), `first` — на бывшее начало (оно стало концом), а `cur` — на первый узел после куска. Два последних присваивания сшивают три части. Четыре строки вывода — это `m=2, n=4`, весь список, `m=1, n=2` и `m == n` (ничего не меняется).

Задача 3.

```
#include <stdio.h>

/* Первый индекс i, где a[i] >= target; n, если такого нет. */
int lower_bound(const int *a, int n, int target)
{
    int lo = 0;
    int hi = n;                 /* полуинтервал [lo, hi) */
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < target) {
            lo = mid + 1;       /* ответ правее mid */
        } else {
            hi = mid;           /* mid может быть ответом */
```

```
        }
    }
    return lo;
}

int main(void)
{
    int a[] = {1, 2, 2, 2, 5, 7};
    int same[] = {4, 4, 4, 4};
    printf("%d %d %d %d\n", lower_bound(a, 6, 2), lower_bound(a, 6, 3),
           lower_bound(a, 6, 0), lower_bound(a, 6, 9));
    printf("%d\n", lower_bound(same, 4, 4));
    return 0;
}
```

```
1 4 0 6
0
```

Здесь другая схема границ, чем в бинарном поиске из задачи 5: полуинтервал `[lo, hi)`, где `hi` не входит. Поэтому `hi = n`, условие `lo < hi` и сдвиг `hi = mid` без −1 — `mid` сам может оказаться ответом. Цикл кончается, когда `lo == hi`, и это единственный кандидат. В массиве `{1, 2, 2, 2, 5, 7}` первая двойка стоит на индексе 1; для 3 ответ 4 (первый элемент ≥ 3 — это 5); для 0 — индекс 0; для 9 — `n = 6`, «такого нет». На массиве из одних четвёрок ответ 0, а обычный бинарный поиск из задачи 5 вернул бы первый попавшийся индекс — 1.

Задача 4.

```
#include <stdio.h>

/* a вмещает na + nb элементов, заняты первые na. Сливаем с конца. */
void merge_into_a(int *a, int na, const int *b, int nb)
{
    int i = na - 1;          /* последний из a */
    int j = nb - 1;          /* последний из b */
    int k = na + nb - 1;     /* куда пишем */
    while (j >= 0) {         /* пока в b что-то осталось */
        if (i >= 0 && a[i] > b[j]) {
            a[k--] = a[i--];
        } else {
            a[k--] = b[j--];
```

```
        }
    }
}

int main(void)
{
    int a[11] = {1, 3, 5, 7, 9};
    int b[] = {2, 4, 6, 8, 10, 12};
    merge_into_a(a, 5, b, 6);
    for (int i = 0; i < 11; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    return 0;
}
```

```
1 2 3 4 5 6 7 8 9 10 12
```

Пишем в `a` с конца: позиция `k` всегда правее или равна `i`, поэтому мы не затираем ещё не прочитанные элементы `a`. Цикл идёт, пока не кончится `b`: если раньше кончится `a`, остаток `b` допишется той же веткой `else`, а если раньше кончится `b` — оставшиеся элементы `a` уже стоят на своих местах.

Задача 5.

```
#include <stdio.h>

/* Два числа, встречающихся нечётное число раз; остальные — чётное. */
void find_two_odd(const int *a, int n, int *x, int *y)
{
    unsigned all = 0;
    for (int i = 0; i < n; i++) {
        all ^= (unsigned)a[i];          /* all = x ^ y */
    }
    unsigned bit = all & -all;          /* младшая единица all */
    unsigned g1 = 0, g2 = 0;
    for (int i = 0; i < n; i++) {
        if ((unsigned)a[i] & bit) {
            g1 ^= (unsigned)a[i];
        } else {
```

```
            g2 ^= (unsigned)a[i];
        }
    }
    *x = (int)g1;
    *y = (int)g2;
}

int main(void)
{
    int a[] = {4, 7, 1, 4, 9, 1, 9, 9, 9, 3};
    int x, y;
    find_two_odd(a, 10, &x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```

```
7 3
```

В массиве `{4, 7, 1, 4, 9, 1, 9, 9, 9, 3}` пары — 4, 1 и 9 (девятка четыре раза), одиночки — 7 и 3. Первый проход даёт `all = 7 ^ 3`: `111 ^ 011 = 100`, то есть 4. Раз бит 4 выставлен, у 7 и 3 он разный. Второй проход делит числа на две группы по этому биту: 7 и 4 (бит есть), 3, 1 и 9 (бита нет). В каждой группе пары снова гасятся, и остаётся по одиночке. Всё считаем в `unsigned`: там `~all` определено всегда (это 2^{32} `- all` на этом маке), а в знаковом `int` при `INT_MIN` было бы переполнение. Проходов два, но время всё равно O(n), а памяти — три переменные.

Задача 6.

```
#include <stdio.h>
#include <stdlib.h>

int fixed_value(void)           /* способ 1: вернуть значение */
{
    int x = 42;
    return x;
}

int *fixed_heap(void)           /* способ 2: память из malloc */
{
    int *p = malloc(sizeof *p);
    if (p != NULL) {
```

```
        *p = 42;
    }
    return p;                   /* free — забота вызывающего */
}

void fixed_out(int *out)        /* способ 3: место даёт вызывающий */
{
    *out = 42;
}

int main(void)
{
    printf("%d\n", fixed_value());

    int *p = fixed_heap();
    if (p != NULL) {
        printf("%d\n", *p);
        free(p);
    }

    int v;
    fixed_out(&v);
    printf("%d\n", v);
    return 0;
}
```

```
42
42
42
```

Все три способа собираются без `-Wreturn-stack-address`. Во втором способе за `free` отвечает вызывающий — это надо оговаривать в комментарии к функции, как мы и сделали.

Мини-резюме

Что должно остаться в голове после этого приложения:

- **Строки** — это `char`-массивы с `'\0'`; любая своя строковая функция идёт по символам до нуля. Литерал `"abc"` менять нельзя, сравнивать символы надо как `unsigned char`.

- **Списки** — три указателя разворачивают, два указателя разной скорости находят середину, ловят цикл и находят его начало (Флойд). Следи за порядком присваиваний и не разыменовывай `NULL`.
- **Биты** — `n & (n-1)` гасит младшую единицу (popcount Кернигана, проверка степени двойки); XOR гасит пары (одиночка за O(1) памяти). Работай на `unsigned`.
- **Поиск и слияние** — бинарный поиск держи на включительных границах `[lo, hi]` с `lo + (hi - lo) / 2`; слияние — два индекса плюс хвосты.
- **Память** — на каждый `malloc` свой `free`; после `free` ставь `p = NULL`; не возвращай адрес локальной переменной; помни про decay массива в указатель и `const`-квалификаторы.

И главное, что проверяют на Middle: ты не зубришь алгоритмы, а понимаешь, **что происходит в памяти**. Поэтому на собеседовании проговаривай вслух, кто на что указывает и кто кого освобождает, — это ценится больше готового ответа.

Что дальше

Дальше — **Приложение В: Senior**. Там задачи злее: quicksort и merge sort с разбором худших случаев, обобщённые структуры через `void *` и указатели на функции, хеш-таблица с цепочками, кольцевой буфер, LRU-кэш, свои `atoi`/`itoa` с защитой от переполнения. А в теоретической части — strict aliasing, `volatile`, две роли `static`, выравнивание и padding в структурах, ловушки макросов, порядок вычисления аргументов и endianness. Если этот раздел дался без больших мучений — ты готов идти выше.

Приложение B. Задачи с собеседований: Senior

Это третий, самый трудный сборник задач с собеседований. Если в «Junior» проверяли, помнишь ли ты синтаксис, а в «Middle» — умеешь ли собрать алгоритм, то здесь смотрят глубже: понимаешь ли ты, **как Си устроен внутри**. Память, указатели на указатели, `void *`, выравнивание структур, неопределённое поведение (UB), тонкости препроцессора — то, на чём спотыкаются даже люди с опытом.

Задачи идут от простого к сложному и делятся на два блока:

- **на код** — написать рабочую программу: быстрые сортировки, обобщённые структуры через `void *`, хеш-таблица, кольцевой буфер, LRU-кэш и другие классические задачи;
- **каверзы и теория** — короткие вопросы, где надо не написать код, а объяснить, **почему так**: что выведет хитрый `sizeof`, чем опасен макрос `SQR(x)`, что такое strict aliasing.

Всё, что нужно, ты уже прошёл в главах 1–18: указатели и память (главы 3, 11), рекурсия (глава 9), сортировки и `qsort` (глава 8), структуры и связные списки (глава 11). Здесь мы доведём это до уровня, который ждут от senior-разработчика на Си.

Совет тот же, что и раньше: сначала реши задачу сам на бумаге и в редакторе, и только потом сверяйся с разбором. Все программы лежат в `code/21-*.c` и собираются без единого предупреждения:

```
cc -std=c11 -Wall -Wextra -O2 code/21-01-quicksort.c -o q && ./q
```

Номер задачи здесь совпадает с номером файла: задача 7 — это `code/21-07-hashtable.c`. Все программы, где есть `malloc`, прогнаны под `-fsanitize=address,undefined` и проверены на утечки — санитайзеры молчат. Каверзы во второй части в `code/` не вынесены: часть из них содержит намеренное UB, и такому не место в файлах книги.

Все выводы программ и все сообщения компилятора получены реальным запуском: Apple clang 21, x86-64, macOS. Где ответ зависит от платформы или от флагов сборки — я это отдельно оговариваю, потому что «одного правильного числа» там просто нет.

В конце приложения — разбор типовых ошибок и семь задач на самостоятельную отработку, а после них — ответы с проверенным кодом.

Часть 1. Задачи на код

Задача 1. Быстрая сортировка (quicksort)

Условие. Реализуй быструю сортировку массива `int` на месте (без дополнительного массива). Объясни выбор опорного элемента и сложность.

Идея. В главе 8 мы разобрали идею словами: выбираем **опорный** (pivot) элемент, переставляем массив так, чтобы слева оказались все меньшие опорного, а справа — большие. После такой перестановки опорный стоит на своём окончательном месте. Дальше рекурсивно сортируем левую и правую части.

Главная деталь — **разбиение** (partition). Возьмём схему Ломуто: опорным берём последний элемент `a[hi]`. Ведём границу `i` — индекс последнего «малого» элемента. Идём `j` от начала: как только встретили элемент меньше опорного, сдвигаем границу и меняем его туда. В конце ставим опорный сразу за границей.

Решение (`code/21-01-quicksort.c`):

```c
#include <stdio.h>

/* Обмен двух int через указатели (см. главу 3). */
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

/* Разбиение Ломуто: опорный — последний элемент a[hi].
   Расставляем меньшие опорного слева, большие — справа,
   и возвращаем итоговую позицию опорного. */
int partition(int a[], int lo, int hi)
{
    int pivot = a[hi];          /* опорный (pivot) */
    int i = lo - 1;             /* граница «меньших опорного» */

    for (int j = lo; j < hi; j++) {
```

```
        if (a[j] < pivot) {    /* нашли элемент меньше опорного */
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[hi]);   /* опорный встаёт на своё место */
    return i + 1;
}

/* Быстрая сортировка: разбили относительно опорного и
   рекурсивно сортируем левую и правую части. */
void quicksort(int a[], int lo, int hi)
{
    if (lo >= hi) {            /* 0 или 1 элемент — уже отсортировано */
        return;
    }
    int p = partition(a, lo, hi);
    quicksort(a, lo, p - 1);
    quicksort(a, p + 1, hi);
}
```

В `main` зовём `quicksort(a, 0, n - 1)` — границы **включительно**, поэтому `hi = n - 1`, а не `n`. Массив там `{5, 2, 9, 1, 5, 6, 3, 8, 7, 0}`, и запуск даёт:

```
0 1 2 3 5 5 6 7 8 9
```

Две пятёрки на месте — дубликаты сортировка переживает нормально, хотя порядок именно этих двух элементов между собой не гарантирован (quicksort нестабильна, об этом ниже).

Объяснение.

- `swap(int *a, int *b)` принимает **адреса** переменных и меняет их содержимое через разыменование `*a`, `*b` (см. главу 3). Передай мы числа по значению — обмен случился бы только внутри функции.
- `int i = lo - 1;` — граница начинается «до» отрезка: малых элементов пока нет. **Инвариант** — это утверждение, которое остаётся верным перед каждым витком цикла. Здесь он такой: всё в `a[lo..i]` меньше опорного, всё в `a[i+1..j-1]` — больше либо равно. На старте обе части пустые, и утверждение верно; каждый виток его сохраняет; значит, после цикла оно верно для всего отрезка `a[lo..hi-1]`.
- цикл `for (j = lo; j < hi; j++)` доходит только до `hi - 1`, сам опорный `a[hi]` не трогаем — его поставим в конце.

- `swap(&a[i + 1], &a[hi])` — после цикла `i+1` это первая «большая» ячейка; меняем её с опорным, и тот оказывается ровно между меньшими и большими. Возвращаем `i + 1` — его финальную позицию.

Сложность: лучший и средний случай. Одно разбиение отрезка из `m` элементов делает ровно `m – 1` сравнений: каждый элемент, кроме опорного, один раз сравнивается с опорным. Если опорный всякий раз делит отрезок пополам, получается дерево вызовов высотой около $\log_2 n$, и на каждом его уровне в сумме меньше `n` сравнений — итого O(n log n). Это **лучший** случай.

Средний случай — это не «опорный всегда попадает в середину», а среднее по всем возможным порядкам входа (или по случайному выбору опорного). Опорный может попасть куда угодно, но в среднем части выходят достаточно ровными: даже если бы каждое разбиение делило отрезок в пропорции 1 к 9, глубина была бы около логарифма `n` по основанию 10/9 — это примерно $6.6 \cdot \log_2 n$, то есть тоже O(log n). Точный подсчёт даёт в среднем примерно $2n \cdot \ln n \approx 1.39 \cdot n \cdot \log_2 n$ сравнений — это O(n log n), всего на 39% больше идеала.

Худший случай. Если опорный каждый раз оказывается минимумом или максимумом, одна часть получается пустой, а другая — из `m – 1` элементов. Тогда сравнений `(n–1) + (n–2) + … + 1 = n(n–1)/2`, это $O(n^2)$, а глубина рекурсии дорастает до `n – 1`. Наша версия (опорный — последний элемент) попадает в худший случай на трёх очень жизненных входах: уже отсортированном, отсортированном наоборот и **из одних одинаковых** чисел (условие `a[j] < pivot` для равных ложно, и все они уходят в одну часть).

Это не теория — я замерил на этом маке (`–O2`):

```
1 000 000 случайных чисел      0.07 с
  100 000 из одних семёрок     2.3 с
  100 000 по убыванию          3.1 с
  100 000 уже отсортированных  4.6 с
  200 000 уже отсортированных  падение, код выхода 139
```

Отсортированные 100 тысяч сортируются в 60 с лишним раз дольше, чем случайный миллион. А на 200 тысячах глубина рекурсии — 200 тысяч вызовов, каждый со своим кадром на стеке, и 8 МБ стека кончаются: `SIGSEGV` (139 = 128 + 11). Рекурсию и переполнение стека мы разбирали в главе 9.

Лечат это выбором опорного поумнее:

- **случайный опорный** — поменять `a[hi]` с ‘a[lo + rand() % (hi - lo
  - 1)]перед разбиением. Тогда никакой вход не бывает плохим «всегда»: $O(n^2)$ случается только при крайне невезучих случайных числах, а в среднем — O(n log n) на любом входе. Одна оговорка: стандарт обещаетRAND_MAXне меньше 32767, и там, где он

такой маленький (так в Windows), на отрезке длиннее 32768 выражение выберет не каждый индекс. На macOSRAND_MAX‘ = 2147483647.

- **медиана трёх** — берут медиану (среднее по величине) из `a[lo]`, `a[mid]`, `a[hi]`. Почти бесплатно и спасает на отсортированном и обратном входе, но не от всего: массив из одних одинаковых чисел с разбиением Ломуто всё равно даст $O(n^2)$, а для медианы трёх существуют специально подобранные «убийственные» входы.

Против одинаковых элементов помогает другое разбиение: схема Хоара или «трёхпутевое» разбиение (меньше / равно / больше опорного).

Память. Quicksort сортирует на месте, дополнительная память — только стек рекурсии: O(log n) в среднем и O(n) в худшем случае, что мы и увидели. Гарантировать O(log n) можно приёмом из тренировочной задачи 2: рекурсивно спускаться только в **меньшую** часть, а большую обрабатывать циклом.

Доп. вопросы.

- *Чем схема Хоара (два бегунка навстречу) лучше Ломуто?* Она делает в среднем примерно втрое меньше обменов, а массив из одинаковых чисел делит пополам, а не на «пусто и всё остальное».
- *Почему quicksort нестабильна?* **Устойчивая** (стабильная) сортировка сохраняет исходный порядок равных элементов. Quicksort переставляет элементы обменами через весь отрезок, и равные могут перепрыгнуть друг через друга. Самый маленький пример для нашей версии: две равные записи `1a, 1b`. Опорный — `1b`, `1a < 1b` ложно, и финальный `swap` ставит `1b` на место `1a`: получается `1b, 1a`.
- *Как сделать quicksort нерекурсивной?* Завести свой стек пар `(lo, hi)` — решение в ответе к тренировочной задаче 2.

Задача 2. Сортировка слиянием (merge sort)

Условие. Реализуй сортировку слиянием. В чём её плюс перед quicksort?

Идея. Делим массив пополам, рекурсивно сортируем каждую половину, потом **сливаем** две отсортированные половины в одну. Слияние — главный шаг: идём двумя бегунками по головам половин и каждый раз забираем меньший. В отличие от quicksort, merge sort даёт **гарантированные** O(n log n) даже в худшем случае и **стабильна**, но требует доп. памяти под временный буфер.

Решение (`code/21-02-mergesort.c`):

```
/* Слияние двух отсортированных половин a[lo..mid] и a[mid+1..hi]
   через временный буфер tmp. Берём меньший из голов и пишем по
   порядку — отсюда стабильность сортировки. */
```

```c
void merge(int a[], int tmp[], int lo, int mid, int hi)
{
    int i = lo;          /* бегунок по левой половине */
    int j = mid + 1;     /* бегунок по правой половине */
    int k = lo;          /* куда пишем в tmp */

    while (i <= mid && j <= hi) {
        if (a[i] <= a[j]) {     /* <= , а не <  — это и даёт стабильность */
            tmp[k++] = a[i++];
        } else {
            tmp[k++] = a[j++];
        }
    }
    while (i <= mid) {          /* хвост левой половины */
        tmp[k++] = a[i++];
    }
    while (j <= hi) {           /* хвост правой половины */
        tmp[k++] = a[j++];
    }
    for (int t = lo; t <= hi; t++) {   /* копируем обратно в a */
        a[t] = tmp[t];
    }
}

/* Сортировка слиянием: делим пополам, сортируем половины, сливаем. */
void merge_sort(int a[], int tmp[], int lo, int hi)
{
    if (lo >= hi) {
        return;
    }
    int mid = lo + (hi - lo) / 2;   /* без переполнения lo + hi */
    merge_sort(a, tmp, lo, mid);
    merge_sort(a, tmp, mid + 1, hi);
    merge(a, tmp, lo, mid, hi);
}
```

На том же массиве `{5, 2, 9, 1, 5, 6, 3, 8, 7, 0}` результат совпадает с quicksort — так и должно быть, отсортированный порядок один:

```
0 1 2 3 5 5 6 7 8 9
```

Объяснение.

- `tmp` — **один** временный буфер на весь алгоритм, выделяем его в `main` через `malloc((size_t)n * sizeof(int))` и передаём внутрь. Так не выделяем память на каждом слиянии.
- `int mid = lo + (hi - lo) / 2;` — почему не `(lo + hi) / 2`? Сумма `lo + hi` может **переполнить** `int` на огромных индексах. Запись `lo + (hi - lo)/2` даёт ту же середину, но без риска переполнения. Это классический приём (та же ловушка живёт и в бинарном поиске, см. главу 7 и Приложение Б, задача 5).
- `a[i] <= a[j]` — именно `<=`, а не `<`: при равенстве берём элемент из **левой** половины, то есть тот, что стоял в массиве раньше, и равные сохраняют исходный порядок. Это и есть **стабильность** (устойчивость, см. главу 8). Я проверил это отдельной программой: отсортировал 2000 случайных массивов из записей «ключ + исходный номер» с ключами от 0 до 9 — ни разу равные ключи не поменялись местами. С `<` вместо `<=` сортировка осталась бы правильной, но перестала бы быть стабильной.
- три `while` подряд: первый сливает, пока живы обе половины; два следующих дописывают остаток той половины, что ещё не кончилась.

Сложность и память. Глубина рекурсии — $\lceil \log_2 n \rceil$: каждый уровень делит отрезок пополам. На каждом уровне все слияния вместе трогают каждый элемент один раз, это O(n). Итого O(n log n) **всегда** — и в лучшем, и в худшем случае, потому что деление не зависит от данных. Сравнений не больше $n \cdot \lceil \log_2 n \rceil$. Память: буфер `tmp` на `n` элементов, то есть O(n), плюс O(log n) на стек рекурсии.

Quicksort или merge sort? Обычно говорят «quicksort быстрее на практике: сортирует на месте, лучше дружит с кэшем». Я проверил наши две версии на этом маке (`-O2`, случайные `int`):

```
              quicksort   merge sort
1 000 000       0.07 с      0.06 с
10 000 000      0.93 с      0.84 с
```

Наша простая merge sort даже чуть обогнала наш простой quicksort на Ломуто. Слова «quicksort быстрее» относятся к хорошо отлаженным реализациям (схема Хоара, вставки на коротких кусках и т. п.), а не к любому коду. Зато разница в худшем случае огромная: отсортированные 100 000 чисел quicksort разбирал 4.6 с, а merge sort — 0.003 с. Merge sort всегда O(n log n) и стабилен, но тратит O(n) дополнительной памяти. Для связных списков merge sort ещё удобнее: слияние не требует доступа по индексу, а буфер вообще не нужен — узлы просто перецепляются.

Доп. вопросы.

- *Сколько памяти под `tmp`?* O(n). *Можно ли слить две половины на месте за O(n)?* Алгоритмы такие есть, но они сложные и медленные на практике; в реальных библиотеках берут буфер.
- *Что такое «восходящий» (bottom-up) merge sort?* Без рекурсии: сначала сливаем соседние пары отрезков длины 1, потом длины 2, 4, 8 и так далее, пока отрезок не станет длиной во весь массив. Сложность та же, стек не нужен.

Задача 3. Обобщённый стек через `void *`

Условие. Напиши стек (или динамический массив), который умеет хранить элементы **любого** типа — `int`, `double`, структуру, — не переписывая код под каждый тип.

Идея. Шаблонов, как в C++, в Си нет. Универсальность строят на `void *` («указатель на что угодно») и на знании **размера элемента в байтах**. Стек хранит сырой байтовый буфер и `elem_size`. `push` **копирует** `elem_size` байт из переданного адреса в буфер, `pop` — обратно. Сам стек не знает, что за тип внутри: он просто двигает байты через `memcpy`.

Решение (`code/21-03-generic-stack.c`):

```c
typedef struct {
    char *data;          /* буфер из байтов */
    size_t elem_size;    /* размер одного элемента в байтах */
    size_t count;        /* сколько элементов лежит */
    size_t capacity;     /* сколько элементов влезает */
} Stack;

/* Готовим стек под элементы размера elem_size. 1 — успех, 0 — ошибка. */
int stack_init(Stack *s, size_t elem_size)
{
    if (elem_size == 0 || elem_size > SIZE_MAX / 4) {
        s->data = NULL;              /* такой размер элемента не годится */
        return 0;
    }
    s->elem_size = elem_size;
    s->count = 0;
    s->capacity = 4;
    s->data = malloc(s->capacity * elem_size);
    return s->data != NULL;
}

/* Кладём элемент: копируем elem_size байт по адресу elem на вершину. */
```

```c
int stack_push(Stack *s, const void *elem)
{
    if (s->count == s->capacity) {            /* места нет — растём */
        if (s->capacity > SIZE_MAX / 2 / s->elem_size) {
            return 0;          /* new_cap * elem_size не влезет в size_t */
        }
        size_t new_cap = s->capacity * 2;
        char *bigger = realloc(s->data, new_cap * s->elem_size);
        if (bigger == NULL) {
            return 0;
        }
        s->data = bigger;
        s->capacity = new_cap;
    }
    memcpy(s->data + s->count * s->elem_size, elem, s->elem_size);
    s->count++;
    return 1;
}

/* Снимаем верхний элемент в out (если out != NULL). 0 — стек пуст. */
int stack_pop(Stack *s, void *out)
{
    if (s->count == 0) {
        return 0;
    }
    s->count--;
    if (out != NULL) {
        memcpy(out, s->data + s->count * s->elem_size, s->elem_size);
    }
    return 1;
}
```

Использование одинаково для разных типов (в этом кратком наброске результаты `stack_init` и `stack_push` не проверяются; в файле они проверены):

```c
Stack si; stack_init(&si, sizeof(int));
int x = 10; stack_push(&si, &x);
```

```
Stack sd; stack_init(&sd, sizeof(double));
double d = 3.14; stack_push(&sd, &d);
```

В `main` файла в целочисленный стек кладут 10, 20, 30, 40, 50, а в вещественный — 3.14 и 2.71, потом обе стопки вычерпывают до дна. Запуск:

```
int:    50 40 30 20 10
double: 2.71 3.14
```

Порядок обратный порядку вставки — это и есть стек (LIFO, последним пришёл — первым ушёл). Обрати внимание: код самого стека между двумя блоками не изменился ни на строку, поменялся только `elem_size`.

Объяснение.

- буфер — `char *`, потому что `sizeof(char)` по определению равен 1; адресная арифметика `data + count * elem_size` ходит по буферу побайтно и попадает ровно в начало нужного элемента. По `void *` так шагать **нельзя**: стандарт разрешает `+` и `-` только указателям на типы с известным размером, а у `void` размера нет. GCC и clang разрешают это как своё расширение и по умолчанию молчат; с флагом `-pedantic` clang честно пишет `arithmetic on a pointer to void is a GNU extension`.
- `const void *elem` в `push` — «адрес элемента, тип неизвестен, менять не буду». Вызывающий передаёт `&x`. Внутри `memcpy` копирует `elem_size` байт — ровно один элемент.
- `pop` пишет результат по адресу `out` (если он не `NULL`) — тем же `memcpy`. Так мы «возвращаем» значение неизвестного типа: через выходной указатель, а не через `return`.
- `realloc` с удвоением вместимости — амортизированная вставка за O(1) (см. растущий массив в главе 11): отдельная вставка иногда стоит O(n) на копирование, но в сумме n вставок стоят O(n). Результат кладём во **временный** `bigger` и проверяем на `NULL`, иначе при неудаче потеряем старый блок.
- проверка `s->capacity > SIZE_MAX / 2 / s->elem_size` — защита от переполнения при умножении. `size_t` беззнаковый, и если `new_cap * elem_size` не влезет, произведение молча завернётся по модулю и станет маленьким: `realloc` выделит крошечный блок, а мы запишем за его границу. Делением такую проверку можно сделать, ни разу не переполнившись. `SIZE_MAX` — самое большое значение `size_t`, оно объявлено в `<stdint.h>`. По той же причине `stack_init` отказывается от нулевого и гигантского `elem_size`: на ноль делить нельзя.

Плата за обобщённость. Нет проверки типов: положил `int`, достал как `double` — компилятор смолчит, а ты получишь мусор. И `memcpy` с размером, известным только во время работы, может оказаться медленнее прямого присваивания. Зато один код работает со всем.

Доп. вопросы.

- *Почему буфер `char *`, а не `void *`?* Арифметика указателей, см. выше.
- *Как сделать обобщённую функцию печати?* Передать ещё указатель на функцию `void print(const void *)` — см. задачу 5 и тренировочную задачу 3.
- *Чем такой стек похож на `qsort`?* Тем же приёмом: `qsort` тоже получает `void *` на начало массива и размер элемента и двигает байты, не зная типа. Разница в том, что ей ещё нужен компаратор — указатель на функцию, которая знает тип и умеет сравнить два элемента.

Задача 4. Свои `atoi` и `itoa` с обработкой краёв

Условие. Напиши `my_atoi` (строка → `int`) и `my_itoa` (`int` → строка). Корректно обработай знак, пробелы, мусор и **переполнение**, включая `INT_MIN`.

Идея. В `atoi` копим число пошагово: `result = result*10 + цифра`. Опасность — переполнение `int`. Поэтому копим в `long long` (стандарт гарантирует ему не меньше 64 бит, а `int` на всех распространённых платформах 32-битный) и после каждой цифры проверяем выход за `INT_MAX`/`INT_MIN`. В `itoa` главная каверза — `INT_MIN`: его модуль `-INT_MIN` в `int` **не помещается** (диапазон несимметричен: от −2147483648 до 2147483647). Спасает работа в `long long`.

Почему именно `long long`, а не `long`? Стандарт обещает про `long` только «не меньше 32 бит». На 64-битных macOS и Linux он 8 байт, а на 64-битной Windows — 4, и там промежуточное `result * 10` переполнилось бы прямо внутри «защиты от переполнения». `long long` по стандарту не меньше 64 бит везде — вот его и берём.

Решение (`code/21-04-atoi-itoa.c`):

```c
#include <limits.h>

/* Строка -> int. Обрабатываем пробелы, знак и переполнение.
   Возвращает 1 при успехе (результат в *out), 0 при ошибке
   (нет цифр или число не влезает в int). */
int my_atoi(const char *s, int *out)
{
    while (*s == ' ' || *s == '\t') {   /* пропускаем ведущие пробелы */
        s++;
    }

    int sign = 1;
    if (*s == '+' || *s == '-') {
        if (*s == '-') {
            sign = -1;
```

```
        }
        s++;
    }

    if (*s < '0' || *s > '9') {          /* ни одной цифры — это не число */
        return 0;
    }

    long long result = 0;                /* копим в long long: он шире int */
    while (*s >= '0' && *s <= '9') {
        result = result * 10 + (*s - '0');
        if (sign == 1 && result > INT_MAX) {       /* вышли за +предел */
            return 0;
        }
        if (sign == -1 && -result < INT_MIN) {     /* вышли за -предел */
            return 0;
        }
        s++;
    }

    *out = (int)(sign * result);
    return 1;
}

/* int -> строка. buf должен быть не меньше 12 байт:
   знак + 10 цифр + завершающий '\0'. */
void my_itoa(int n, char *buf)
{
    char tmp[12];
    int i = 0;
    int neg = 0;

    /* Берём значение в long long: -INT_MIN в int не влезает. */
    long long value = n;
    if (value < 0) {
        neg = 1;
        value = -value;
    }
```

```
    do {                            /* цифры выходят задом наперёд */
        tmp[i++] = (char)('0' + value % 10);
        value /= 10;
    } while (value > 0);

    int j = 0;
    if (neg) {
        buf[j++] = '-';
    }
    while (i > 0) {                         /* разворачиваем в buf */
        buf[j++] = tmp[--i];
    }
    buf[j] = '\0';
}
```

Вывод программы целиком:

```
my_atoi("42") = 42
my_atoi("  -17") = -17
my_atoi("+5") = 5
my_atoi("2147483647") = 2147483647
my_atoi("2147483648") = ошибка
my_atoi("-2147483648") = -2147483648
my_atoi("abc") = ошибка
my_atoi("-2147483649") = ошибка
---
my_itoa(0) = "0"
my_itoa(42) = "42"
my_itoa(-17) = "-17"
my_itoa(2147483647) = "2147483647"
my_itoa(-2147483648) = "-2147483648"
```

Обрати внимание на две соседние строки: `2147483647` разбирается, а `2147483648` — уже нет. Это ровно граница `int`, и проверка на переполнение срабатывает на единицу за ней, а не «примерно там». Симметрично внизу: `-2147483648` разбирается (это минимальный `int`), `-2147483649` — ошибка.

Объяснение.

- `*s - '0'` — перевод символа-цифры в число. Цифры в таблице ASCII идут подряд: `'7' - '0' == 7` (см. главу 10).
- проверки переполнения **после каждой цифры**: для плюса `result` превысил `INT_MAX`, для минуса `-result` ушёл ниже `INT_MIN`. Важно, что проверка стоит внутри цикла, а не после него. Пока проверка не сработала, `result` не больше 2147483648 (это модуль `INT_MIN`), и следующая цифра даст не больше `2147483648 * 10 + 9 = 21474836489` — это легко влезает в `long long` (его потолок около $9.2 \cdot 10^{18}$). Значит, само промежуточное вычисление никогда не переполнится, сколько бы цифр ни было во входе: строку из 23 девяток функция честно отвергает.
- в `my_itoa` цикл `do { } while` обязателен: иначе для `n == 0` мы не выведем ни одной цифры. `do/while` гарантирует хотя бы один проход.
- `value = -value` для отрицательного — безопасно только потому, что `value` это `long long`. Сделай мы `int value = n; value = -value;` при `n == INT_MIN` получили бы **UB** (переполнение знакового): у −2147483648 нет положительной пары в `int`.
- размеры буферов: у 32-битного `int` не больше 10 цифр. В `tmp` лежат только цифры, так что 12 байт там с запасом. В `buf` идут знак, 10 цифр и `'\0'` — ровно 12 байт, поэтому `buf` должен быть не меньше
  12.

Что функция считает числом. Мы пропускаем только пробелы и табы (стандартная `atoi` пропускает всё, что `isspace` считает пробельным, включая `'\n'`). Разбор останавливается на первой не-цифре, и хвост молча игнорируется — как у `atoi`. Я проверил края: `""` и `"-"` — ошибка (нет цифр), `"12abc"` — 12, `"-0"` — 0, строка `"00000000000000000000042"` — 42 (ведущие нули не раздувают `result`), `"\n5"` — ошибка. Если хвост должен считаться ошибкой, проверь после цикла, что `*s == '\0'`.

Доп. вопросы.

- *Почему `abs(INT_MIN)` — это UB?* Результат, 2147483648, не помещается в `int`, а стандарт (C11, 7.22.6.1) прямо называет такой случай неопределённым поведением.
- *Как `my_itoa` обобщить на любую систему счисления?* Делить не на 10, а на `base`, а цифры брать из строки `"0123456789abcdef"` — решение в ответе к тренировочной задаче 4.
- *Что вернёт стандартный `atoi("99999999999")`?* Неизвестно: если значение не помещается в `int`, у `atoi` UB (C11, 7.22.1). Поэтому в реальном коде берут `strtol`: при выходе за пределы `long` она возвращает `LONG_MAX` или `LONG_MIN` и ставит `errno = ERANGE`. Но на macOS и Linux `long` 64-битный, и `"99999999999"` в него влезает без ошибки — проверку на границы `int` всё равно придётся сделать самому.

Задача 5. Таблица обработчиков на указателях на функции

Условие. Сделай диспетчер: по символу операции (+, −, ∗) вызывается нужная функция. Заодно покажи компаратор для `qsort`.

Идея. Функцию в Си можно передать как аргумент и хранить в переменной — через **указатель на функцию**. Связку «ключ → функция» складываем в массив структур (таблицу). Вместо длинного `switch` пробегаем таблицу и зовём найденный обработчик. Это тот же приём, что делает `qsort` универсальной: ей передают указатель на компаратор.

Решение (`code/21-05-funcptr.c`):

```c
#include <stdio.h>
#include <stdlib.h>

/* Три простые операции — все одной формы int(int,int). */
int op_add(int a, int b) { return a + b; }
int op_sub(int a, int b) { return a - b; }
int op_mul(int a, int b) { return a * b; }

/* Таблица обработчиков: символ операции -> указатель на функцию. */
typedef struct {
    char symbol;
    int (*fn)(int, int);   /* указатель на функцию (int, int) -> int */
} Handler;

/* Компаратор для qsort: сортировка по УБЫВАНИЮ, без вычитания
   (вычитание a-b может переполнить int, см. главу 8). */
int cmp_desc(const void *pa, const void *pb)
{
    int a = *(const int *)pa;
    int b = *(const int *)pb;
    return (a < b) - (a > b);   /* +1 если a<b (a идёт позже), -1 если a>b */
}

int main(void)
{
    Handler table[] = {
        {'+', op_add},
        {'-', op_sub},
        {'*', op_mul},
```

```
    };
    int n = (int)(sizeof table / sizeof table[0]);

    int x = 6, y = 4;
    char ops[] = {'+', '-', '*'};
    for (int k = 0; k < 3; k++) {
        char op = ops[k];
        for (int i = 0; i < n; i++) {
            if (table[i].symbol == op) {           /* нашли обработчик */
                int r = table[i].fn(x, y);         /* вызов через указатель */
                printf("%d %c %d = %d\n", x, op, y, r);
                break;
            }
        }
    }

    printf("---\n");

    int a[] = {3, 1, 4, 1, 5, 9, 2, 6};
    int m = (int)(sizeof a / sizeof a[0]);
    qsort(a, (size_t)m, sizeof a[0], cmp_desc);
    for (int i = 0; i < m; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    return 0;
}
```

Запуск:

```
6 + 4 = 10
6 - 4 = 2
6 * 4 = 24
---
9 6 5 4 3 2 1 1
```

Сверху — три вызова через таблицу, снизу — тот же массив, отсортированный по убыванию силами `qsort` с нашим компаратором.

Объяснение.

- `int (*fn)(int, int);` — самая пугающая запись в Си. Читай изнутри: `fn` — это указатель (`*fn`), на функцию, принимающую `(int, int)` и возвращающую `int`. **Скобки вокруг `*fn` обязательны**: без них `int *fn(int, int)` означало бы «функция, возвращающая `int *`» — совсем другое.
- имя функции `op_add` в выражении само превращается в указатель на неё (так записано в стандарте, C11 6.3.2.1), поэтому `{'+', op_add}` записывает указатель. Амперсанд не нужен (хотя `&op_add` тоже допустимо и означает то же).
- `sizeof table / sizeof table[0]` — размер всей таблицы в байтах, делённый на размер одной записи, то есть число записей (здесь 3).
- `table[i].fn(x, y)` — вызов через указатель. Можно писать и `(*table[i].fn)(x, y)` — это эквивалентно.
- `cmp_desc` возвращает `(a < b) - (a > b)`. Сравнение в Си даёт 1 или 0, поэтому при `a = 3, b = 9` выйдет `1 - 0 = 1`, при `a = 9, b = 3` — `0 - 1 = -1`, при равных — `0 - 0 = 0`. `qsort` ставит `a` раньше `b`, когда компаратор вернул отрицательное число, — значит, большее идёт раньше, это и есть убывание. И никакого опасного `a - b`, которое может переполнить `int` на больших числах (см. ловушку в главе 8).

Где это применяют по-настоящему. Таблицы команд в интерпретаторах, обработчики событий, колбэки (callbacks), state-машины, плагины. Указатель на функцию — основа любого «подключаемого» поведения в Си.

Доп. вопросы.

- *Что такое `typedef int (*BinOp)(int, int);`?* Короткое имя для типа «указатель на функцию `(int, int) -> int`». После него поле структуры пишется просто `BinOp fn;`, а параметр — `BinOp op`.
- *Чем `qsort_r` удобнее обычной `qsort`?* Она передаёт в компаратор ещё один указатель — «контекст», например направление сортировки, — без глобальных переменных. Но это **не** стандартный Си: `qsort_r` есть в BSD/macOS и в glibc, причём с разным порядком аргументов, а в стандарте C11 её нет (там есть необязательная `qsort_s` из приложения K, которую macOS и glibc не поддерживают).
- *Можно ли сделать массив указателей на функции без структуры?* Да: `int (*ops[])(int, int) = {op_add, op_sub};`, а вызов — `ops[0](6, 4)`.

Задача 6. Кольцевой буфер (ring buffer)

Условие. Реализуй кольцевой буфер фиксированного размера: запись в хвост, чтение из головы, переполнение и опустошение — корректно.

Идея. Это очередь (FIFO) на кольцевом буфере — ты уже встречал её в главе 11, здесь она оформлена аккуратной структурой. Храним массив, индекс головы `head` и количество `count`.

Хвост вычисляем: `(head + count) % CAP`. Остаток от деления `%` заворачивает индекс с конца массива в начало — массив словно свёрнут в кольцо. Пуст при `count == 0`, полон при `count == CAP`.

Решение (`code/21-06-ringbuf.c`):

```c
#define CAP 4   /* размер кольца */

/* Кольцевой буфер на массиве фиксированного размера.
   Храним голову и количество; хвост вычисляем. */
typedef struct {
    int data[CAP];
    int head;     /* индекс первого элемента */
    int count;    /* сколько элементов лежит */
} Ring;

/* Кладём в хвост. 0 — буфер полон. */
int ring_push(Ring *r, int value)
{
    if (r->count == CAP) {
        return 0;
    }
    int tail = (r->head + r->count) % CAP;   /* первое свободное место */
    r->data[tail] = value;
    r->count++;
    return 1;
}

/* Забираем из головы в *out. 0 — буфер пуст. */
int ring_pop(Ring *r, int *out)
{
    if (r->count == 0) {
        return 0;
    }
    *out = r->data[r->head];
    r->head = (r->head + 1) % CAP;           /* голова заворачивается */
    r->count--;
    return 1;
}
```

Объяснение.

- почему хранить `count`, а не индекс `tail`? Если держать `head` и `tail`, возникает путаница: при `head == tail` буфер пуст **или** полон — состояния неразличимы. С `count` всё однозначно.
- `(head + count) % CAP` — позиция за последним элементом, она же первое свободное место. Когда сумма дорастает до `CAP`, `%` сбрасывает её в 0.
- `head = (head + 1) % CAP` — после чтения голова шагает вперёд и тоже заворачивается.
- никаких `malloc` — буфер на массиве фиксированного размера внутри самой структуры, освобождать нечего.
- `%` здесь безопасен: `head` и `count` никогда не бывают отрицательными, а для неотрицательных чисел остаток `%` в Си совпадает с математическим (для отрицательных — нет, см. задачу 10).

Вывод целиком:

```
push 1: ok
push 2: ok
push 3: ok
push 4: ok
push 5: полон
push 6: полон
pop 1
pop 2
pop 3
pop 4
pop 7
pop 8
```

Читается он так. Первые четыре `push` заполнили кольцо (`CAP` равен 4), пятый и шестой уже не влезли. Дальше два `pop` забрали `1` и `2` — и в освободившиеся ячейки легли `7` и `8`, завернувшись в начало массива. Финальный `while` вычерпывает остаток: сначала «старые» `3` и `4`, и только потом `7` и `8`. Порядок FIFO соблюдён, хотя физически в массиве `7` и `8` лежат раньше `3` и `4`.

Зачем нужно. Кольцевые буферы — это потоки аудио/видео, буфер клавиатуры, логи фиксированного размера, обмен данными между потоком- производителем и потоком-потребителем. Размер заранее известен, память не скачет.

Доп. вопросы.

- *Чем плох вариант с двумя индексами без `count`?* Неразличимы «пусто» и «полно»; лечат «потерей одной ячейки» (буфер считается полным, когда `(tail + 1) % CAP == head`, как в главе 11) или отдельным флагом.
- *Почему `CAP` — степень двойки иногда быстрее?* Для неотрицательного `x` остаток `x % 4` равен `x & 3`: это младшие два бита. Побитовое И дешевле деления. С константой `CAP` компилятор делает такую замену сам, особенно если индексы беззнаковые.
- *Как сделать буфер растущим?* Решение — в ответе к тренировочной задаче 6.

Задача 7. Хеш-таблица «строка → счётчик» с цепочками

Условие. Посчитай, сколько раз встретилось каждое слово. Реализуй хеш-таблицу с разрешением коллизий **цепочками**.

Идея. Хеш-таблица — массив «корзин». Функция-хеш превращает строку в число, остаток от деления на число корзин даёт индекс. Разные строки могут попасть в одну корзину — это **коллизия**. Решаем цепочками: каждая корзина это связный список записей, в ней лежащих. Поиск: вычислили корзину, прошли её цепочку, сравнили ключи `strcmp`.

Решение (`code/21-07-hashtable.c`):

```c
#define NBUCKETS 16   /* число корзин (степень двойки — удобно) */

/* Запись в цепочке: ключ-строка, счётчик и указатель на следующую
   запись в той же корзине (разрешение коллизий цепочками). */
typedef struct Entry {
    char *key;
    int count;
    struct Entry *next;
} Entry;

typedef struct {
    Entry *buckets[NBUCKETS];
} HashTable;

/* Хеш строки djb2 — простой и хороший для коротких ключей. */
unsigned long hash_str(const char *s)
{
    unsigned long h = 5381;
    int c;
    while ((c = (unsigned char)*s++) != 0) {
        h = h * 33u + (unsigned long)c;
```

```
    }
    return h;
}

void table_init(HashTable *t)
{
    for (int i = 0; i < NBUCKETS; i++) {
        t->buckets[i] = NULL;
    }
}

/* Увеличить счётчик слова на 1 (или добавить новое). 0 — нет памяти. */
int table_inc(HashTable *t, const char *key)
{
    size_t idx = hash_str(key) % NBUCKETS;

    /* проходим цепочку этой корзины — нет ли уже такого ключа */
    for (Entry *e = t->buckets[idx]; e != NULL; e = e->next) {
        if (strcmp(e->key, key) == 0) {
            e->count++;
            return 1;
        }
    }

    /* нового ключа нет — создаём запись и вешаем в начало цепочки */
    Entry *e = malloc(sizeof(Entry));
    if (e == NULL) {
        return 0;
    }
    e->key = malloc(strlen(key) + 1);   /* +1 под '\0' */
    if (e->key == NULL) {
        free(e);
        return 0;
    }
    strcpy(e->key, key);
    e->count = 1;
    e->next = t->buckets[idx];
    t->buckets[idx] = e;
```

```
    return 1;
}
```

В `main` мы прогоняем список слов `{"cat", "dog", "cat", "bird", "dog", "cat"}` и потом спрашиваем счётчики, в том числе у слова, которого в таблице нет:

```
cat  = 3
dog  = 2
bird = 1
fish = 0
```

Ноль у `fish` — это не «встретилось ноль раз», а «не нашли». Такой интерфейс удобен для счётчиков, но для таблицы «ключ → значение» он бы не годился: там надо отличать «нет ключа» от «значение равно нулю», и функция должна возвращать признак успеха отдельно (как `lru_get` в задаче 9).

Объяснение.

- `hash_str` (алгоритм djb2 Дэниела Бернстайна): начинаем с 5381, на каждый символ `h = h*33 + c`. Множитель 33 и старт 5381 подобраны опытным путём — на обычных словах функция неплохо разбрасывает ключи по корзинам. Тип `unsigned long`, чтобы переполнение было определённым: для беззнаковых это не UB, а честный остаток по модулю 2^{64} (у `unsigned long` на этом маке 64 бита; см. каверзу 1 про UB).
- `(unsigned char)*s` — важная деталь: `char` может быть знаковым (на этом маке так и есть), и байты с кодом больше 127 — например, буквы кириллицы в UTF-8 — превратились бы в отрицательные числа. Хеш всё равно считался бы, но **другой**, чем на платформе с беззнаковым `char`: я проверил на букве «ж» — 5866955 с приведением и 5858251 без него. Одна и та же строка на разных машинах попадала бы в разные корзины. Приведение к `unsigned char` делает хеш одинаковым везде.
- коллизии: при `strcmp` мы **сравниваем сами ключи**, а не только хеши. Два разных слова могут попасть в одну корзину, и `strcmp` их различит. Это не выдумка: в нашем примере `dog` (хеш 193489663) и `fish` (хеш 6385224719) оба дают остаток 15 от деления на 16. Поиск `fish` приходит в корзину 15, находит там `dog`, `strcmp` говорит «не то» — и функция честно возвращает 0.
- **владение памятью**: ключ копируем своим `malloc(strlen(key)+1)` + `strcpy`. Нельзя просто сохранить переданный указатель — строка вызывающего может измениться или исчезнуть. На каждый `malloc` есть `free` в `table_free` (сначала `free(e->key)`, потом `free(e)`).
- вставка в **начало** цепочки — O(1), порядок внутри корзины неважен.

Про сложность. Главное число хеш-таблицы — **коэффициент загрузки** (load factor) $\alpha = n / m$, где n — сколько записей лежит, m — сколько корзин. Это средняя длина цепочки. У нас после шести слов записей три (`cat`, `dog`, `bird`), корзин 16: $\alpha = 3/16 \approx 0.19$.

- **В среднем** (если хеш разбрасывает ключи равномерно) поиск проходит цепочку средней длины α, то есть стоит $O(1 + \alpha)$. Пока α ограничено константой — скажем, не больше 1, — это $O(1)$.
- **В худшем случае** все ключи попадают в одну корзину, цепочка длиной n, и поиск — $O(n)$. От этого не спасает никакое число корзин: это вопрос хеш-функции и входных данных (злоумышленник может нарочно подобрать ключи с одинаковым хешем).
- **Наша таблица** держит 16 корзин навсегда. Значит, при n словах $\alpha = n/16$ растёт вместе с n, и даже средний поиск — $O(n/16)$, то есть $O(n)$. Для счётчика в несколько сотен слов это не страшно, но «O(1)» у этого кода честно не выполняется.

Лечат **рехешированием**: когда α превысит порог (обычно от 0.75 до 1–2), число корзин удваивают и перекладывают все записи. Одно рехеширование стоит $O(n)$, но случается всё реже, и в сумме вставка выходит **амортизированно** $O(1)$ — тот же расчёт, что у растущего массива в главе 11. Не путай: «в среднем» — про случайность входа, «амортизированно» — про сумму по длинной серии операций.

Доп. вопросы.

- *Чем цепочки отличаются от открытой адресации?* При открытой адресации записи лежат прямо в массиве, а при коллизии ищут соседнюю свободную ячейку (линейное пробирование — следующую, квадратичное — через 1, 4, 9… шагов). Плюсы: нет `malloc` на каждую запись, данные лежат подряд и хорошо дружат с кэшем. Минусы: α должен оставаться меньше 1, производительность резко падает при заполнении, а удаление требует особых «надгробий» на месте удалённых записей.
- *Когда делать рехеш?* Когда α перешёл выбранный порог — см. выше; реализация — в ответе к тренировочной задаче 5.
- *Почему число корзин берут простым или степенью двойки?* Остаток от деления на простое число зависит от всех битов хеша, и это сглаживает слабую хеш-функцию. Степень двойки позволяет заменить `%` на быстрое `&`, но тогда в индекс идут только младшие биты хеша, и хеш должен хорошо перемешивать именно их.

Задача 8. Точка пересечения двух односвязных списков

Условие. Даны два односвязных списка, которые в какой-то момент сливаются в общий хвост. Найди **узел слияния** (по адресу, не по значению). Если не пересекаются — `NULL`.

Идея. Наивно — для каждого узла A проверять весь B: $O(n \cdot m)$. Умнее: если списки сливаются, у них **общий хвост одинаковой длины**. Значит, разница только в длине «головных» частей. По-

считаем длины, проматаем длинный список вперёд на разницу — и теперь оба узла на равном расстоянии до конца. Шагаем параллельно: первый узел, где **адреса совпадут**, и есть точка пересечения. Это O(n + m) времени и O(1) памяти.

Решение (`code/21-08-list-intersection.c`):

```c
typedef struct Node {
    int value;
    struct Node *next;
} Node;

int length(Node *head)
{
    int len = 0;
    for (Node *p = head; p != NULL; p = p->next) {
        len++;
    }
    return len;
}

/* Узел пересечения двух списков или NULL.
   Идея: выравниваем старты. Длинный список проматываем вперёд на
   разницу длин — теперь до конца обоим идти поровну. Дальше шагаем
   параллельно: где адреса совпадут, там и общий узел. */
Node *intersection(Node *a, Node *b)
{
    int la = length(a);
    int lb = length(b);

    while (la > lb) { a = a->next; la--; }
    while (lb > la) { b = b->next; lb--; }

    while (a != b) {        /* сравниваем АДРЕСА узлов, не значения */
        a = a->next;
        b = b->next;
    }
    return a;               /* общий узел или NULL */
}
```

В `main` мы строим общий хвост `7 -> 8 -> 9 -> NULL`, вешаем на него список A (`1 -> 2 -> 3 ->` `хвост`) и список B (`4 -> 5 -> хвост`), а потом ищем точку слияния:

```
Пересечение в узле со значением 7
```

Длины 6 и 5, разница 1: указатель по списку A делает один холостой шаг, и дальше оба бегут синхронно — совпадение случается ровно на первом узле общего хвоста.

Объяснение.

- ключевая мысль — сравнивать `a != b`, то есть **указатели**, а не `a->value != b->value`. Пересечение списков означает, что с какого-то узла они физически идут по **одной и той же** памяти. Значения могут совпадать и случайно, а адрес узла уникален.
- после выравнивания обоим до конца идти поровну. Если есть общий хвост, бегунки войдут в него синхронно и встретятся на узле слияния. Если пересечения нет, оба одновременно дойдут до `NULL` — и `a == b == NULL`, цикл закончится, вернём `NULL`. Один и тот же код покрывает оба случая.
- освобождение тут с подвохом: общий хвост нельзя `free` дважды. Сначала чистим уникальные части A и B **до** общего узла `tail`, потом — сам хвост один раз. Двойной `free` — это UB (см. блок ниже).

Доп. вопросы.

- *А если в списках может быть цикл?* Тогда `length` зациклится. Сначала проверь оба списка алгоритмом Флойда — он и начало цикла находит, всё это разобрано в Приложении Б, задача 3.
- *Как решить без подсчёта длин?* Два указателя: дойдя до конца своего списка, каждый перепрыгивает на начало чужого. Оба пройдут одинаковый путь `la + lb` и встретятся на пересечении (или вместе придут в `NULL`).
- *Почему сравнение по значению дало бы неверный ответ?* Представь A = `1 -> 7 -> 8 -> 9` и B = `5 -> 7 -> 8 -> 9`, где у каждого списка свои узлы: значения совпадут на 7, хотя списки вообще не пересекаются.

Задача 9. Набросок LRU-кэша

Условие. Реализуй кэш на N записей с вытеснением **давно не использованных** (LRU — Least Recently Used). `get` и `put` — в среднем за O(1).

Идея. Нужны две вещи сразу: быстрый **поиск по ключу** и быстрое определение, **кто давно не использовался**. Поиск даёт хеш-таблица. Порядок использования — **двусвязный список**: голова это самый свежий, хвост — самый старый кандидат на вылет. Каждый узел живёт одновременно в обеих структурах. `get`/`put` находят узел через хеш и переставляют его в голову списка. При переполнении удаляем хвост — и из списка, и из хеша.

Ключевой код (`code/21-09-lru.c`; функции `lru_init`, `list_unlink`, `list_push_front`, `hash_find` и `lru_free` — в файле, они короткие и прямолинейные):

```
typedef struct Node {
    int key;
    int value;
    struct Node *prev;    /* соседи в списке свежести */
    struct Node *next;
    struct Node *hnext;   /* следующий в хеш-корзине */
} Node;

typedef struct {
    int capacity;         /* сколько записей помещается */
    int size;             /* сколько записей сейчас */
    Node *head;           /* самый свежий */
    Node *tail;           /* самый старый */
    Node **buckets;       /* массив корзин: их столько же, сколько мест */
    unsigned nbuckets;
} LRU;

/* убрать узел из его хеш-цепочки */
static void hash_remove(LRU *c, Node *n)
{
    Node **pp = &c->buckets[bucket_of(c, n->key)];
    while (*pp != n) {
        pp = &(*pp)->hnext;
    }
    *pp = n->hnext;
}

/* get: 1 и значение в *out, если ключ есть. Обращение освежает запись. */
int lru_get(LRU *c, int key, int *out)
{
    Node *n = hash_find(c, key);
    if (n == NULL) {
        return 0;
    }
    list_unlink(c, n);
    list_push_front(c, n);   /* подняли наверх как самый свежий */
```

```
    *out = n->value;
    return 1;
}

/* put: 1 — записано, 0 — не хватило памяти (кэш тогда не меняется). */
int lru_put(LRU *c, int key, int value)
{
    Node *n = hash_find(c, key);
    if (n != NULL) {                /* ключ уже есть — обновим и освежим */
        n->value = value;
        list_unlink(c, n);
        list_push_front(c, n);
        return 1;
    }

    n = malloc(sizeof(Node));       /* сначала память, потом вытеснение */
    if (n == NULL) {
        return 0;
    }

    if (c->size == c->capacity) {   /* кэш полон — вытесняем хвост */
        Node *old = c->tail;
        list_unlink(c, old);
        hash_remove(c, old);
        free(old);
        c->size--;
    }

    n->key = key;
    n->value = value;
    unsigned b = bucket_of(c, key);
    n->hnext = c->buckets[b];       /* в начало хеш-цепочки */
    c->buckets[b] = n;
    list_push_front(c, n);          /* и в голову списка свежести */
    c->size++;
    return 1;
}
```

Объяснение.

- узел в **двух** структурах: `prev/next` — порядок свежести, `hnext` — цепочка хеша. Двусвязный список позволяет вынуть узел из середины за O(1): соседи известны сразу (у односвязного пришлось бы искать предыдущий проходом по списку).
- `list_unlink` + `list_push_front` — «освежить»: при каждом обращении узел переезжает в голову. Значит, у хвоста всегда самый давний доступ.
- в `hash_remove` интересна строка `Node **pp = &c->buckets[...]` — **указатель на указатель**. Он позволяет удалить узел из односвязной хеш-цепочки без отдельной проверки «это голова или середина»: `pp` всегда указывает на то поле (`hnext` предыдущего узла или сам `buckets[b]`), которое надо перенаправить. Классический senior-приём.
- `lru_put` сначала выделяет память под новый узел и только потом вытесняет старый. Если `malloc` не удался, кэш остаётся нетронутым, а функция возвращает 0. В обратном порядке мы бы выкинули запись и ничего не положили взамен.
- `lru_init` отказывается от вместимости меньше 1: при `capacity == 0` условие «кэш полон» было бы верно сразу, а хвоста ещё нет, и `list_unlink(c, NULL)` разыменовал бы `NULL`.

Правда ли тут O(1)? Честно разберём по частям. Операции со списком (`list_unlink`, `list_push_front`) — O(1) всегда: несколько присваиваний. Поиск `hash_find` и удаление `hash_remove` проходят одну хеш-цепочку, и их цена — длина цепочки. Корзин мы заводим **столько же, сколько мест в кэше**, поэтому записей никогда не больше, чем корзин: коэффициент загрузки $\alpha \le 1$ (см. задачу 7). Значит, **в среднем** цепочка короткая, и `get`/`put` — O(1) в среднем. В **худшем** случае (все ключи дают один и тот же остаток `key % nbuckets`, например кратны вместимости) цепочка одна на всех, и операции стоят O(N), где N — вместимость кэша.

Частая ошибка на собесе — завести фиксированное число корзин, скажем 16, как в задаче 7. Тогда на кэше в миллион записей средняя цепочка была бы длиной 1 000 000 / 16 = 62 500, и обещанное O(1) превратилось бы в O(N). Поэтому корзины выделяются в `lru_init` под размер кэша.

Я прогнал кэш против простой «эталонной» реализации (массив и перебор): вместимости от 1 до 9, по 200 случайных серий из 200 операций `get`/`put` с ключами от −11 до 11, всего 360 тысяч операций под `-fsanitize=address,undefined`. Результаты совпали везде, санитайзеры молчали.

Вывод программы подтверждает логику: после `get 1` ключ 1 стал свежим, и `put 3` вытеснил именно 2 (его давно не трогали), а 1 уцелел:

```
get 1 = 100 (теперь 1 свежий)
get 2 = промах (вытеснен)
get 1 = 100
get 3 = 300
```

Доп. вопросы.

- *Почему именно **дву**связный список, а не односвязный?* Удаление узла из середины за O(1), см. выше.
- *Чем LRU отличается от LFU и FIFO?* LRU выкидывает то, к чему дольше всего не обращались; LFU (Least Frequently Used) — то, к чему обращались реже всего; FIFO — то, что раньше всех положили, даже если им пользуются постоянно.
- *Как сделать кэш потокобезопасным?* Самое простое — один мьютекс вокруг каждой операции. Заметь: даже `get` меняет список, поэтому «читателей» тут тоже надо запирать.

Задача 10. Разворот числа с защитой от переполнения

Условие. Развернуть цифры знакового `int` (`123 → 321`, `-456 → -654`). Если перевёрнутое число не помещается в `int` — сообщить об ошибке, а не выдать мусор.

Идея. Снимаем последнюю цифру `n % 10`, отбрасываем её `n /= 10`, дописываем в результат `result = result*10 + цифра`. Ловушка та же, что в `atoi`: `result*10` может переполнить `int`. В `atoi` мы спаслись более широким `long long`, но на собеседовании эту задачу часто дают с условием «более широких типов нет, работай в `int`». Поэтому проверяем **до** умножения: не выйдет ли `result*10 + цифра` за границу.

Решение (`code/21-10-reverse-int.c`):

```c
#include <limits.h>

int reverse_int(int n, int *out)
{
    int result = 0;
    while (n != 0) {
        int digit = n % 10;   /* знак сохраняется: -123 % 10 == -3 (C99) */
        n /= 10;

        /* Проверяем ДО умножения: влезет ли result*10 + digit.
           Для положительной части сравниваем с INT_MAX, для
           отрицательной — с INT_MIN. */
        if (result > INT_MAX / 10 ||
            (result == INT_MAX / 10 && digit > INT_MAX % 10)) {
            return 0;
        }
        if (result < INT_MIN / 10 ||
            (result == INT_MIN / 10 && digit < INT_MIN % 10)) {
```

```
            return 0;
        }
        result = result * 10 + digit;
    }
    *out = result;
    return 1;
}
```

Запуск на наборе из семи чисел:

```
reverse(123) = 321
reverse(-456) = -654
reverse(120) = 21
reverse(0) = 0
reverse(1534236469) = переполнение
reverse(1000000003) = переполнение
reverse(-2147483648) = переполнение
```

Три случая, ради которых задачу и дают. `120` разворачивается в `21`, а не в `021` — ведущий ноль просто исчезает, потому что мы работаем с числом, а не со строкой. `1000000003` перевернулось бы в `3000000001` — это больше `INT_MAX`. А `INT_MIN` не разворачивается: `-8463847412` не влезает в 32-битный `int`.

Объяснение.

- работаем **без** перевода в `unsigned` или `long` — прямо в `int`. Поэтому `result` хранит уже со знаком: для отрицательного `n` и `digit`, и `result` отрицательны. Так работает деление в Си начиная с C99: частное округляется **к нулю** (`-123 / 10 == -12`), а остаток подбирается так, чтобы выполнялось `a == (a / b) * b + a % b`, — то есть `-123 % 10 == -3` и знак остатка совпадает со знаком делимого. Это **не** математический остаток: в математике остаток от деления −123 на 10 равен 7 (−123 = −13·10 + 7). Нам как раз удобен сишный вариант: цифры отрицательного числа выходят отрицательными, и `result` растёт вниз, к `INT_MIN`, без единого взятия модуля.
- проверка «влезет ли `result*10 + digit`» сделана **арифметикой без переполнения**. Подставим числа: `INT_MAX / 10` = 214748364, `INT_MAX % 10` = 7. Если `result` больше 214748364, то `result*10` не меньше 2147483650 — это больше `INT_MAX`, мимо. Если `result` ровно 214748364, влезет только цифра не больше 7 (2147483647 — это `INT_MAX`). Внизу симметрично: `INT_MIN / 10` = −214748364, `INT_MIN % 10` = −8. Так мы ни разу не выходим за границы `int` в самих вычислениях.
- `1534236469` развернётся в `9646324351` — это больше `INT_MAX` (2147483647), поэтому функция честно вернёт «переполнение», а не мусор.

Доп. вопросы.

- *Почему опасно «развернуть, потом проверить»?* Переполнение знакового — это UB ещё **до** проверки, и компилятор вправе выкинуть саму проверку (пример — в каверзе 1).
- *Как развернуть строку на месте?* Два указателя навстречу, см. главу 10.
- *Чем отличается `%` для отрицательных в C89 и C99?* В C89, если хоть один операнд отрицательный, направление округления частного (к нулю или вниз) выбирала реализация, а знак остатка шёл за ним. C99 и C11 закрепили округление к нулю.

Часть 2. Каверзы и теория

Здесь код не пишут — объясняют. Эти вопросы отделяют тех, кто «знает синтаксис», от тех, кто понимает Си. Отвечай по существу и с примером.

Каверза 1. Что такое UB и где оно прячется

Неопределённое поведение (undefined behavior, UB) — это ситуации, для которых стандарт Си **не ставит никаких требований** (так и сказано в определении, C11 3.4.3). Компилятор вправе делать что угодно: выдать ожидаемый результат, выдать мусор, уронить программу или (самое коварное) «оптимизировать» твой код, считая, что UB невозможно. Программа с UB не «иногда работает» — она **некорректна**, даже если сегодня выводит правильное.

Запомни главное правило: **на UB полагаться нельзя**. Поведение может меняться от компилятора, флагов оптимизации, версии и даже соседних строк кода. Частые источники:

```c
/* 1. Переполнение знакового int — UB */
int x = INT_MAX;
x = x + 1;              /* НЕ «станет INT_MIN», а UB */

/* 2. Разыменование NULL */
int *p = NULL;
int y = *p;             /* падение или хуже */

/* 3. Выход за границы массива */
int a[3];
a[3] = 5;               /* записали мимо — UB */

/* 4. Чтение неинициализированной локальной переменной */
int z;
printf("%d", z);       /* значение не определено — UB */
```

```c
/* 5. Изменение строкового литерала */
char *s = "hello";
s[0] = 'H';            /* UB; на macOS — падение, см. Приложение Б */

/* 6. Использование памяти после free */
int *q = malloc(sizeof *q);
free(q);
*q = 7;                /* «use after free» — UB */
```

Про пункт 4 есть тонкость для самых въедливых интервьюеров: стандарт (C11, 6.3.2.1) называет UB чтение неинициализированной локальной переменной, у которой ни разу не брали адрес, — ровно наш `z`. Если адрес брали, значение просто «неопределённое», но полагаться на него всё равно нельзя.

Почему переполнение **знакового** — это UB, а беззнакового — нет? Для `unsigned` стандарт прямо говорит: считаем по модулю 2^n, где n — число битов типа (заворачивается). А для знаковых C11 разрешал три разных представления отрицательных чисел (дополнительный код, обратный код и «знак + модуль»), и результат переполнения на них разный, — поэтому его оставили неопределённым. В C23 дополнительный код стал обязательным, но переполнение так и осталось UB: компиляторы слишком активно этим пользуются для оптимизаций. Например, оптимизатор вправе считать выражение `x + 1 > x` заведомо истинным: раз переполнения «не бывает», сравнение всегда верно. Ровно поэтому самодельные проверки вида «сложу, а потом посмотрю, не стало ли меньше» не работают — компилятор их вырезает.

Вот это надо увидеть. Программа:

```c
#include <stdio.h>
#include <limits.h>

int bigger(int x)
{
    return x + 1 > x;    /* «x + 1 больше x?» */
}

int main(void)
{
    volatile int v = INT_MAX;  /* volatile: не даём подставить число */
    printf("%d\n", bigger(v));
    return 0;
}
```

`volatile` (подробно — в каверзе 3) здесь только для того, чтобы компилятор не вычислил всё заранее, ещё при сборке. Запуск:

```
-O0:  0
-O2:  1
```

Без оптимизации процессор честно сложил, `INT_MAX + 1` завернулось в отрицательное число, и сравнение дало 0. С `-O2` clang заменил всю функцию на `return 1;` — я посмотрел ассемблер, там ровно одна команда `movl $1, %eax`. Один и тот же код, два разных ответа.

Спорить, какой из них «правильный», бесполезно: правильного нет. Зато есть инструмент, который скажет правду. Соберём ту же программу с `-fsanitize=undefined`:

```
ovf2.c:6:14: runtime error: signed integer overflow: 2147483647 + 1
cannot be represented in type 'int'
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior ovf2.c:6:14
```

Санитайзер не гадает «что напечатается» — он прямо говорит: здесь UB. Это и есть правильный ответ на такой вопрос на собеседовании.

Лечение: не выходить за границы, инициализировать переменные, проверять указатели на `NULL`, для изменяемых строк заводить `char s[] = "hello";` (массив, а не указатель на литерал), не трогать память после `free`. Под рукой держи санитайзеры: `cc -fsanitize=address,undefined` ловит многие UB на лету — и именно так проверены все программы этого приложения.

И запомни разницу в терминах, за неё цепляются:

- **undefined behavior** — стандарт не обещает ничего: переполнение знакового, выход за границы, use-after-free. Ответа нет.
- **unspecified behavior** — стандарт даёт несколько вариантов и не говорит, какой выберут: порядок вычисления аргументов функции. Программа корректна, но результат может отличаться.
- **implementation-defined** — вариант тоже свой у каждой платформы, но компилятор обязан его задокументировать: размер `int`, знаковость `char`, сдвиг вправо отрицательного числа. Ответ есть — он просто не один на всех.

Каверза 2. Strict aliasing — почему `int *` и `float *` нельзя путать

Отдельный, неочевидный вид UB. Правило **strict aliasing** («строгое правило псевдонимов», C11 6.5, пункт 7) гласит: к объекту можно обращаться только через его собственный тип (с точностью до `signed`/ `unsigned` и `const`) или через символьный тип. Прочитать или записать `float`

через `int *` — UB. Поэтому компилятор считает, что `int *` и `float *` никогда не указывают на одно место, и оптимизирует, опираясь на это.

```
float f = 1.0f;
int *p = (int *)&f;    /* смотрим на float как на int */
*p = 0;                /* нарушение strict aliasing — UB */
```

Так часто пытаются «достать биты» числа. Что из этого выходит, видно на маленькой программе:

```
#include <stdio.h>

int f(int *ip, float *fp)
{
    *ip = 1;
    *fp = 0.0f;      /* компилятор считает: ip и fp — разная память */
    return *ip;
}

int main(void)
{
    int x;
    printf("%d\n", f(&x, (float *)&x));
    return 0;
}
```

Оба указателя смотрят на один и тот же `x`. Запись `0.0f` кладёт туда четыре нулевых байта, значит, `*ip` после неё должен быть 0. Запуск:

```
-O0:  0
-O2:  1
```

С `-O2` clang рассудил: «`fp` указывает на `float`, `ip` — на `int`, по правилу они не пересекаются, значит, `*ip` всё ещё 1», — и вернул единицу, не перечитывая память. Ни одного предупреждения при этом не было, даже с `-Wall -Wextra`.

Правильный способ посмотреть на биты — `memcpy` (или `union`, это C11 тоже разрешает):

```
int bits;
memcpy(&bits, &f, sizeof bits);   /* законно: memcpy работает с байтами */
```

Размеры тут должны совпадать: на этом маке и `int`, и `float` — по 4 байта. Надёжнее взять `uint32_t` из `<stdint.h>`, у него размер зафиксирован.

Исключение из правила — символьные типы (`char`, `signed char`, `unsigned char`): через них **можно** смотреть на байты чего угодно. Обобщённый стек из задачи 3 ни разу не обращается к элементу через чужой тип: он двигает байты `memcpy` и шагает по буферу через `char *` — поэтому с ним всё законно.

Каверза 3. `volatile` — зачем он нужен

`volatile` говорит компилятору: «эта переменная может измениться **в любой момент** способом, которого ты не видишь, — не кэшируй её, не выкидывай обращения, читай каждый раз заново».

Где это критично:

```
volatile int *reg = (volatile int *)0x40000000;  /* аппаратный регистр */
while (*reg == 0) { }   /* ждём, пока железо изменит регистр */
```

Без `volatile` компилятор видит, что внутри цикла `*reg` никто не меняет, и вправе прочитать его один раз. Я проверил на обычной глобальной переменной `int flag;` и цикле `while (flag == 0) { }`: clang с `-O2` выкинул цикл **целиком** — функция сразу возвращается. Стандарт это разрешает (C11 6.8.5, пункт 6): цикл без ввода-вывода, без `volatile` и без атомарных операций компилятор вправе считать завершающимся. С `volatile int flag` в ассемблере честный цикл: чтение памяти, сравнение с нулём, переход назад. Ровно это `volatile` и обещает: каждое обращение в коде — настоящее обращение к памяти.

Второй классический случай — переменная, меняемая в обработчике сигнала:

```
volatile sig_atomic_t stop = 0;   /* флаг «пора остановиться» */
```

Чего `volatile` **не** делает: он **не** обеспечивает потокобезопасность и атомарность. Для многопоточности нужны `_Atomic` / мьютексы, а не `volatile`. Это частая путаница на собеседовании.

Каверза 4. Две роли `static`

Слово `static` в Си означает **разные вещи** в зависимости от места.

1. Внутри функции — переменная получает **статический срок жизни**: она существует всё время работы программы (а не создаётся заново при каждом вызове) и сохраняет значение между вызовами:

```
int next_id(void)
{
    static int id = 0;   /* создаётся один раз, помнит значение */
    return ++id;
```

```
}
/* вызовы вернут 1, 2, 3, ... */
```

Три вызова подряд в `main` печатают `1 2 3` — проверено. Инициализатор `= 0` срабатывает **один раз**, причём ещё до запуска `main`, а не при первом вызове функции (C11 6.2.4, пункт 3). Поэтому он обязан быть константой: `static int id = f();` в Си не соберётся. Память — не на стеке, а в статической области.

2. На уровне файла (у глобальной переменной или функции) — `static` ограничивает видимость **этим файлом** (внутренняя компоновка). Из других `.c`-файлов к ней не достучаться:

```
static int counter;          /* виден только в этом файле */
static void helper(void) { } /* приватная функция модуля */
```

Это способ скрыть детали реализации — аналог `private`. Без `static` глобальные имена видны всей программе, и легко получить конфликт имён при сборке из нескольких файлов.

Каверза 5. Stack vs heap и причины segfault

Стандарт Си слов «стек» и «куча» не знает — он говорит о сроках жизни объектов: автоматическом (локальные переменные), статическом, выделенном (`malloc`). Но на всех обычных платформах это устроено так, и на собесе говорят именно этими словами. Новичку важны две области:

- **Стек** (stack) — здесь живут локальные переменные и кадры вызовов функций. Выделяется и освобождается **автоматически**, быстро, но размер ограничен (обычно несколько МБ). Бесконечная рекурсия его переполняет — **stack overflow** (см. главу 9).
- **Куча** (heap) — память от `malloc`/`calloc`/`realloc`. Большая, живёт, пока сам не вызовешь `free`. За ней следишь **ты**: забыл `free` — утечка; освободил дважды — UB.

```
int arr[1000000];     /* на стеке — рискуешь переполнить стек */
int *arr = malloc(1000000 * sizeof(int));  /* на куче — надёжнее */
```

Цифры не абстрактные. На этой машине лимит стека главного потока — 8192 КБ (проверяется командой `ulimit -s`), то есть 8 388 608 байт. Я проверил программу, которая первой строкой `main` печатает «вошли в main», потом заполняет локальный массив и печатает его последний элемент:

```
int arr[1000000]   4 000 000 байт    работает
int arr[2000000]   8 000 000 байт    работает (впритык)
int arr[4000000]  16 000 000 байт    падает, код выхода 139
```

139 — это 128 плюс сигнал 11, `SIGSEGV`. Причём на 4 миллионах программа не успела напечатать даже «вошли в main»: падает она не на обращении к элементу, а на самом входе в функцию, когда резервируется место под локальные переменные. Отсюда правило: всё, что больше десятков килобайт, — на куче.

Segmentation fault («сегфолт») — аварийное завершение при обращении к памяти, которая тебе не принадлежит. Типичные причины:

- разыменование `NULL` или «висячего» указателя (после `free`);
- выход далеко за границы массива;
- запись в строковый литерал (на macOS это приходит как `SIGBUS`, код 138, — см. Приложение Б, каверза 1);
- переполнение стека глубокой рекурсией.

Сегфолт — это ещё везение: программа хотя бы падает сразу. Хуже, когда UB тихо портит данные.

Каверза 6. Выравнивание и padding: что выведет `sizeof`

```c
struct A {
    char  c;    /* 1 байт */
    int   i;    /* 4 байта */
    char  d;    /* 1 байт */
};
printf("%zu\n", sizeof(struct A));   /* НЕ 6, а обычно 12 */
```

Проверим на этой машине. Заодно попросим компилятор показать смещения полей — их даёт макрос `offsetof` из `<stddef.h>`:

```c
printf("sizeof(struct A) = %zu\n", sizeof(struct A));
printf("offsetof c=%zu i=%zu d=%zu\n",
       offsetof(struct A, c),
       offsetof(struct A, i),
       offsetof(struct A, d));
printf("_Alignof(int) = %zu\n", _Alignof(int));
```

```
sizeof(struct A) = 12
offsetof c=0 i=4 d=8
_Alignof(int) = 4
```

Почему не `1 + 4 + 1 = 6`? Процессор читает данные эффективнее, когда они лежат по «круглым» адресам: `int` (4 байта) должен начинаться с адреса, кратного 4. Это **выравнивание** (alignment). Чтобы его соблюсти, компилятор вставляет невидимые **байты-заполнители** (padding):

```
[ c ][pad][pad][pad][  i (4)  ][ d ][pad][pad][pad]
  0    1    2    3    4 5 6 7    8    9   10  11
```

- после `c` (адрес 0) — 3 байта padding, чтобы `i` встал на адрес 4;
- `d` на адресе 8;
- в конце ещё 3 байта, чтобы **весь размер** структуры был кратен выравниванию самого «требовательного» поля (`int`, 4). Это нужно, чтобы в массиве структур каждая следующая тоже была выровнена.

Итого 12. А если переставить поля по убыванию размера — padding почти исчезнет:

```c
struct B {
    int   i;    /* 4 */
    char  c;    /* 1 */
    char  d;    /* 1 */
};              /* sizeof == 8: i, c, d, и 2 байта хвоста */
```

И это тоже проверено запуском:

```
sizeof(struct B) = 8
```

Правило: **группируй поля от больших к меньшим** — структура займёт меньше. Здесь мы сэкономили треть размера, просто переставив поля; на массиве из миллиона структур это 4 МБ.

Важная оговорка: числа 12 и 8 — не гарантия стандарта, а implementation-defined поведение. Стандарт (C11 6.7.2.1) лишь требует, чтобы поля шли в памяти в порядке объявления, чтобы перед первым полем padding не было (его смещение всегда 0) и чтобы выравнивание соблюдалось; сколько именно padding вставить между полями и в конце, решает ABI платформы — свод правил, по которым компиляторы одной системы раскладывают данные. Узнать выравнивание типа можно через `_Alignof(int)` (здесь 4), а смещение поля — через `offsetof(struct A, i)` из `<stddef.h>`. Это и есть правильный ответ на собесе: «на x86-64 будет 12, а вообще — считается по правилам выравнивания, вот они».

Что спросят дальше. «Как убрать padding?» Нестандартными расширениями: `#pragma pack(1)` или `__attribute__((packed))`. Но платить придётся скоростью — процессор читает невыровненные данные медленнее, а на некоторых архитектурах это вообще падение. Упаковку применяют, когда структура ложится в бинарный формат файла или сетевого пакета, где раскладка задана извне.

Каверза 7. Опасный макрос `#define SQR(x) x*x`

```
#define SQR(x) x*x
```

Препроцессор делает **текстовую подстановку**, не понимая арифметики. Отсюда два бага.

Баг 1 — приоритет операций. `SQR(a + b)` развернётся в `a + b*a + b`, то есть `a + (b*a) + b`, а вовсе не $(a+b)^2$:

```
int r = SQR(a + b);   /* a + b*a + b — НЕ то, что хотели */
```

Подставим числа: при `a = 2`, `b = 3` мы ждём $(2+3)^2 = 25$, а получаем `2 + 3*2 + 3 = 11`. Проверка запуском:

```
SQR(a+b) = 11, a + b*a + b = 11
```

Ошибка на 14 — и ни одного предупреждения от компилятора: с точки зрения языка выражение совершенно законно.

Лечится скобками вокруг **каждого аргумента и всего выражения**:

```
#define SQR(x) ((x) * (x))
```

Теперь `SQR(a + b)` → `((a + b) * (a + b))` — верно.

Баг 2 — двойное вычисление аргумента. Даже с правильными скобками `SQR(x)` подставляет `x` **дважды**. Если аргумент имеет побочный эффект, он сработает два раза:

```
int i = 3;
int r = SQR(i++);     /* ((i++) * (i++)) — i++ два раза */
```

Это вдобавок UB: `i` меняется дважды без упорядочивания (см. каверзу 9), поэтому даже «вырастет дважды» обещать нельзя. clang с `-Wall` замечает и это, сквозь макрос: `multiple unsequenced modifications to 'i' [-Wunsequenced]`. От этого скобки не спасают — лекарство только одно: **не передавай в макросы выражения с побочными эффектами**, а лучше используй `static inline`-функцию:

```
static inline int sqr(int x) { return x * x; }
```

Функция вычислит аргумент один раз и уважает типы. В современном Си макрос для такого уже почти не нужен.

Каверза 8. Идиома `do { ... } while(0)` в макросах

Как написать макрос из нескольких инструкций, чтобы он вёл себя как одна? Наивно:

```c
#define SWAP(a, b) { int t = a; a = b; b = t; }
```

Сломается в `if`-без-скобок из-за лишней `;`:

```c
if (x) SWAP(p, q); else foo();   /* ; после } разрывает if/else — ошибка */
```

Это не теория — компилятор действительно отказывается это собирать:

```
error: expected expression
    if (x) SWAP(p, q); else foo();
                       ^
```

Макрос развернулся в блок `{ ... }`, а точка с запятой после него стала **отдельным пустым оператором**. Получилось, что у `if` уже два оператора, и `else` повис в воздухе.

Канонический приём — обернуть тело в `do { ... } while(0)`:

```c
#define SWAP(a, b) do { int t = (a); (a) = (b); (b) = t; } while (0)
```

Теперь `SWAP(p, q);` — это один корректный оператор с точкой с запятой на конце, и он спокойно живёт в `if/else`. `while(0)` гарантирует ровно один проход, а компилятор с оптимизацией выкинет фиктивный цикл. Это стандартная идиома многострочных макросов в Си.

Каверза 9. Порядок вычисления аргументов не определён

```c
int i = 1;
printf("%d %d\n", i++, i++);   /* что напечатается? */
```

Ответ: **неизвестно**, и «правильного числа» тут не существует. Тут наложились два разных сорта неопределённости, и на собесе полезно различать их.

Первый: порядок вычисления аргументов функции стандарт не задаёт вовсе — слева направо, справа налево или вперемешку. Это **unspecified behavior**: сам по себе он программу не ломает, просто результат может отличаться от компилятора к компилятору.

Второй, куда серьёзнее: два изменения одной переменной, не упорядоченные между собой, — это **undefined behavior** (C11 6.5, пункт 2). Вычисления аргументов функции друг относительно друга не упорядочены, так что два `i++` в одном вызове — ровно этот случай. В старых стандартах то же правило формулировали через «точки следования». Программа некорректна целиком, и обсуждать её вывод бессмысленно.

Компилятор это видит и говорит прямо:

```
warning: multiple unsequenced modifications to 'i' [-Wunsequenced]
    printf("%d %d\n", i++, i++);
                      ^    ~~
```

Если всё-таки запустить, clang на этой машине печатает `1 2` — и с `-O0`, и с `-O2`. Но это не ответ: другой компилятор имеет полное право выдать `2 1`, `2 2` или что угодно ещё. Правильный ответ на собеседовании — «так писать нельзя, это UB», и в подтверждение — `-Wunsequenced`.

То же относится к выражениям вроде:

```
a[i] = i++;          /* UB: и читаем, и меняем i без упорядочивания */
foo(i++, i++);       /* UB */
x = f() + g();       /* порядок вызовов f и g не задан */
```

Последняя строка — не UB: сами вызовы не перемешиваются, просто неизвестно, какой будет первым (unspecified). Если `f()` и `g()` имеют побочные эффекты и зависят от порядка — результат непредсказуем. Вывод: **не меняй одну переменную несколько раз в одном выражении** и не полагайся на порядок вычисления аргументов. Разнеси по разным строкам.

Каверза 10. Endianness — порядок байтов

Многобайтовое число (`int` — 4 байта) можно хранить в памяти двумя способами:

- **little-endian** — младший байт первым (x86; ARM умеет оба порядка, но почти всегда работает в этом; значит, и большинство ПК и телефонов);
- **big-endian** — старший байт первым (некоторые сетевые протоколы, старые процессоры; «сетевой порядок» — big-endian).

Проверить порядок на своей машине можно так:

```
int x = 1;
unsigned char *p = (unsigned char *)&x;   /* смотрим на байты числа */
if (p[0] == 1) {
    printf("little-endian\n");   /* младший байт (1) лежит первым */
} else {
    printf("big-endian\n");
}
```

Число `1` в памяти это байты `01 00 00 00` (little) или `00 00 00 01` (big). Читаем первый байт через `unsigned char *` — это **разрешено** strict aliasing (см. каверзу 2). Если там `1` — машина little-endian.

На этой машине (x86-64) программа, которая заодно печатает все четыре байта, выдаёт:

```
little-endian
байты числа 1: 01 00 00 00
```

Единица лежит первой — младший байт впереди. Это и есть little-endian. Тот же код на big-endian-машине напечатал бы `00 00 00 01`, и никакой «правильный» ответ тут не существует: порядок байтов — свойство платформы, а не языка.

Когда это важно: чтение бинарных файлов и сетевых пакетов, обмен данными между разными машинами. Поэтому в сети числа гоняют в фиксированном порядке через `htons`/`htonl` (host-to-network) и обратно `ntohs`/`ntohl`.

Ошибки, на которых сыплются в этих задачах

Всё перечисленное — из настоящих решений, а не выдумано для примера.

- **Опорный элемент = последний.** На уже отсортированном входе quicksort скатывается в $O(n^2)$, а на 200 000 элементов переполняет стек (мы это видели). Случайный опорный или медиана трёх обязательны, если вход может быть отсортирован; от массива одинаковых чисел спасает только другое разбиение (Хоара или трёхпутевое).
- **`(lo + hi) / 2` вместо `lo + (hi - lo) / 2`.** Переполнение знакового `int` — UB, а не «отрицательный индекс».
- **Арифметика на `void *`.** Стандартом не определена; для побайтового хода бери `char *` (он же единственный тип, которым законно смотреть на чужие байты — см. каверзу 2).
- **`realloc` прямо в поле структуры.** `s->data = realloc(s->data, …)` теряет старый блок при неудаче. Клади результат во временную переменную и проверяй.
- **`long` вместо `long long` в защите от переполнения.** На платформах, где `long` — 4 байта, «защита» сама переполняется.
- **`char` без приведения к `unsigned char` в хеш-функции.** Байты с кодом больше 127 становятся отрицательными там, где `char` знаковый, и одна и та же строка получает разный хеш на разных платформах.
- **Хранение чужого указателя как ключа.** Хеш-таблица обязана копировать ключ себе (`malloc` + `strcpy`), иначе он протухнет.
- **Два индекса `head`/`tail` в кольцевом буфере без счётчика.** Состояния «пусто» и «полно» становятся неразличимы.
- **Односвязный список в LRU.** Удаление узла из середины превращается в O(n), и обещанное O(1) исчезает.
- **Фиксированное число корзин.** Хеш-таблица на 16 корзин без рехеширования — это O(n) на поиск, как только записей становится много. Обещать «O(1)» можно, только пока коэффициент загрузки ограничен.
- **`free` общего хвоста дважды** при пересечении списков — double free.

- **Макрос без скобок вокруг аргументов** и с побочными эффектами внутри.
- **`volatile` вместо синхронизации.** Атомарности он не даёт.

Тренировочные задачи

1. Перепиши `partition` из задачи 1 по схеме Хоара (два бегунка навстречу) и добавь выбор опорного «медиана трёх». Проверь на уже отсортированном массиве из 100 000 элементов, что глубина рекурсии не взлетела.
2. Сделай quicksort нерекурсивной: заведи явный стек пар `(lo, hi)` и всегда клади в него **бóльшую** половину, а меньшую обрабатывай циклом. Так глубина стека остаётся O(log n) даже в худшем случае.
3. Добавь в обобщённый стек из задачи 3 функцию `void stack_foreach(const Stack *s, void (*fn)(const void *))` — печать элементов через переданный обработчик. Проверь на `int` и на `double`.
4. Обобщи `my_itoa` на любое основание от 2 до 16: `void my_itoa_base(int n, int base, char *buf)`. Не забудь про `INT_MIN` и про буквы `a–f`.
5. Допиши в хеш-таблицу из задачи 7 рехеширование: когда число записей превысит число корзин вдвое, удвой массив корзин и перераспредели записи. Убедись под `-fsanitize=address`, что ничего не потерялось и не утекло.
6. Сделай кольцевой буфер динамическим: вместо `int data[CAP]` — буфер от `malloc` с функцией `ring_grow`. Аккуратно перенеси элементы — при завороте они лежат в массиве двумя кусками.
7. Возьми `struct A` из каверзы 6 и напиши программу, которая печатает `sizeof` и `offsetof` для всех полей. Переставляй поля и смотри, как меняется размер. Найди перестановку с минимальным padding.

Ответы к тренировочным задачам

Все решения собраны с `-std=c11 -Wall -Wextra -O2` без предупреждений и прогнаны под `-fsanitize=address,undefined`; те, где есть `malloc`, ещё и через `leaks` — чисто. Выводы ниже — реальные.

Задача 1. Хоар и медиана трёх:

```c
#include <stdio.h>
#include <stdlib.h>

static int depth, max_depth;    /* для замера глубины рекурсии */
```

```c
static void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

/* Медиана трёх: после неё a[lo] <= a[mid] <= a[hi]. */
static int median_of_three(int a[], int lo, int hi)
{
    int mid = lo + (hi - lo) / 2;
    if (a[mid] < a[lo]) swap(&a[mid], &a[lo]);
    if (a[hi] < a[lo]) swap(&a[hi], &a[lo]);
    if (a[hi] < a[mid]) swap(&a[hi], &a[mid]);
    return a[mid];
}

/* Разбиение Хоара: после него a[lo..j] <= pivot <= a[j+1..hi]. */
static int partition_hoare(int a[], int lo, int hi)
{
    int pivot = median_of_three(a, lo, hi);
    int i = lo - 1;
    int j = hi + 1;
    for (;;) {
        do { i++; } while (a[i] < pivot);   /* слева ищем >= pivot */
        do { j--; } while (a[j] > pivot);   /* справа ищем <= pivot */
        if (i >= j) {
            return j;
        }
        swap(&a[i], &a[j]);
    }
}

static void quicksort_hoare(int a[], int lo, int hi)
{
    if (lo >= hi) {
        return;
    }
```

```
    depth++;
    if (depth > max_depth) max_depth = depth;
    int p = partition_hoare(a, lo, hi);
    quicksort_hoare(a, lo, p);          /* p входит в левую часть */
    quicksort_hoare(a, p + 1, hi);
    depth--;
}

static void test(const char *name, int *a, int n)
{
    max_depth = 0;
    quicksort_hoare(a, 0, n - 1);
    for (int i = 1; i < n; i++) {
        if (a[i - 1] > a[i]) {
            printf("%s: НЕ отсортировано!\n", name);
            return;
        }
    }
    printf("%s: глубина %d\n", name, max_depth);
}

int main(void)
{
    int n = 100000;
    int *a = malloc((size_t)n * sizeof *a);
    if (a == NULL) {
        return 1;
    }
    for (int i = 0; i < n; i++) a[i] = i;
    test("отсортирован", a, n);
    for (int i = 0; i < n; i++) a[i] = n - i;
    test("по убыванию", a, n);
    for (int i = 0; i < n; i++) a[i] = 7;
    test("одни семёрки", a, n);
    srand(1);
    for (int i = 0; i < n; i++) a[i] = rand();
    test("случайный", a, n);
    free(a);
```

```
    return 0;
}
```

```
отсортирован: глубина 17
по убыванию: глубина 17
одни семёрки: глубина 17
случайный: глубина 32
```

Вместо 100 000 уровней (и падения) — 17: это $\lceil \log_2 100\,000 \rceil$, идеальное деление пополам. Медиана трёх на отсортированном входе всегда выбирает настоящую середину. Одни семёрки тоже делятся пополам: бегунки Хоара останавливаются на **равных** опорному (`<` и `>` в циклах `do`) и меняют их местами, так что встречаются посередине. Обрати внимание на два отличия от Ломуто: опорный не встаёт на своё место, поэтому левая часть — `lo..p` **включая** `p`; и циклы `do { } while` не выбегают за границы — после медианы трёх `a[lo] <= pivot <= a[hi]`, и эти элементы остановят бегунков.

Задача 2. Quicksort без рекурсии:

```c
#include <stdio.h>
#include <stdlib.h>

static void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

static int partition(int a[], int lo, int hi)   /* Ломуто, задача 1 */
{
    int pivot = a[hi];
    int i = lo - 1;
    for (int j = lo; j < hi; j++) {
        if (a[j] < pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[hi]);
    return i + 1;
```

```c
}

/* Без рекурсии: бо́льшую часть — в стек, меньшую — сразу дальше. */
static int quicksort_iter(int a[], int n)
{
    int stack[64][2];           /* 64 пар хватит: глубина <= log2(n) */
    int top = 0, max_top = 0;
    stack[top][0] = 0;
    stack[top][1] = n - 1;
    top++;
    while (top > 0) {
        top--;
        int lo = stack[top][0];
        int hi = stack[top][1];
        while (lo < hi) {
            int p = partition(a, lo, hi);
            if (p - lo < hi - p) {          /* левая часть меньше */
                stack[top][0] = p + 1;      /* правую — в стек */
                stack[top][1] = hi;
                hi = p - 1;                 /* левую — продолжаем */
            } else {
                stack[top][0] = lo;         /* левую — в стек */
                stack[top][1] = p - 1;
                lo = p + 1;                 /* правую — продолжаем */
            }
            top++;
            if (top > max_top) max_top = top;
        }
    }
    return max_top;
}

int main(void)
{
    int n = 100000;
    int *a = malloc((size_t)n * sizeof *a);
    if (a == NULL) {
        return 1;
```

```
    }
    for (int i = 0; i < n; i++) a[i] = i;
    printf("отсортирован: стек %d\n", quicksort_iter(a, n));
    srand(1);
    for (int i = 0; i < n; i++) a[i] = rand();
    printf("случайный: стек %d\n", quicksort_iter(a, n));
    for (int i = 1; i < n; i++) {
        if (a[i - 1] > a[i]) {
            printf("НЕ отсортировано\n");
        }
    }
    free(a);
    return 0;
}
```

```
отсортирован: стек 1
случайный: стек 10
```

Почему глубина стека не больше $\log_2 n$: мы всегда продолжаем с **меньшей** частью, а она не больше половины текущего отрезка. Каждая лежащая в стеке пара «сторожит» отрезок, который начался в предыдущем, вдвое большем, — вдвое делить 100 000 можно не больше 17 раз. Поэтому 64 пар хватит на любой `int`-массив. Время на отсортированном входе осталось $O(n^2)$ — у меня 2.95 с на 100 000 (со старым разбиением Ломуто иначе и быть не может), зато программа больше не падает.

Задача 3. Добавляем к стеку из задачи 3 (его функции без изменений):

```
/* Обойти стек от дна к вершине, вызывая fn для каждого элемента. */
void stack_foreach(const Stack *s, void (*fn)(const void *))
{
    for (size_t i = 0; i < s->count; i++) {
        fn(s->data + i * s->elem_size);   /* адрес i-го элемента */
    }
}

static void print_int(const void *p)
{
    printf("%d ", *(const int *)p);
}
```

```c
static void print_double(const void *p)
{
    printf("%.2f ", *(const double *)p);
}

int main(void)
{
    Stack si, sd;
    if (!stack_init(&si, sizeof(int))) {
        return 1;
    }
    if (!stack_init(&sd, sizeof(double))) {
        stack_free(&si);
        return 1;
    }
    for (int i = 1; i <= 6; i++) {
        double d = i / 2.0;
        if (!stack_push(&si, &i) || !stack_push(&sd, &d)) {
            printf("Не хватило памяти\n");
            break;
        }
    }
    stack_foreach(&si, print_int);
    printf("\n");
    stack_foreach(&sd, print_double);
    printf("\n");
    stack_free(&si);
    stack_free(&sd);
    return 0;
}
```

```
1 2 3 4 5 6
0.50 1.00 1.50 2.00 2.50 3.00
```

`stack_foreach` не знает тип элемента — она передаёт обработчику **адрес** i-го элемента, а приводит его к нужному типу уже обработчик: `*(const int *)p`.

Задача 4.

```c
#include <stdio.h>
#include <limits.h>

/* int -> строка в системе счисления base (2..16).
   buf: не меньше 34 байт (знак + 32 двоичные цифры + '\0'). */
void my_itoa_base(int n, int base, char *buf)
{
    const char *digits = "0123456789abcdef";
    char tmp[33];
    int i = 0;
    int neg = 0;

    if (base < 2 || base > 16) {
        buf[0] = '\0';                  /* такое основание не умеем */
        return;
    }
    long long value = n;                /* -INT_MIN в int не влезает */
    if (value < 0) {
        neg = 1;
        value = -value;
    }
    do {
        tmp[i++] = digits[value % base];
        value /= base;
    } while (value > 0);

    int j = 0;
    if (neg) {
        buf[j++] = '-';
    }
    while (i > 0) {
        buf[j++] = tmp[--i];
    }
    buf[j] = '\0';
}

int main(void)
{
```

```
    char buf[34];
    int vals[] = {0, 255, -255, INT_MAX, INT_MIN};
    int bases[] = {2, 8, 10, 16};
    for (int v = 0; v < 5; v++) {
        for (int b = 0; b < 4; b++) {
            my_itoa_base(vals[v], bases[b], buf);
            printf("%s ", buf);
        }
        printf("\n");
    }
    return 0;
}
```

```
0 0 0 0
11111111 377 255 ff
-11111111 -377 -255 -ff
11111111111111111111111111111111 17777777777 2147483647 7fffffff
-10000000000000000000000000000000 -20000000000 -2147483648 -80000000
```

Самая длинная запись — `INT_MIN` в двоичной: знак и 32 цифры, плюс `'\0'` — 34 байта. Отрицательные числа здесь печатаются со знаком минус, как математическая запись, а не как биты дополнительного кода.

Задача 5. Хеш-таблица с рехешированием:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Entry {
    char *key;
    int count;
    struct Entry *next;
} Entry;

typedef struct {
    Entry **buckets;
    size_t nbuckets;    /* число корзин, степень двойки */
    size_t size;        /* число записей */
} HashTable;
```

```
unsigned long hash_str(const char *s)        /* djb2, как в задаче 7 */
{
    unsigned long h = 5381;
    int c;
    while ((c = (unsigned char)*s++) != 0) {
        h = h * 33u + (unsigned long)c;
    }
    return h;
}

static Entry **alloc_buckets(size_t n)
{
    Entry **b = malloc(n * sizeof *b);
    if (b != NULL) {
        for (size_t i = 0; i < n; i++) {
            b[i] = NULL;
        }
    }
    return b;
}

int table_init(HashTable *t)
{
    t->nbuckets = 4;
    t->size = 0;
    t->buckets = alloc_buckets(t->nbuckets);
    return t->buckets != NULL;
}

/* Удвоить корзины и переложить записи. Узлы не копируем —
   только перецепляем. 0 — нет памяти (таблица остаётся старой). */
static int rehash(HashTable *t)
{
    size_t nn = t->nbuckets * 2;
    Entry **nb = alloc_buckets(nn);
    if (nb == NULL) {
        return 0;
```

```
    }
    for (size_t i = 0; i < t->nbuckets; i++) {
        Entry *e = t->buckets[i];
        while (e != NULL) {
            Entry *next = e->next;
            size_t idx = hash_str(e->key) % nn;
            e->next = nb[idx];
            nb[idx] = e;
            e = next;
        }
    }
    free(t->buckets);
    t->buckets = nb;
    t->nbuckets = nn;
    return 1;
}

int table_inc(HashTable *t, const char *key)
{
    size_t idx = hash_str(key) % t->nbuckets;
    for (Entry *e = t->buckets[idx]; e != NULL; e = e->next) {
        if (strcmp(e->key, key) == 0) {
            e->count++;
            return 1;
        }
    }
    if (t->size + 1 > 2 * t->nbuckets) {    /* α превысит 2 — растём */
        if (!rehash(t)) {
            return 0;
        }
        idx = hash_str(key) % t->nbuckets;  /* корзина поменялась */
    }
    Entry *e = malloc(sizeof(Entry));
    if (e == NULL) {
        return 0;
    }
    e->key = malloc(strlen(key) + 1);
    if (e->key == NULL) {
```

```
        free(e);
        return 0;
    }
    strcpy(e->key, key);
    e->count = 1;
    e->next = t->buckets[idx];
    t->buckets[idx] = e;
    t->size++;
    return 1;
}

int table_get(const HashTable *t, const char *key)
{
    size_t idx = hash_str(key) % t->nbuckets;
    for (Entry *e = t->buckets[idx]; e != NULL; e = e->next) {
        if (strcmp(e->key, key) == 0) {
            return e->count;
        }
    }
    return 0;
}

void table_free(HashTable *t)
{
    for (size_t i = 0; i < t->nbuckets; i++) {
        Entry *e = t->buckets[i];
        while (e != NULL) {
            Entry *next = e->next;
            free(e->key);
            free(e);
            e = next;
        }
    }
    free(t->buckets);
    t->buckets = NULL;
}

int main(void)
```

```
{
    HashTable t;
    if (!table_init(&t)) {
        return 1;
    }
    char word[16];
    for (int round = 0; round < 3; round++) {   /* 1000 слов по 3 раза */
        for (int i = 0; i < 1000; i++) {
            snprintf(word, sizeof word, "w%d", i);
            if (!table_inc(&t, word)) {
                printf("Не хватило памяти\n");
                table_free(&t);
                return 1;
            }
        }
    }
    int bad = 0;
    for (int i = 0; i < 1000; i++) {
        snprintf(word, sizeof word, "w%d", i);
        if (table_get(&t, word) != 3) {
            bad++;
        }
    }
    printf("записей %zu, корзин %zu, с ошибкой %d\n",
           t.size, t.nbuckets, bad);
    table_free(&t);
    return 0;
}
```

```
записей 1000, корзин 512, с ошибкой 0
```

Стартуем с 4 корзин; каждый раз, когда записей стало бы больше удвоенного числа корзин, число корзин удваивается: 4, 8, 16, … 512. При рехешировании записи не копируются — узлы просто перецепляются в новые корзины, поэтому `malloc` нужен один, на новый массив корзин. После рехеша индекс новой записи пересчитываем: корзин стало больше. Все 1000 слов нашлись с правильным счётчиком 3.

Задача 6. Растущее кольцо:

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    int cap;      /* сколько мест */
    int head;
    int count;
} Ring;

int ring_init(Ring *r, int cap)
{
    r->data = malloc((size_t)cap * sizeof *r->data);
    r->cap = cap;
    r->head = 0;
    r->count = 0;
    return r->data != NULL;
}

/* Вдвое больше места. Элементы переносим по порядку, от головы:
   при завороте они лежат двумя кусками, а в новом буфере — подряд. */
static int ring_grow(Ring *r)
{
    int ncap = r->cap * 2;
    int *nd = malloc((size_t)ncap * sizeof *nd);
    if (nd == NULL) {
        return 0;
    }
    for (int i = 0; i < r->count; i++) {
        nd[i] = r->data[(r->head + i) % r->cap];
    }
    free(r->data);
    r->data = nd;
    r->cap = ncap;
    r->head = 0;
    return 1;
}
```

```
int ring_push(Ring *r, int value)
{
    if (r->count == r->cap && !ring_grow(r)) {
        return 0;                       /* полон и расти некуда */
    }
    r->data[(r->head + r->count) % r->cap] = value;
    r->count++;
    return 1;
}

int ring_pop(Ring *r, int *out)
{
    if (r->count == 0) {
        return 0;
    }
    *out = r->data[r->head];
    r->head = (r->head + 1) % r->cap;
    r->count--;
    return 1;
}

int main(void)
{
    Ring r;
    int x;
    if (!ring_init(&r, 4)) {
        return 1;
    }
    for (int v = 1; v <= 4; v++) ring_push(&r, v);
    ring_pop(&r, &x);
    ring_pop(&r, &x);                   /* забрали 1 и 2 */
    for (int v = 5; v <= 9; v++) {      /* 5, 6 — заворот; 7 — рост */
        if (!ring_push(&r, v)) {
            printf("Не хватило памяти\n");
        }
    }
    printf("мест %d:", r.cap);
    while (ring_pop(&r, &x)) {
```

```
        printf(" %d", x);
    }
    printf("\n");
    free(r.data);
    return 0;
}
```

```
мест 8: 3 4 5 6 7 8 9
```

После двух `pop` голова стоит на индексе 2, `5` и `6` завернулись в индексы 0 и 1 — элементы лежат двумя кусками: `[5 6 3 4]`. На `7` кольцо полное, и `ring_grow` переносит элементы **в порядке очереди** (`(head + i) % cap`), так что в новом буфере они лежат подряд `[3 4 5 6 …]`, и `head` снова 0. Скопировать старый массив «как есть» было бы ошибкой: заворот сломал бы порядок.

Задача 7.

```
#include <stdio.h>
#include <stddef.h>

struct A1 { char c; int i; char d; };
struct A2 { char c; char d; int i; };
struct A3 { int i; char c; char d; };

int main(void)
{
    printf("c,i,d: sizeof %zu, c=%zu i=%zu d=%zu\n", sizeof(struct A1),
           offsetof(struct A1, c), offsetof(struct A1, i),
           offsetof(struct A1, d));
    printf("c,d,i: sizeof %zu, c=%zu d=%zu i=%zu\n", sizeof(struct A2),
           offsetof(struct A2, c), offsetof(struct A2, d),
           offsetof(struct A2, i));
    printf("i,c,d: sizeof %zu, i=%zu c=%zu d=%zu\n", sizeof(struct A3),
           offsetof(struct A3, i), offsetof(struct A3, c),
           offsetof(struct A3, d));
    return 0;
}
```

```
c,i,d: sizeof 12, c=0 i=4 d=8
c,d,i: sizeof 8, c=0 d=1 i=4
i,c,d: sizeof 8, i=0 c=4 d=5
```

Минимум — 8 байт, и его дают обе раскладки, где два `char` стоят рядом: 6 байт полезных данных плюс 2 байта padding (в первом случае между `d` и `i`, во втором — в хвосте). Меньше 8 не выйдет: размер обязан быть кратен выравниванию `int`, то есть 4, а 6 байт данных в 4 не влезают. Остальные три перестановки (`d,c,i`, `d,i,c`, `i,d,c`) зеркальны этим трём.

Итог практикума

Ты прошёл три сборника задач с собеседований — Junior, Middle и Senior — и это достойный финал всей книги. Пробежимся по тому, что осело.

На уровне **кода** ты теперь умеешь не просто «сортировать массив», а реализовать quicksort и merge sort с пониманием их сложности, худших случаев и выбора опорного. Ты собрал **обобщённые** структуры через `void *` и размер элемента, написал хеш-таблицу с разрешением коллизий цепочками, кольцевой буфер, нашёл пересечение списков за O(n+m) и набросал LRU-кэш на связке «двусвязный список + хеш». Это уже не учебные игрушки, а кирпичики, из которых сложены настоящие программы.

На уровне **понимания языка** ты разобрался с тем, что отделяет крепкого Си-разработчика от новичка: что такое UB и почему на него нельзя полагаться, чем опасны strict aliasing и переполнение знакового, зачем нужны `volatile` и две роли `static`, как устроены стек и куча и откуда берётся segfault, почему `sizeof` хитрой структуры больше суммы полей (выравнивание и padding), как не подорваться на макросе `SQR(x)` и зачем макросам идиома `do { } while(0)`, и почему порядок вычисления аргументов и endianness — вещи, о которых нельзя забывать.

Напутствие

Если ты дочитал до этих строк и хотя бы половину задач решил сам — поздравляю, ты уже не новичок. Ты прошёл путь от первого `printf` (глава 1) до LRU-кэша и тонкостей UB. Си не прощает небрежности, и именно поэтому он так многому учит: каждый указатель, каждый `malloc`, каждый байт выравнивания заставляют **понимать**, а не угадывать.

Что делать дальше:

- **пиши код каждый день** — алгоритмы забываются без практики; бери задачи, реализуй, ломай, чини;
- **читай чужой Си** — исходники SQLite, ядра Linux, Redis; там живут лучшие образцы того, как пишут на этом языке;
- **гоняй санитайзеры и предупреждения** — `-Wall -Wextra`, `-fsanitize=address,undefined`, valgrind: пусть инструменты ловят UB за тебя;
- **не бойся низкого уровня** — теперь ты понимаешь, что под капотом, и любой язык повыше будет казаться проще.

Спасибо, что прошёл эту книгу до конца. Главного ты уже добился — научился думать как программист, который **знает, что происходит в памяти**. Остальное — дело практики. Удачи на собеседованиях и в реальных проектах.